

SEARCH — does MILTANK choose better than MAG

Owens: `engine/miltank.js` , `engine/rollout_leaf.js` , the bring/lead search, the opponent model, the mega choice, post-KO replacement. Design notes in [MILTANK.md](#).

Its one number: the SPRT verdict against the named champion.

May not: fix an engine bug it trips over — file it in [ENGINE.md](#). Patching mechanics mid-run silently invalidates the run, and the run still prints a result.

<!-- GENERATED: engine/status.js -->

SEARCH — does MILTANK choose better than MAG

R1 leaf accuracy: QUARANTINED — the figure is withheld, not annotated.

`data/rollout-r1-explore1.json` is downstream of MEDICHAM: `engine/rollout_r1_artifact.js` reads `rollout-r1-artifact.jsonl` — MEDICHAM is not correct — 2 of 9 gate clauses fail (whole-game differential / BOARD-MATERIAL — games won) — it becomes quotable again when the gate opens AND this is re-run: `node engine/rollout_r1_artifact.js`

R2 leaf cost: QUARANTINED — the figure is withheld, not annotated.

`data/rollout-cost.json` is downstream of MEDICHAM: its generator `engine/rollout_r2.js` is in the play layer — MEDICHAM is not correct — 2 of 9 gate clauses fail (whole-game differential / BOARD-MATERIAL — games won) — it becomes quotable again when the gate opens AND this is re-run: `node engine/rollout_r2.js`

R3 divergence: QUARANTINED — the figure is withheld, not annotated.

`data/rollout-r3.json` is downstream of MEDICHAM: its generator `engine/rollout_r3.js` is in the play layer — MEDICHAM is not correct — 2 of 9 gate clauses fail (whole-game differential / BOARD-MATERIAL — games won) — it becomes quotable again when the gate opens AND this is re-run: `node engine/rollout_r3.js`

R4 does it win: QUARANTINED — the figure is withheld, not annotated.

`data/rollout-r4.json` is downstream of MEDICHAM: `engine/rollout_r4.js` reads `games.r4-decided.jsonl` — a MEDICHAM is not correct — 2 of 9 gate clauses fail (whole-game differential / BOARD-MATERIAL — games won) — it becomes quotable again when the gate opens AND this is re-run: `node engine/rollout_r4.js`

runs vs engine (newest engine source: `engine/medicham2-browser.js` 2026-09-06 00:08):

PRE-CHANGE `games.r4c-shipped2.jsonl` 2026-08-14 22:28

PRE-CHANGE `games.r4c-shipped.jsonl` 2026-08-14 17:21

PRE-CHANGE `games.r4b-search.jsonl` 2026-08-14 13:02

PRE-CHANGE `games.r4a-switching.jsonl` 2026-08-14 04:34

PRE-CHANGE `games.r4-decided.jsonl` 2026-08-04 00:41

stamped 2026-09-06 01:28

<!-- /GENERATED -->

R20 — THE SKY EXPIRES AND THE DEAD ARE COUNTED. #276 AND #283 CLOSED; THE GATE'S THIRD CLAUSE GOES 4 → 2. 2026-08-17.

ROADMAP #276 and #283 closed; #286 and #287 opened by what the work found. Gate: `tests/test-board-clock-power.js` , 46 assertions, shown **RED on four separate deliberate breaks**. Artifact: `data/feature-shift.json` (`engine/feature_shift.js`). **No run was launched, no refit was run, and**

#275 is still unmeasured.

The verdict in one line

Both rows are LIVE rather than stale, both are closed, and the fit-invalidation is now a NAMED LIST instead of a warning: 32 of 58 features and 6 of 18 joint features move, on 1.576% of 51,399 candidate vectors across 41.67% of 300 fit games. And the instrument everyone has been quoting for this — the 58 frozen fixture hashes — CANNOT SEE EITHER ROW, which is measured here rather than assumed.

WHY THEY WERE TAKEN, BECAUSE IT REVERSES AN EARLIER READING

Will's ruling is that the MAG refit waits for MEDICHAM — "i dont want to constantly come back to medicham" — and R17, R18 and R19 each read that as "do not touch a fitted feature". That was the conservative reading and it was wrong: data/policy-weights.json is already quarantined and already owes a refit, so moving a fitted value adds to a debt that exists rather than creating a new one. What the rule forbids is RUNNING the refit early. Fixing these is what lets the refit, when it happens, be run ONCE against a correct board.

#276 — THE WEATHER EXPIRES FOR THE FEATURES, AND THE IDEMPOTENCE CLAUSE IS THE WHOLE ROW

what the board said	weather was a bare string that only changed when a NEW weather arrived
what the position says	a weather is up at 32.2% of decision points with a mean of 2.95 turns left
the fix	board.weather is an accessor over weatherLeft() = MEDI.weatherTurns(word, rock, TAGS) - weatherAge() — #271's shape, and the ONE implementation
what stopped being a second copy	rollout_leaf.applyFieldClock computed that same subtraction itself; it calls the board now

THE WORD IS STILL RECORDED AND THAT IS LOAD-BEARING. weatherWord() returns what was set whatever the clock says. Without it the accessor would hand the seed a pre-empted string and fieldClockCounters.weatherExpired — #270's own proof-of-firing — would become unreachable: a dead guard wearing the shape of a fixed one. engine/miltank.js 's two field builds and tests/test-seed-clock.js 's harness both read the word now, and that gate went 133/1 → 134/0 on exactly that line.

RE-DECLARING A WEATHER THAT IS ALREADY UP DOES NOT RESTART IT, AND WITHOUT THAT CLAUSE THE WHOLE ROW IS A NO-OP. Both worlds announce one weather twice — the click through noteMove (which is where the setter's ROCK is picked up), then the store's w event or the live |-weather| line. A naive version resets the age on the turn it is laid, every turn it is laid, and nothing ever reaches its length. Showdown's own Field#setWeather refuses an active weather, so the idempotence is the FACT rather than a choice. A weather that has LAPSED is not "already up", so re-laying it starts a fresh clock — the case naive idempotence gets wrong, and an asserted arm.

#283 — AND THE CLASS OF SIX WAS WRONG IN BOTH DIRECTIONS

data/seed-source-audit.json derived its six by SUBSTRING-MATCHING each callback's source against STUB_HAS_NO, a hand-typed array of fifteen field names. That is the ban-list-of-four shape CLAUDE.md opens with, sitting inside the artifact that defined a register row's scope.

the audit said	the callbacks say
Water Shuriken reads battle	it reads hasAbility("battlebond") . It answers 15 and has never fallen back
—	Triple Axel returns NaN (20 * move.hit), 899 uses
—	Rage Fist returns NaN (pokemon.timesAttacked), 585 uses

the audit said	the callbacks say
—	Avalanche THROWS (<code>pokemon.attackedBy</code>), 29 uses

Asked of the callbacks instead — a member is one that THROWS or returns a non-number — the real class is **seven, carrying 9,163 corpus uses**. `board.unmodelledBasePower(dex, board)` is that question, derived and printed on every gate run, the shape `rollout_leaf.unseededVolatiles()` uses.

WHAT IS SUPPLIED

<code>side.totalFainted</code>	<code>board.graveyard[side].size</code> . Last Respects 7,306 uses: 50 → 50/100/150/200
<code>side.pokemon</code>	the party, so the FORMAT's own <code>onModifyMove</code> builds <code>move.allies</code> . Beat Up 337 uses: it THREW and fell back to its printed o , so <code>damaging</code> was false and the move was invisible to the ranking — the Rock Slide bug a fourth time
<code>move.hit = 1</code>	Triple Axel. The ANSWER does not move — 20 either way — but the printed value stops being a silent fallback

THE ALLY FILTER IS NOT RE-IMPLEMENTED. `onModifyMove` runs on an `Object.create` copy, because `move.allies.shift()` MUTATES and mutating the dex's move would corrupt it for every later reader. The blast radius was measured rather than assumed: of the 29 legal callback moves, **exactly one** also carries an `onModifyMove` , and it is Beat Up. A regulation that adds a second is handled with no edit.

WHAT IS REFUSED, BY NAME — Rage Fist (`timesAttacked`) and Avalanche (`attackedBy`) need a per-body hit ledger the board does not keep, and building one needs a write in the LIVE adapter **and** in the OFFLINE replay or the fit and the player diverge, which is the error CLAUDE.md opens with. Payback (`queue.willMove` , 7 uses) is asking the turn's resolution order, which is the search's question. Assurance COMPUTES its printed 60 and that is correct where it is asked: a decision point is the top of a turn and Showdown clears `hurtThisTurn` at `nextTurn` . Spit Up returns `false` — the move fails — which is right with no Stockpile up, and the board records no layer count.

THE FIT-INVALIDATION, MEASURED AND NAMED — AND THE FROZEN FIXTURE CANNOT SEE EITHER ROW

`engine/feature_shift.js` compiles a textually-patched `board.js` into `require.cache` (#254's technique, so nothing on disk moves under another agent) and compares per-feature columns across three populations. Every patch is asserted to have applied; a patch that matched nothing would report "no columns moved", which is the most dangerous output this file could produce. **PURITY — head against head — is 0 columns.**

population	pre-276	pre-283
the FROZEN fixture, 384 candidates	(none)	(none)
the same boards aged +9 turns	22 features, 3 joint	(none)
the FIT's own rows, 300 games / 51,399 candidate vectors	27 features, 4 joint — 0.889% of vectors, 18% of games	12 features, 6 joint — 0.687% of vectors, 28.33% of games
pooled	—	32 of 58 features, 6 of 18 joint — 1.576% of vectors, 41.67% of games

BOTH ZEROES IN THE FIRST ROW ARE R7 AGAIN AND THEY HAVE DIFFERENT CAUSES. `feature_fixture.buildScenario` sets `board.turn` and THEN calls `setWeather` , so **every fixture weather is zero turns old** and no board in it can have one that has run out — #276 is inert there by construction. And **no fixture board carries Last Respects, Beat Up or Triple Axel**, so #283 is inert there too. Reading either "(none)" as "no refit is owed" would have been the sixth time a guard was trusted for something it does not exercise.

AND THAT CORRECTS R19 RATHER THAN LEAVING IT. R19 reported that the 58 fixture hashes MOVE for #283. They do not. R19 disabled the WHOLE `movePower` callback path, which reaches Low Kick, Grass Knot, Gyro Ball and Acrobatics — moves the fixture DOES carry. The invalidation this row actually owes is real and lives in the corpus, not in the fixture.

THE COLUMNS, FOR THE REFIT. #276: `deadWeather`, `stab`, `eff2`, `effHalf`, `immune`, `accuracy`, `chargeTurn`, `movesFirst`, `speedSwing`, `abilityBlock`, `setupTurns`, `screenValue`, `benchRisk`, `protectThreatened`, `stallIntoEncore`, `diesBeforeMoving`, `koTarget`, `dmgFrac`, `killIsRoll`, `killsThreat`, `koFirst`, `switchSurvives1`, `switchSurvives2`, `switchFaster`, `switchKOFast`, `switchKOSlow`, `switchDiesFirst`; joint `partnerCoversMe`, `focusFireKills`, `overkill`, `doubleKO`. #283: `bp`, `isStatus`, `dmgFrac`, `koTarget`, `killIsRoll`, `koFirst`, `killsThreat`, `eff2`, `effHalf`, `tgtBulk`, `tgtMayProtect`, `defMismatch`; joint `redirectThenAttack`, `bothStatus`, `focusFireKills`, `overkill`, `partnerCoversMe`, `doubleKO`. **Every one of them is inside the class its own row named** — a weather-scaled read, or a damage read downstream of a base power. Nothing outside that moved.

ENGINE WAS LIVE IN THE TREE AND THE TWO HALVES OF THIS RESULT ARE AFFECTED DIFFERENTLY — SAID PLAINLY. `engine/medicham2-browser.js` moved at 02:55 and `data/tags.json` at 02:54; this run finished at 02:45. The **column SETS are unaffected**: both arms live in ONE process with ONE loaded simulator and one loaded tag artifact, so the diff is a property of the `board.js` change and of nothing else — that is the whole reason it is run in lockstep. The **absolute percentages are a reading of the tree at 02:41** and no more, because the damage features underneath them come from a simulator that has since moved. If the refit wants the reach figure rather than the column list, `engine/feature_shift.js` should be re-run against a frozen release.

The proof, red first — four breaks, four attributable reds

deliberate break	result
<code>weatherLeft()</code> always returns null — the pre-#276 board	10 FAIL , including both behavioural arms and both counters
<code>setWeather</code> restarts on every re-declaration — the naive fix	2 FAIL
the pre-#283 stub side ({ <code>sideConditions</code> : {} } , no allies)	12 FAIL
<code>onModifyMove</code> never run and no <code>move.hit</code>	6 FAIL

After: **46 / 46**, and `tests/test-seed-clock.js` **134 / 134**.

THREE ARMS ARE BEHAVIOURAL, because reading `board.weather` back would only prove the accessor returned what the accessor returned: `deadWeather` **1** → **0** across the expiry boundary; a weather-scaled damage column moving for a (weather move, attacking move) pair that is SWEPT FOR rather than chosen — the first version took the first weather-setting move the format offers, which is Sandstorm, which boosts no attacking type, and the arm went silent; and Beat Up's `isStatus` **1** → **0**, plus the Last Respects vector's `dmgFrac` **0.429** → **1.000** at three dead.

Nothing in the gate is typed. The weather move, its length, the rock, Last Respects' `50 + 50N`, Beat Up's `5 + floor(baseAtk/10)` and the CARRIERS of both moves (out of `data/move-priors.json`, so every body standing there is one real teams bring) are all read at run time.

TWO THINGS FOUND AND NOT FIXED, FILED RATHER THAN CARRIED

- **#286** — `weatherSetupHelpsPartner` **guards itself on** `A.__weather`, **which NOTHING WRITES.** One occurrence in the live tree and it is that read, so the comparison is `' ' !== w` and the clause *"only counted when the weather is not already up"* has never bound. The board holds the answer now — `board.weather` is the expiry-aware accessor — but repairing it moves a fitted JOINT column that neither #276 nor #283 names, and the standing instruction is to stop rather than widen a batch.

- **#287** — `data/seed-source-audit.json` **is stale and its** `openAndNotFixed` **block now states something false**, since it says #244's remainder is unfixed. The repair is one line (call `board.unmodelledBasePower`), and it is NOT taken because regenerating that artifact today would bake ENGINE's in-flight `data/tags.json` into it. Until it is re-run, its `basePowerCallbackClass` and `openAndNotFixed` blocks are WITHHELD.

A RED GATE THAT IS NOT MINE, REPORTED RATHER THAN FILED

`tests/test-stadium-roster.js` is **1 FAILURE**: `engine/register_reality.js` → `data/register-reality.json` is in neither `docs/MODELS.md` nor the Stadium nor `NOT_A_MODEL` . It is MEASURE's new generator, untracked in the working tree at the start of this session. This pass's own new generator (`engine/feature_shift.js`) was declared in the same file, which is the action the check prescribes. **Routed to MEASURE; not fixed here, and not called known.**

What was deliberately NOT done

- **NO REFIT WAS RUN.** That is the whole point of the reading above: the debt is named and sized so the refit can be run once, after MEDICHAM, against a correct board.
- **NO HEAD-TO-HEAD WAS PREPARED OR LAUNCHED, AND #275 IS STILL UNMEASURED.** This pass adds a second unmeasured arm in the honest sense — the feature vector moved — and it is LABELLED as one here rather than folded into #275's.
- `engine/medicham2-browser.js` , `engine/tag_dex.js` , `data/tags.json` **and** `engine/game_differential.js` **WERE NOT TOUCHED**, nor were MEASURE's four files.
- `engine/feature_fixture.js` **WAS NOT EDITED.** Adding a scenario for an aged weather is the obvious repair and it moves every stored hash at once, which is a refit-time decision rather than this pass's. `feature_shift.js` ages the same boards instead, which takes nothing away from what is already there.
- **The 58 fixture hashes were NOT restamped.** Restamping is what a refit does; doing it here would hide exactly the invalidation this pass exists to name.

R19 — THE GUARD MOVED FILES, AND THE WALK STOPPED BEING THE NATIONAL DEX. #245 AND #282 CLOSED, #244 CLOSED WITH ITS REMAINDER NAMED. 2026-08-14.

ROADMAP **#244 closed** (seed 2026-08-13, turn one as #246), **#245 closed, #282 closed; #283 and #284 opened** by what the work found. Gates: `tests/test-rollout-fallen.js` 28 → **43** and `tests/test-seed-clock.js` 119 → **134**, shown **RED on four deliberate breaks between them**. Artifact: `data/seed-source-audit.json` (`engine/seed_source_audit.js`). **No run was launched and #275 is still unmeasured.**

The verdict in one line

The seed has carried the dead since 2026-08-13 and the guard watching it could not have seen otherwise — that is now fixed by MOVING the guard, because no counter written inside `fallenCount` could ever fire on this bug. The unfiltered walk is closed and it moved exactly ONE number: a heal block is two turns, not five. And nothing here says MILTANK chooses better.

#244 — CARRIED, AND THE ANSWER IS YES

Asked and answered rather than assumed: `rollout_leaf.buildSide` appends `sideFallen` 's corpses, `battleInit` calls `fallenSettle` (#246, ENGINE), and the gate reads the count back through damage rather than through a field.

the position holds	Last Respects prices at
o in the ground (CONTROL)	50 BP — unchanged, so the fix does not invent deaths
1 / 2 / 3	100 / 150 / 200 BP

Reach was measured (8.75% of 183,840 decision points, `data/rollout-fallen-prevalence.json`) and then converted to effect (6.1% argmax flips over 131 paired decision points, #278) — and three of that arm's eight sampled flips have Last Respects on the board.

THE ROW IS CLOSED AND ITS REMAINDER IS #283, WHICH IS NOT A BOOKKEEPING CHOICE. `board.movePower` still builds no `side.totalFainted`, so the FEATURE vector MAG ranks with prices the move at its printed floor at every depth of the game. A row "closed in part" is read as closed by the register's own detector — that is exactly how #269's six volatiles nearly vanished and why #277 had to exist.

AND IT IS A CLASS, COUNTED AS ONE. `data/seed-source-audit.json` : **29 legal moves carry a** `basePowerCallback` , **and 6 read state** `movePower` 's **stub does not build** — Last Respects (`totalFainted`), Assurance (`hurtThisTurn`), Beat Up (`allies`), Payback (`queue / newlySwitched`), Spit Up (`stockpile`), Water Shuriken (`battle`). A fix aimed at Last Respects leaves five.

IT WAS NOT TAKEN AND THE REASON IS A MEASUREMENT. Disabling `movePower` 's callback path MOVES `engine/feature_fixture.js` 's 58 per-feature column hashes. So a fix there reaches `data/policy-weights.json` , and Will's ruling is that the refit waits for MEDICHAM. It belongs in the same batch as #276.

#245 — THE GUARD WAS IN THE WRONG FILE, AND THAT IS THE FINDING
`fallenCount` 's own comment states the principle correctly: *"a quiet fallback here is indistinguishable from the bug."* `MEDFAILS.fallenNoRoster` then fires only on an ABSENT roster, and #244's roster was present, non-empty and pre-filtered.

ANOTHER COUNTER IN THE SAME PLACE WOULD HAVE BEEN THE SAME GUARD A SECOND TIME. `fallenCount(sf, act, bench)` sees three arrays. Nothing computable from them distinguishes *this side has lost nobody* from *somebody pruned the corpses before I was called* — the information is not in scope, so the file is not where the check can live.

So it moved to the seam:

what it asks	<code>S.sfA.fainted</code> — settled by <code>fallenSettle</code> , and the field Last Respects and Supreme Overlord actually read
what it compares against	<code>board.graveyard[side]</code> — written by <code>board.faint()</code> , and not the array <code>buildSide</code> returns, <code>battleInit</code> slices or <code>sf.team</code> holds
where	both <code>battleInit</code> sites, including <code>rolloutAfterActions</code> , the ranking path
what it says	<code>FALLEN_GUARD.checked</code> (proof of firing), <code>.mismatch</code> (the claim), and it PRINTS the first disagreement — once per process, because a line per rollout in a 200,000-game run is a line nobody reads

It catches strictly more than #244: an unbuildable corpse, a `bringIn` splice that loses one, and a future caller that "tidies" the array all arrive as a mismatch rather than as a confident zero.

THE RED DEMONSTRATION IS A STANDING ASSERTION, NOT A BREAK DONE ONCE BY HAND. The pre-filtered roster is rebuilt inside the gate and both guards are asked the same question in the same block:

ok	the pre-filtered roster is PRESENT and NON-EMPTY – which is why the old guard cannot see it	sfa.t
ok	and the engine confidently counts ZERO fallen on a position with two in the ground	sfa.f
ok	THE OLD GUARD IS SILENT ON THE REAL DEFECT – fallenNoRoster does not move	0
ok	THE NEW GUARD FIRES ON THE REAL DEFECT – exactly one mismatch, on my side	misma

AND A NUMBER IN THAT SAME COMMENT IS STALE — RE-DERIVED, AND IT IS NEITHER OF THE TWO ALREADY WRITTEN DOWN. The comment says 19,299 Last Respects uses; the row said 6,650; data/tags.json says **6,970** today. Three numbers for one fact and none derived at read time, sitting inside the justification for a guard. Filed as **#284** — it is in medicham2-browser.js and is not SEARCH's.

#282 — ONE KEY MOVED, AND THE NAIVE FIX WAS DEMONSTRATED RATHER THAN DESCRIBED

the walk	dex.moves.all() skipping only !m.exists — 954 exist, 500 are legal here
what it cost	the illegal namesake of the heal-block volatile declares condition.duration: 5 ; the format's own durationCallback opens if (effect?.name === "Psychic Noise") return 2
why it got worse	since #277 the seed carries that number into the payout, so it stopped being live-only

THE FILTER ALONE TRADES A WRONG 5 FOR A WRONG 3, AND THAT WAS SHOWN RED. The one legal carrier declares no volatileStatus and no condition of its own — it reaches the volatile through a 100% SECONDARY — so the old expression finds nothing and the lookup falls through to the bare fallback. The naive version fails three arms with Heal Block -> 3 .

A DURATION IS A PROPERTY OF THE CONDITION, NOT OF THE MOVE.

dex.conditions.get(id) resolves the standalone conditions and a move's private one, so the table now asks each legal move which volatiles it can start and runs the authority's own durationCallback with that move as the effect. Same function the server runs; the 2 is DERIVED and self-correcting rather than transcribed.

THE WHOLE MEASURED EFFECT IS ONE KEY (data/seed-source-audit.json):

keys	84 → 56
corrected values	1 — healblock 5 → 2
keys with no legal carrier at all	34 — nothing in this regulation can ever look them up
taunt / encore / disable / throatchop	byte-identical , asserted as controls
computed by the authority / refused / ambiguous	18 / 7 (the trapping family wants this.random ; the DECLARED value is used and the refusal is counted) / 0

AMBIGUITY IS COUNTED BECAUSE A | -start| LINE DOES NOT NAME THE MOVE THAT CAUSED IT. Two legal carriers of one volatile with different durations would be genuinely unresolvable here. Today there are none — measured — and a future regulation adding one must be loud rather than silently resolved.

AND THE ARM IS BEHAVIOURAL. Reading the table back would only prove the table says what the table says, so the gate asserts the corrected number reaches the seeded body's _healBlock .

IS THE FIT INVALIDATED? NO — CHECKED, AND THE INSTRUMENT WAS PROVED LIVE IN THE SAME PASS

58 per-feature column hashes, with both changes, vs a deliberate break restoring both old behaviours	byte-identical
--	-----------------------

the same hashes with <code>movePower</code> 's callback path disabled	THEY MOVE — so the check is not vacuous
---	--

That second row is the control R18's version did not have, and it is also the measurement behind **#283**: `movePower` is in the feature path, so the remainder of #244 owes a refit.

The proof, red first — four breaks, four attributable reds

deliberate break	result
<code>sideFallen</code> returns <code>[]</code> — the pre-#244 seed	17 FAIL , and the guard fires through both real entry points (<code>rolloutWinProb</code> 4 mismatches, <code>rolloutAfterActions</code> 6)
<code>checkFallenSeeded</code> returns immediately	8 FAIL — including both wiring arms and the printed-warning arm
the legality filter removed from the walk	6 FAIL , one behavioural (<code>_healBlock = 5</code>), and <code>ambiguousKeys</code> names the conflict: <code>healblock:5</code> vs <code>psychicnoise:2</code>
the NAIVE fix — filter added, nothing else	3 FAIL , <code>Heal Block -> 3</code>

After: **43 / 43** and **134 / 134**.

A RED GATE THAT IS NOT MINE, REPORTED RATHER THAN FILED

`tests/test-docs-current.js` is **20/1**: `docs/ABRA-whitepaper.md` holds 12 untraceable figures against a baseline of 11, stamped 23:11 tonight. **It is not this work's**, and that is evidence rather than a claim: this pass edited no living document the rule scans (only `docs/ROADMAP.md` , which is not among its eight), none of the twelve figures appears in any file it wrote, and removing `data/seed-source-audit.json` entirely leaves the gate red. Two artifacts were rewritten by other agents inside the failing window — `data/engine-release.json` at 23:39 and `data/tags.json` at 23:41. The rule's own header calls the census *"a pressure gauge and not a verdict"* and forbids re-baselining. **Routed to MEASURE; not fixed here, and not called known.**

What was deliberately NOT done

- **NO HEAD-TO-HEAD WAS PREPARED OR LAUNCHED.** Writing agents were live in the tree all evening — `data/tags.json` moved at 23:41 while this landed — which is the rule that cost this project 7,100 games. **#275 still owes its arm and this pass does not add a second unmeasured one**: the guard reads state and writes none, and the duration fix moves exactly one key.
- **#283 AND #276 WERE NOT TOUCHED.** Both move fitted feature values; the refit is gated behind MEDICHAM by Will's own instruction, and #283's fit-invalidation is measured above rather than assumed.
- `engine/medicham2-browser.js` **WAS NOT EDITED.** Both halves of #245 that live in it — the too-narrow counter and the stale corpus number — are reported, and the counter is now covered from outside rather than patched from inside, which is the stronger fix in any case.
- **The other six callers' dead constants were left in place**, unchanged from R18.

R18 — THE LAST CONSTANTS AND THE LAST VOLATILES. #275 LANDED UNMEASURED, #277 WORKED. 2026-08-14.

ROADMAP **#275 landed and #277 worked; #282 opened** by what the work found. Gate: `tests/test-seed-clock.js` , 63 assertions → **119**, shown **RED on four separate deliberate breaks**. **No artifact was written and no run was launched — see the verdict.**

The verdict in one line, and the first half of it is a caveat

Eight callers typed a Tailwind as four turns whatever was left on it and the leaf reads the real remainder now — and NOTHING HERE SAYS THAT CHOOSES BETTER. #275 is IMPLEMENTED AND UNMEASURED, it owes its own arm, and it changes the seeded speed order of a live search in eight places at once.

The half that is not a caveat: **three of #277's six volatiles are carried, three are refused BY NAME, the choice lock turns out to have existed inside `board.candidates()` the whole time and never to have reached a payout, and the FOE's Protect streak is counted for the first time.**

#275 — one read at the leaf, and the eight copies are gone

what the callers said	<code>twA: hasSide(side,'tailwind') ? 4 : 0 , tr: hasField('trickroom') ? 5 : 0</code>
where	<code>miltank.js</code> ×2, <code>rollout_r1/r2/r3.js</code> , <code>rollout_switch_probe.js</code> , <code>leaf_position_contrast.js</code> , <code>argmax_paired.js</code> — eight sites in seven files , counted rather than quoted: the row said seven, and R17's own harness had grown the eighth while R16 was writing it. (<code>rollout_r4.js</code> builds no field of its own, so "the four R-gates" is three.)
what the position says	<code>board.sideLeft</code> / <code>board.fieldLeft</code> , on the board since #249
the fix	<code>rollout_leaf.applySpeedClocks</code> , called from <code>applyFieldClock</code> , overwriting the caller

IT IS ONE READ AT THE LEAF AND NOT EIGHT EDITS, which is the whole shape of the row. Only `miltank.js` 's two copies were deleted; the other six are now inert, and the ninth caller that gets written tomorrow is correct without knowing this row exists.

NEITHER KEY IS SPELLED THERE. `board.speedSideKeys()` answers from `derived()` 's `speedSide` — the side conditions whose dex condition declares an `onModifySpe` handler — and falls back to the `doublesSideSpeed` tag on a board that has never scored a feature, counted at both tiers, which is `permanentSide` 's three-tier shape from #268. `board.roomFieldKey()` is the declared irreducible `GAME_RULES` already held, so the room is not spelled a second time either.

AN INFINITE REMAINDER IS REFUSED AND COUNTED. `sideLeft` returns `Infinity` for a permanent side condition (#268) and the engine's tick is `if (field[k] > 0 && --field[k] <= 0)` — so handing over `Infinity` would be this row's own bug with the sign flipped. Nothing in this family is permanent today; the guard costs nothing and its alternative failure is silent.

AND IT IS UNMEASURED. SAID PLAINLY, BECAUSE THE ROW ITSELF WARNED ABOUT THIS

The register's own note on #275: *"it changes the seeded speed order of a live search in seven callers at once, so it owes its own arm rather than a free ride."* **That arm was not run and could not be.** Writing agents were live in the tree all evening, which is the rule that cost this project 7,100 games, and every downstream figure is quarantined behind MEDICHAM in any case. R17 is the instrument this owes — a paired `argmax` run under common random numbers — and it is a hand-over, not something to start beside three other agents.

So: a Tailwind with one turn left is now seeded with one turn instead of four, the search sees a different speed order, and whether it picks better is an open question with no number against it.

#277 — three carried, three refused by name

volatile	verdict	why
healblock	CARRIED → _healBlock	pure vocabulary. _vol.healblock is read by NOTHING (medicham2-browser.js:10160); the consumer is _healBlock (healBlocked , :3659)
throatchop	CARRIED → _noSound	same shape, same tick (:20859 beside _healBlock 's :20854 and _vol 's :20937)
the CHOICE LOCK	CARRIED → _lock + _lockT: Infinity	it was never missing — see below
substitute	REFUSED	_sub is the doll's REMAINING hp and the wire never states it
leechseed	REFUSED	_seededBy is a body reference — and the board cannot hold it either : the move declares no condition.duration , so the adapter's table falls back to 3 and the board forgets a seed the real game keeps until the body leaves
perishsong	REFUSED	the two counters are OFF BY ONE, measured — see below

rollout_leaf.unseededVolatiles() prints all three refusals with their reasons on every gate run, and the gate asserts each one still carries one. A silent omission and a considered one look identical in the code; that is why they are printed rather than deleted.

THE PERISH REFUSAL IS A MEASUREMENT, NOT A PREFERENCE, AND IT IS STRONGER THAN THE ROW'S. The row said the engine clears _perish on switch-out, so seeding it would put that rule in two places. True, and not the blocker. The blocker is arithmetic: magnemite.js:548 reads the count straight off the wire's own name (perish3 → three board turns), while the engine holds _perish = 4 at that instant (:15551 announces perish + tn-1) and kills at the end of the turn it reaches 0. So **the board reads ZERO on the very turn the body dies**. A seed would end the count a turn early or drop the lethal turn outright, and the repair is in the live adapter, which is not SEARCH's.

THE TRANSLATION IS A DECLARED JOIN AND THE GATE CHECKS BOTH HALVES.

VOL_ENGINE_FIELD sits beside VOL_MOVE_FIELD for the reason #269 gives: no data field states which engine field a protocol volatile lands in. So the gate asserts the KEY is the format's own condition NAME — which is the word the wire carries and magnemite.js normalises — and that the FIELD still appears in the engine as a read. A name that stops existing fails loudly instead of going inert, which is exactly what _vol.healblock did.

AND THE PAIR IS WRITTEN ABOVE THE _vol TABLE, NOT BESIDE IT. Writing both would leave a consumer-less duplicate on the body — the duplicate the engine refuses at its own owner for Substitute and confusion — and the roster differential compares _vol , so it would read as a divergence that is really a copy.

THE CHOICE LOCK WAS NOT MISSING. IT WAS PRIVATE, AND THAT IS THE FINDING

The row called it *"not on the board at all"*. The FIELD was not; the FACT was, and had been since 2026-07-31. board.candidates() held this inline and nowhere else:

```
const lockedTo = (() => {
  const last = norm(user && user.lastMove || '');
  if (!last) return null;
  const it = dex && dex.items && dex.items.get(norm(user.item || ''));
  return (it && it.exists && it.isChoice) ? last : null;
})();
```

So **the FIT knew a locked body had one legal move and the PLAYOUT did not**: every locked body in every rollout this project has ever run was offered all four of its moves, and the search planned turns the game refuses. It is now board.choiceLockOn(mon, dex) , asked by candidates() and by the seed — **FACTS ARE**

GLOBAL, one implementation.

Only **one** Choice item is legal in this regulation and the two sources that name it — the dex's `isChoice` and the item tag `medicham2.lockMenuMove` itself reads — agree exactly, so both are asked and neither can go quietly empty.

THE FOE'S PROTECT STREAK, WHICH WAS CALLER STATE AND IS NOW A FACT

`mag_bot.js` counts consecutive Protects off the wire and gates it `if (mine)` . So **the opponent's shield was priced as certain on every turn of every game** — while the engine has had the correct $1/(3 \cdot 3^{n-1})$ rule the whole time — and the fitter's board and every offline harness had no count at all, because only the live bot kept one.

The counter is `board.noteMove` 's now: it already read `move.stallingMove` , it is called on every `|move|` line from BOTH sides (`magnemite.js:626`) and by every offline replay, and the feeding set is the dex's own flag plus the engine's `stallCounterFeeds` tag rather than a list. A switch clears it for free, because `switchIn` builds a new body — the species-keyed map could not do that and would have carried the count back in. The map survives only as a counted fallback for a body the board has never seen move.

IS THE FIT INVALIDATED? NO — AND IT WAS CHECKED, NOT ARGUED

`engine/feature_fixture.js` hashes every feature's column separately over the frozen scenario boards, precisely so a changed MEANING under an unchanged NAME is visible.

58 per-feature column hashes, with the change	byte-identical to a run against a deliberate break restoring the old inline lock and removing the streak write
the shared lock vs the dex's own <code>isChoice</code> , over every legal item	148 items, 0 disagreements — and <code>lastMove</code> + item are the WHOLE input to <code>candidates()</code>
<code>board.js</code> FEATURES vs the fitted vector	58 / 58 (<code>tests/test-rollout-switch.js</code>)

Nothing here adds a feature and nothing is read by `featuresFor` . **No refit is owed by either row.** #276 — the board's own weather never expiring for the features — is the one that owes one, and it was deliberately not touched: Will's ruling is that the refit waits for MEDICHAM.

The proof, red first — four breaks, four attributable reds

deliberate break	result
<code>applySpeedClocks</code> returns immediately	9 FAIL — including the CONTROL, which is what says the arms cannot pass by zeroing both fields
the <code>VOL_ENGINE_FIELD</code> branch disabled	4 FAIL , one of them the behavioural arm: the silenced body clicked its sound move
<code>choiceLockOn</code> returns null	4 FAIL , one of them behavioural: the locked body's other move came back onto the menu
the streak write removed from <code>noteMove</code>	5 FAIL

After: **119 passed, 0 failed.**

TWO ARMS ARE BEHAVIOURAL, BECAUSE READING THE FIELD BACK WOULD ONLY PROVE THE SEED WROTE WHAT THE SEED WROTE. A seeded sound lock must REFUSE the sound click, and a seeded choice lock must REMOVE the other move from the menu over six different dice. **The sound-lock fixture is CONSTRUCTED, not found:** the first version handed one species one sound move, the unsilenced control declined to click it, and a control that does not fire proves nothing — so every (species, sound move) pair in the pool is tried until the control actually clicks.

The #275 arms are STATE reads and the file says so. The engine consumes `field.twA` and `field.tr` in its own speed sort and residual tick; what the seed produces is exactly those two numbers and there is no observable between them. The caller deliberately passes the OLD constants in every one of those arms, so a green row can only mean the board overruled it — `twCallerDiffered` counts the overrides and the gate fails on a zero.

#282 — FILED, NOT FIXED: the live adapter reads a duration off a move this format does not have

`engine/magnemite.js:130` walks `dex.moves.all()` and skips only `!m.exists` — **no** `isNonstandard` **filter**, the exact walk CLAUDE.md names. The move sharing the heal-block volatile's name is `isNonstandard: 'Past'` with `condition.duration: 5`, while this regulation's only carrier declares **2** through its `blocksHealing` tag, matching Showdown's own `durationCallback`. So the live board records that volatile as five turns where the game says two — and since #277 the seed carries that number into the playout.

It is filed rather than fixed for two reasons: `magnemite.js` is the live adapter and not SEARCH's, and **the repair is not just adding the filter** — with the illegal row gone the table has no entry at all and the fallback is a bare 3, so the duration has to come from the tag the engine already reads. The seed carries whatever the board holds, and this says so out loud rather than quietly compensating.

What was deliberately NOT done

- **NO HEAD-TO-HEAD WAS PREPARED OR LAUNCHED, AND #275 IS UNMEASURED.**
Writing agents were live in the tree; a rollout measurement against the live tree is void by the rule that cost 7,100 games. Checked rather than assumed: no self-play or `mew.js` process was running while this landed.
- **#276 WAS NOT TOUCHED.** It moves fitted feature values, and the refit is gated behind MEDICHAM by Will's own instruction.
- **NO REFIT WAS RUN AND NONE IS OWED BY THESE ROWS**, measured above rather than asserted.
- `engine/medicham2-browser.js`, `engine/magnemite.js` **and** `engine/mag_bot.js` **WERE NOT EDITED.** The Protect streak was derivable inside `board.js`, which is what made that possible — the brief allowed `mag_bot.js` and it turned out not to be needed, because the fix was to STOP reading caller state rather than to extend it.
- **The other six callers' dead constants were left in place.** They are inert and correcting them would mean editing four quarantined R-gates and two probes for no behavioural change.

Adjacent gates, run and reported

Green, each read off its own output: `tests/test-seed-clock.js` (119/119), `tests/test-rollout-seed.js` (48/48), `tests/test-seed-residue.js` (20/20), `tests/test-rollout-fallen.js` (28/28), `tests/test-rollout-switch.js` (16/16), `tests/test-rollout-gates.js` (16/16), `tests/test-pp-fact.js` (33/33), `tests/test-feature-semantics.js` (18/18), `tests/test-hazard-side.js` (11/11), `tests/test-switch-carry.js` (27/27), `tests/test-board-browser.js` (14/14, 58 of 58 features agree), `tests/test-engine-consistency.js`, `tests/test-switch-features.js`, `tests/test-forced-switch.js`, `tests/test-wiring.js` (*"every capability proved it ran"*), `tests/test-miltank-release.js` (25/25).

`tests/test-rollout-switch.js` and `tests/test-pp-fact.js` are the real control again: both assert exact, dice-for-dice win probabilities, which is only possible if a board with no Tailwind, no room, no translated volatile and no lock is byte-identical to what it was.

`engine/miltank.js` **reports** `OFF_RELEASE` **and that is correct rather than a failure** — `board.js` has moved since the last release was cut, which is what the stamp exists to say. Nothing here was measured through a release, and the next measurement needs a fresh cut.

R17 — THE PAIRED ARGMAX RUN. REACH BECOMES EFFECT, AND IT IS 47.3%. 2026-08-14.

ROADMAP #278. Artifacts: `data/argmax-paired.json` (headline) with `data/argmax-paired-n12.json` and `data/argmax-paired-n100.json` folded into it as the N-sweep, all written by `engine/argmax_paired.js`. Engine release `957c638ba6e5`, asserted to be the committed engine at `58a26a7` rather than assumed.

The verdict in one line

Every prevalence figure R13 through R16 published was a ceiling on reach, and it converts: over the same 131 decision points under common random numbers, the seed work changes MILTANK's argmax on 62 of them — 47.3%, against a paired null of exactly 0 that this run demonstrates three separate times. It changes MAG's argmax on ZERO of them. And nothing here says the new choice is better.

What was run

Six arms, each a COMMIT, so the delta between consecutive arms is one landing and attribution is a subtraction rather than an assertion. `Module._compile` into `require.cache` — #254's technique — because three other agents were live in the tree and reverting it would have fought them. The board, the leaf and the replay closure that calls them are per-arm; the simulator, the dex, the tags, the priors, the engine data and **the weights** all come from one frozen release and are byte-identical across every arm.

arm	commit	what it adds
pre	f8f2c67	baseline — before any of it
r13	16bd0e5	#244 the fallen count reaches the seed
h254	1cd6af5	#254 a side condition lands on the right side
r14	25d67c5	#247 #248 #249 #250 bench state, move count, side state
r15	435be2b	#271 a knocked-off item
head	58a26a7	#267 #268 #269 #270 the clocks

131 paired decision points, 75 open-sheet games, every 3rd board, side p1, top-3 per slot by MAG's own score, n=40 rollouts per cell, explore=1.0 — the same sampling `engine/rollout_r3.js` uses, so this sits on R3's axis and not on a new one.

THE RESULT, PER LANDING WHERE IT SEPARATES AND POOLED WHERE IT DOES NOT

pair	rows	argmax flips	reach that row published
pre -> r13	#244 the fallen count	8/131 = 6.1%	8.75%
r13 -> h254	#254 the side a condition lands on	0/131 = 0.0%	(hazards: 60 clicks in 14,288 games)
h254 -> r14	#247 #248 #249 #250	61/131 = 46.6%	WITHHELD
r14 -> r15	#271 a removed item	0/131 = 0.0%	3.62% — and see below
r15 -> head	#267 #268 #269 #270	13/131 = 9.9%	39.72%
pre -> head POOLED	all of it	62/131 = 47.3%	WITHHELD

The pooled figure is barely above the R14 row on its own, and that is not an error: later landings re-flip decisions the earlier ones already moved, and two of them flip a decision back.

THE ARGMAX-FLIP COLUMN IS UNAFFECTED; THE REACH COLUMN'S R14 CELLS ARE WITHHELD. The flip counts are this run's own paired measurement and are untouched. The two reach cells came from `data/rollout-seed-prevalence.json`, whose ANY figure was inflated by a species lookup that

dropped every forme — mechanism, scope and the pinned re-run command are in R14 below. The other three reach cells come from different artifacts and are unaffected.

THE CONTROLS, AND THE ONE THAT MATTERS IS NOT THE ONE YOU EXPECT

control	result	what it says
PURITY — the same arm, the same dice	0/131	the null for every row above; a paired run that cannot reproduce itself is measuring its own state
UNPAIRED — the same arm, different dice	82/131 = 62.6%	what this comparison invents WITHOUT common random numbers
INERT BY CONSTRUCTION — r14 -> r15 , which is #271 alone	0/131	a code change that cannot reach the position returns exactly 0, measured rather than argued

THE UNPAIRED FIGURE IS NOT THE COMPARATOR, AND READING IT AS ONE WOULD DECLARE EVERY POSSIBLE RESULT NULL. The same search, the same menu, the same opponent policy, the same everything it can see — only the dice differ — and it disagrees with itself on 62.6% of these decisions. That is R3's finding arriving again from a different direction: the argmax is taken over nine estimates that are mostly near-ties, so an unpaired instrument here is saturated before it starts. The pairing is what makes the question askable at all, and the null for a paired rate is zero — which this run establishes three times over, twice from a code change that provably cannot reach the position and once from the dice.

DOES IT SURVIVE A BIGGER BUDGET? MEASURED — IT SHRINKS AND IT DOES NOT GO AWAY

The one thing neither control can settle: common random numbers pair the SEED, not the TRAJECTORY. Once two seeded states differ the playouts consume the stream differently, so some share of those 62 flips is a rerouted die rather than a different answer. That component falls as the rollout budget grows; a real difference in the true values does not. So it was swept.

rollout budget	pre -> head
n=12	70/131 = 53.4%
n=40	62/131 = 47.3%
n=100	54/131 = 41.2%

An 8.3x budget costs 12 points. Some of the rate is the residual and it is not most of it. The flip rate is budget-dependent and every figure above is printed with its budget — do not extrapolate it to a limit this run did not measure.

AND WHETHER THE NEW CHOICE IS BETTER IS UNMEASURED. SAID PLAINLY, WITH THE NUMBER THAT SAYS SO

A flip is a different choice under a more correct board. It is not a win.

The obvious way to give it a direction is to price both picks under the newer arm's own leaf, and this run does compute that: when they disagree, head 's pick is worth +9.70 points on average (median +6.25) over pre 's pick by head 's own leaf. That number carries no information, and the control is what proves it: the identical statistic computed for the UNPAIRED control — where the truth is that both picks came from the same player and neither is better — is +7.96 mean, +7.50 median. The gap is argmax selection bias almost exactly, and it shrinks with budget the way selection bias does (+15.83 at n=12, +9.70 at n=40, +7.86 at n=100).

So: the search now chooses differently, and this run does not say it chooses better. The thing that answers "better" is an SPRT and this is not one.

WHAT THE FLIPS LOOK LIKE, BECAUSE A RATE IS NOT A MECHANISM

The artifact records eight flipped decisions PER ARM PAIR, and each pair's examples carry its own row's fingerprint. This is the check that the rate is the fix rather than the dice, and it is the one a control cannot give you.

h254 -> r14 — **three of its eight drop Fake Out:**

```
was  M:thunderbolt@a + M:fakeout@a      now  M:protect@-      + M:spiritbreak@a
was  M:fakeout@b      + M:closecombat@b  now  M:closecombat@b + M:closecombat@b
was  M:hurricane@a    + M:fakeout@a      now  M:hurricane@a    + M:gunkshot@b
```

The old seed built every body with `_mvActs: 0`, so **Fake Out was not merely offered, it WORKED** inside the payout for a body that had been on the field for six turns — and the search ranked it on a payoff the server would have refused. That is not a mispriced estimate, it is a rule the payout was not obeying, and it is why #250's batch is 46.6% while its neighbours are single digits.

pre -> r13 — **three of its eight have Last Respects on one side or the other**, which is the move whose whole identity is that it grows as your team dies, and it is the one #244 was priced at 50 base power for:

```
was  M:psychicfangs@a + M:allyswitch@-  now  M:psychicfangs@a + M:lastrespects@a
was  M:blizzard@-     + M:lastrespects@b  now  M:moonblast@a   + M:lastrespects@b
was  M:partingshot@b  + M:lastrespects@b  now  M:spiritbreak@b + M:lastrespects@b
```

r15 -> head — **three of its eight are weather-dependent moves**, which is #270 arriving in a click: a weather that used to run for sixty turns now expires, so Aurora Veil, Weather Ball and Solar Beam are all priced differently and two of them get displaced:

```
was  M:auroraveil@-  + M:protect@-      now  M:blizzard@-     + M:protect@-
was  M:heatwave@-    + M:partingshot@b  now  M:weatherball@b  + M:partingshot@b
was  M:earthpower@b  + M:solarbeam@a   now  M:earthpower@b   + M:weatherball@a
```

#271 AND #269 CONTRIBUTE EXACTLY ZERO HERE, AND THAT IS A PROPERTY OF THE HARNESS

The decision points come from `engine/joint_rows.js` 's replay — the FIT's replay, and the only implementation of it. It applies switches, moves (through `B.noteMove`, so side conditions and move counts land), damage, status, hp, boosts, faints, weather and field. It applies **neither** `board.noteItem` **nor** `board.startVolatile . board.noteItem` has exactly one caller in the repository and it is `engine/magnemite.js`, the live protocol reader.

So **#271 (3.62% reach) and #269 (4.99%, itself a floor) are structurally invisible to this run**, and every flip rate above is a **FLOOR on what the live board would show**. This is not a gap to be patched here: it is MEASURE's **PRIORITIES 13e**, the missing offline event, named in R15 as still open. Turning #271 from 0.0% into its real number requires the replay to emit item events, which changes what the FIT sees, which is a refit question.

The upside is that it gave this run its best control for free. `r14 -> r15` is #271 alone, it is provably inert on these positions, and it returned **exactly 0** — a code change producing zero flips when it cannot reach the position, measured rather than asserted.

FOUR DECISIONS INSIDE THE HARNESS THAT WERE LOAD-BEARING

1. COMMON RANDOM NUMBERS ARE KEYED ON THE CANDIDATE'S IDENTITY, NOT ITS INDEX.

`rollout_r3.js` seeds each cell with `ia * 31 + ib`, which is correct when both runs see the same menu in the same order and is exactly wrong for a paired run: a board change can reorder or resize the menu, and an index-keyed seed would then hand the SAME candidate DIFFERENT dice in the two arms — noise attributed to the fix. The seed here is a hash of `move@target + move@target`. (As it turns out the menus were identical on all 131 decisions in all six arms, so it changed no number in this run. It would have been the first thing to break the day a fix moved a feature, and the failure would have looked like a result.)

2. THE ARM IS A COMMIT AND THE HARNESS MOVES WITH IT.

`joint_rows.js` grew a `B.sideFor` call in `1cd6af5`, so HEAD's replay throws `B.sideFor is not a function` against the `pre` board. Seven files are compiled per arm; everything else is the frozen release.

3. THE RELEASE IS ASSERTED TO BE THE COMMITTED ENGINE, NOT ASSUMED TO BE.

`cut()` freezes whatever is on disk, and ENGINE was mid-edit in `medicham2-browser.js` while this ran — a release cut in that moment VERIFIES perfectly, is internally consistent, and is a build nobody has. So the run takes `REL_ID` and then compares the manifest's digests against the bytes git holds at the `head` arm's commit, for the simulator and for both files the arms swap. It refused twice while being built: once on a modified arm file, once on a release that did not match. (Line endings are normalised out first, and that is not a shortcut — this checkout stores LF and writes CRLF, so `git show` and the snapshot disagree on 4,075 bytes of `board.js` and agree on every character of it.)

4. THE SHOWDOWN CHECKOUT IS RESOLVED BEFORE ANYTHING LOADS.

`showdown_path.js` walks up from its OWN `__dirname`, and a copy sitting in `data/releases/<id>/engine/` walks into `data/`, finds nothing, falls through to the `/tmp/ps` candidate and kills the run with a message about npm. The live copy is required first for its documented side effect. This is what a release CANNOT freeze, and the release records the Showdown commit instead.

Ties are broken by the cell's NAME, and that is not style

Menu order is exactly what a board change can move, so an enumeration-order tiebreak would manufacture flips out of exact ties between two arms that agree. At $n=40$ a tie is a 1-in-40 grid and they are common.

Adjacent gates, run and reported

Run in this pass and green, every one of them read off its own output rather than remembered: `tests/test-stadium-roster.js` (ALL PASS, 122 generators — the new one carries a `NOT_A_MODEL` entry with its TRIGGER, and it was the only gate this work turned red), `tests/test-seed-clock.js` (63/63), `tests/test-seed-residue.js` (20/20), `tests/test-rollout-seed.js` (48/48), `tests/test-rollout-fallen.js` (28/28), `tests/test-rollout-switch.js` (16/16), `tests/test-rollout-gates.js` (16/16), `tests/test-pp-fact.js` (33/33), `tests/test-engine-release.js` (66/66), `tests/test-miltank-release.js` (25/25), `tests/test-artifact-keys.js` (4/4), `tests/test-provenance-discovery.js` (229 rows, all clear), `tests/test-timestamps.js` (6/6), `tests/test-docs-current.js` (21/21), `tests/test-roadmap-register.js` (3/3).

`tests/test-rollout-switch.js` and `tests/test-pp-fact.js` are the real control again: both assert exact, dice-for-dice win probabilities, which is only possible if the leaf this run measured through is byte-identical to the one they pin.

No engine file was touched by this run at all — it reads two files it does not write and six commits it does not move — so the red gates standing at R16 are standing unchanged and this work contributes zero rows to any of them. **ENGINE landed two commits on `medicham2-browser.js` and `engine/game_differential.js` while this pass was in flight, so the gate readings above are a reading of the tree at that moment and nothing more. THE RUN ITSELF IS UNAFFECTED, and that is the whole point of the release: it read `957c638ba6e5` and never the tree.**

What was deliberately NOT done

- **No SPRT was prepared.** This run establishes the PRECONDITION for one — the two builds are different players on nearly half of decisions — and says nothing about which wins. That is R4's question, it needs a frozen release and a hand-over, and R4 is still quarantined behind MEDICHAM.
- **No engine file was edited to make #271 or #269 measurable.** The replay's missing item event is PRIORITIES 13e and it is MEASURE's; adding it here would change what the FIT sees inside a run measuring something else.
- **#275, #276 and #277 were not taken.** #276 in particular would move fitted feature values, and this run is the argument for doing it rather than a licence to do it inside the same pass.
- **The leaf was not re-calibrated and this result inherits that.** Every decision here is an argmax over an uncalibrated leaf; the 62.6% unpaired control is what that looks like from the inside. A flip rate is a statement about two builds ranking the same menu differently, and it is only as meaningful as the ranking. That item is MEASURE's.

R16 — EVERY CLOCK THE POSITION WAS RUNNING. #267, #268, #270 CLOSED AND #269 CLOSED IN PART, 2026-08-14.

ROADMAP #267, #268 and #270 closed; #269 closed in part, as one batch — they are one surface and the same defect asked four times. Gate: tests/test-seed-clock.js (63 assertions, auto-discovered by tests/run-all.js), shown RED first. Prevalence artifact: data/rollout-clock-prevalence.json , written by engine/rollout_clock_prevalence.js . ROADMAP #275, #276 and #277 opened by what the work found. This completes the five-row sweep R14 filed; #271 landed separately on 2026-08-13 so its result stays attributable on its own.

#269 is CLOSED IN PART and its remainder is a row of its own, #277, deliberately. A row closed in part is a row whose leftovers have no home, and a defect with no home is the UNREGISTERED shape open_work.js exists to print — the register's closed-detector reads "closed in part" as closed, so the six volatiles and the foe's Protect streak would have vanished from the work list on the day they were half fixed.

The verdict in one line

A weather was up at 32.2% of decision points with a mean of 2.95 turns left, and the seed ran every one of them for sixty turns — that single row is bigger than the other three put together, and #268, the one the roadmap called fit-invalidating, turns out to move exactly two candidates in 139,340.

What each one was, and what closed it

row	the seed said	the position says	closed by
#270	weatherT: 0 — and zero is the engine's word for NEVER EXPIRES	the sun has two turns left	board.setWeather stamps weatherSince / weatherRock ; rollout_leaf.applyFieldClock asks MEDI.weatherTurns for the length and deletes an expired weather
#268	a permanent hazard is up for one turn offline, five live, and is always one layer	the rocks stay until they are removed, and Spikes stack three deep	startSide asks the FORMAT whether a condition is permanent; the Map's value is {until, layers} and board.sideLayers is the one reader
#269	no Taunt, no Encore, no Disable	all three are on the live board with a duration	rollout_leaf.seedVolatiles , over the engine's OWN durationVolatiles() join
#267	a body carries a status and no counter	it is three turns into the sleep	board.statusClock , booked in endTurn ; seedStatusClock writes slpTurns / toxTurns / frzTurns

Does it change decisions? MEASURED — and the four are wildly unequal

`data/rollout-clock-prevalence.json` , over **14,288 open-sheet bo3 games / 192,912 decision points**. It reads the store and `data/tags.json` , **plays no game and opens no** `Dex` — no `battleInit` , no rollout, no board — so it is not downstream of MEDICHAM, is not quarantined, and needed no engine release with three other agents in the tree. **Same denominator as** `data/rollout-item-prevalence.json` **by construction**, so R15's 3.622% and these sit on one axis.

	share of decision points
#270 a WEATHER is up (mean 2.95 turns left)	32.223%
#267 a body is some turns into slp / tox / frz	8.243% (7.405% of them on the field)
#269 a duration volatile is up — a FLOOR, and the worst of the four	4.994%
#268 a hazard is up	0.271% (0.08% older than five turns, 0.015% deeper than one layer)
(context) after a terrain was set — a CEILING , kept out of the headline	17.857%
ANY of them — the ceiling on this batch's reach	39.724%

Every floor and ceiling is stated with its direction, because three of the four have one. #269 is a FLOOR for a structural reason: **the store records no** `|-start|` **at all**, so a Taunt is countable only from the CLICK that caused it and every volatile an item or an ability starts is invisible — the live board sees strictly more, by an unknown amount. #267 is a CEILING the other way: the store records a status landing and never a cure, so a Heal Bell, a Lum Berry and a Natural Cure pivot all leave the body counted as still stasured. The terrain figure is a CEILING because a terrain's duration is not in the tag artifact and reading it would mean opening a `Dex` — which is the one thing that keeps this scan honest — so it is printed and deliberately excluded from the union. And all of it is a **ceiling on decisions**: it counts positions the seed described wrongly, not argmaxes that flip.

#268 WAS FILED AS FIT-INVALIDATING AND THAT IS CORRECTED — MEASURED, NOT ARGUED

The row says *"fixing it moves* `deadSide` *and* `setupTurns` *, which are fitted FEATURES, so it invalidates* `data/policy-weights.json` ... *THIS IS NOT SEARCH'S.*" It is fit-**affecting** in kind and immaterial in size:

<code>deadSide</code> fires, with the fix	927 of 139,340 candidates
<code>deadSide</code> fires, against a deliberate break restoring the old semantics	925
every other feature in <code>engine/feature_coverage.js</code> , 800 fit games	byte-identical
<code>engine/feature_fixture.js</code> , 58 per-feature column hashes	byte-identical

Two candidates. The reason is the format rather than the code: **60 hazard clicks across 14,288 open-sheet games**, and not one hazard is laid in the first 120 games of the fit corpus — measured by counting `startSide` calls through the fitter's own replay, which returns `{tailwind: 80, reflect: 18, lightscreen: 18, wideguard: 17, auroraveil: 3, quickguard: 3}` and no hazard at all. `setupTurns` does not move, exactly as #254's own note predicted: no hazard carries a `condition.duration` , so `dur()` is 0 whatever `alreadyUp` says.

This is the same correction shape as #271's, and it is worth as much as the fix. A row that says "belongs with the refit" stops work; the number says it does not. The fix is still correct and still worth having — 0.271% is small and it is not zero, and a model that re-lays Stealth Rock because it forgot the rocks is wrong in a way that is visible on the screen.

Four decisions inside the fix that were load-bearing

1. `startSide` asks the **FORMAT, not the caller, and that is** `sideFor` 's shape one screen up. Whose side a condition lands on (#254) and how long it lasts are both FACTS; a fact answered independently by two callers agrees only by accident. So permanence is derived — a move declaring no `condition.duration` — which overrules the live adapter's guessed `fieldDuration` of 5 without `magnemite.js` being touched at all.

2. The **FIRST version of that read only** `derived()` and was **MEASURED WRONG**. `derived()` needs a dex and is filled by `featuresFor`, so on a board that had never scored a feature the layer ceiling silently fell back to one — **four Spikes read one layer deep**, the defect wearing the shape of the fix, caught by the gate rather than by review. The `hazard` tag answers both questions with no dex at all (its `hazard` param IS the side-condition key), so it is the primary source and `derived()` the refinement, and `board.sideCounters` counts every fallback taken.

3. `applyFieldClock` is a new seam, **NOT a new argument on** `applyField`. Seven callers build the field object by hand — `miltank.js` twice, the four R-gates, the switch probe, the contrast tool — and every one would have grown the same two lines. Seven copies of one fact is the defect this whole batch is made of. The board is already in scope at both leaves, so the clock reads it there and every caller is fixed without being edited.

4. An expired weather is **DELETED, not counted down to zero**. Zero means "runs forever" to the engine, so handing over an expired weather with `weatherT: 0` is this row's bug spelled differently. The board's own `weather` field is deliberately left alone — it feeds `deadWeather` and the weather-scaled features, and expiring it moves fitted values. That it never expires for those features either is real, is filed as **#276**, and is not fixed here.

#269's FIRST TASK WAS A VOCABULARY CHECK AND THE MISMATCH IS REAL

The row warned that `_vol` 's keys come from a tag param and the board's from the protocol's own name, and that a key that does not match seeds a volatile **nothing reads** — silently. It is not hypothetical, and the engine says so itself at `medicham2-browser.js:10160`: `_vol.healblock` is read by **NOTHING**; the consumer is `_healBlock`. Writing the board's key straight into `_vol` would have produced exactly the green-looking no-op that comment describes.

So the seeded set is **the engine's own table, rebuilt by the same expression** — the `sealsMoves + statusInflict join durationVolatiles()` uses, which yields exactly `taunt`, `encore`, `disable`. The two vocabularies cannot come apart, and the gate asserts the seed's set against the engine's rather than against a list. Encore and Disable need the MOVE as well as the count; the board records it at `startVolatile` from the body's own `moveThisTurn || lastMove`, which is what both moves act on, so the protocol did not have to grow an argument. The one thing that **cannot** be derived — which of the two engine fields each takes, since no data field states that Encore locks IN and Disable locks OUT — is a declared JOIN in one place (`VOL_MOVE_FIELD`) that the gate checks against the derived table.

Five are declared UNSEEDDED with a reason each, and `rollout_leaf.unseededVolatiles()` **prints them on every run**: `substitute` (`_sub` is the sub's remaining hp, which the wire never states), `leechseed` (`_seededBy` is a body reference), `healblock`, `perishsong` (`_perish`, which the engine clears on switch-out) and `throatchop` (`_noSound`). A silent omission and a considered one look identical in the code, which is why they are printed.

The proof, red first

`tests/test-seed-clock.js` was run before a byte of the fix existed: **11 FAIL** and then an abort on `bd.sideLayers` is not a function, every failure one of the four counters reading its floor — `weatherT 0` on a weather five turns long, a hazard gone one `endTurn()` after it was laid, no `_vol`, no `slpTurns`. A second

honest red came mid-fix and is the more useful one: with the layer ceiling reading only `derived()` , **5 FAIL** — four Spikes at one layer and the behavioural arm at `-17 / -17` . After: **63 passed, 0 failed**.

Every control was green in both reds, which is what says the file cannot pass by making everything permanent or by inventing a clock: a timed side condition still expires exactly on its own duration (Reflect, Light Screen, Aurora Veil at 5, Quick Guard at 1), an unstatused body two turns in is byte-identical, a paralysed body gets no counter, and a body with no Taunt carries none.

Nothing in it is typed. The weather-setting move and the terrain-setting move are the first the format offers; the weather's length is `MEDI.weatherTurns` ; the rock is found by probing every legal item until one lengthens it; the hazards and their ceilings come from the `hazard` tag; the duration-volatiles from the engine's own join; the timed controls are every legal move that declares a `condition.duration` . Two observables are behavioural rather than field reads — **three layers of Spikes take more HP off a switch-in than one (-35 against -17)**, and **a seeded Taunt actually refuses the status click** — because reading `_vol` back would only prove the seed wrote what the seed wrote.

And the wires prove they ran (§5): the gate asserts `fieldClockCounters.weatherKnown` , `weatherExpired` , `terrainKnown` and `volCounters.seeded` are all non-zero, that the hazard table loaded, and it prints `volCounters.unmapped` — the vocabulary mismatch as a number, so a volatile arriving with nowhere to go is visible on a live board instead of silent.

Adjacent gates, run and reported

Green: `tests/test-rollout-seed.js` (48/48), `tests/test-seed-residue.js` (20/20), `tests/test-rollout-fallen.js` (28/28), `tests/test-rollout-switch.js` (16/16), `tests/test-rollout-gates.js` (16/16), `tests/test-pp-fact.js` (33/33), `tests/test-hazard-side.js` (11/11), `tests/test-weather-duration.js` (60/60), `tests/test-feature-semantics.js` (18/18), `tests/test-switch-features.js` , `tests/test-switch-carry.js` (27/27), `tests/test-board-browser.js` (14/14), `tests/test-engine-consistency.js` , `tests/test-forced-switch.js` , `tests/test-miltank-release.js` (25/25), `tests/test-wiring.js` (*"every capability proved it ran"*), `tests/test-artifact-keys.js` (4/4), `tests/test-provenance-discovery.js` , `tests/test-timestamps.js` (6/6), `tests/test-fixture-legality.js` , `tests/test-docs-current.js` (21/21), `tests/test-roadmap-register.js` (3/3), `tests/test-lownode.js` (4/4).

`tests/test-rollout-switch.js` and `tests/test-pp-fact.js` are the real control again: both seed boards with nothing up and assert exact, dice-for-dice win probabilities, which is only possible if a board with no weather, no hazard, no volatile and no status is byte-identical to what it was.

`tests/test-stadium-roster.js` **was RED on this work and is GREEN**: the new prevalence generator was undeclared and now carries a `NOT_A_MODEL` entry with its reason. It was the only gate this work turned red, and it was fixed rather than filed.

Four gates are RED, none is this work's, and this work contributes ZERO rows to any of them — verified by name, not by assertion. `tests/test-volatile-duration.js` and `tests/test-perish-song.js` fail their FIXTURE LEGALITY audit (*"clefable can't learn Perish Song"*, *"snorlax can't learn Pound"*), which is **ROADMAP #266** — those two files are named in that row with three verdicts each, and the failure is a moveset declaration with nothing to do with a clock. `tests/test-no-silent-failure.js` is red at **81** NEW silent catches against R15's 80; grep of its full list for `board.js` , `rollout_leaf.js` , `rollout_clock_prevalence.js` and `test-seed-clock.js` **returns nothing** — and the one catch this work did add, `hazardSideTable` 's, was flagged and then made to speak (`sideCounters.hazardTableFailed` plus the reason), which is why the count did not go to

1. `tests/test-effective-identity.js` is 18/1 at **1,471** raw reads against a 1,198 baseline; the 20 contributing files are named in its own output and **none of the four is mine**.

What was deliberately NOT done

- **No SPRT was prepared and no leaf value was re-measured.** Three other agents were in the tree tonight — `engine/medicham2-browser.js` on Feint / Phantom Force, `engine/residual_order.js`, the divergence cards — and the census moved 556 → 570 while this landed, so a rollout measurement against the live tree is void by the rule that cost this project 7,100 games. The prevalence figure above is a store scan by construction and is the number that can honestly be taken today. **The paired argmax run is now unblocked:** all five rows of the R14 sweep are closed or declared, so a frozen release and a common-random-numbers comparison is the next thing this division owes.
- **No refit was run and none is owed by these rows.** #267, #269 and #270 add no feature and change no fitted value; #268's is two candidates in 139,340, measured above. **#276 is the one that owes a refit** and it is filed rather than taken.
- `engine/medicham2-browser.js`, `engine/magnemite.js` and `engine/mag_bot.js` **were not touched.** The Encore/Disable move and the hazard ceiling were both derivable inside `board.js`, which is what made that possible; the foe's `protectTurns` living in `mag_bot.js` is still open and is still not SEARCH's to move.
- **#275 was found and left open on purpose.** `miltank.js` types a Tailwind as four turns and a Trick Room as five whatever is left on them — #270 in a different file — and it changes the seeded speed order of a live search in seven callers at once. Landing it inside this batch is exactly what makes a batch unattributable.

R15 — A KNOCKED-OFF ITEM WAS STILL ON THE BOARD'S BODY. ROADMAP #271 CLOSED 2026-08-14.

ROADMAP #271 closed. Gate: `tests/test-seed-residue.js` (20 assertions, auto-discovered by `tests/run-all.js`), shown **RED first**. Prevalence artifact: `data/rollout-item-prevalence.json`, written by `engine/rollout_item_prevalence.js`. **#267, #268, #269 and #270 were deliberately NOT taken in this pass**, so this result is attributable on its own.

The verdict in one line

Every damage feature MAG scores with, and every body in every playout, was priced holding an item the game had already removed — and the board had known it was gone the whole time.

The defect, exactly as it was measured

```
declare Life Orb -> noteItem('p1','garchomp','') ->
  board.sheetItem = ''          (correct)
  slot.item       = 'lifeorb'   (stale)
  dmgMon(...).item = 'lifeorb'  <- what MAG scores with AND what every playout gets
```

`board.switchIn` **copied** the item off the sheet onto the slot object. `board.noteItem` — the one thing `| -item|` and `| -enditem|` reach — wrote only `itemNow`. `sheetItem()` was the **sole reader** of `itemNow`. So the board held two answers to one question and every consumer read the wrong one.

This is CLAUDE.md's founding lesson arriving through a new door, and the door was in the file the lesson was written about. *"the damage and speed calculations keep applying an Assault Vest, Choice Scarf or Life Orb that is gone — and those apply at ANY hp"*, and **PREFER OBSERVED OVER DECLARED**. The rule was written down. The tracking existed. One reader read the wrong field.

It is ONE SOURCE, and a write-through was refused

`noteItem` could have been made to walk the slots and patch them. That fixes today's symptom and leaves **two** places that answer *"what is this body holding"* — which is what created this in the first place. So `mon.item` is now an **accessor** that calls `board.sheetItem(side, mon.base)` :

- **Every existing reader is fixed without being edited.** `dmgMon` , `monSpeedMult` , `effSpecies` 's mega-stone check, `candidates()` 's choice lock and every caller outside `board.js` now ask the one function. An accessor cannot drift; two copies always eventually do.
- **The key is `base` , frozen at entry** — `baseSpecies(species)` computed from the same string the old copy used — so a mid-battle mega, a Ditto transform or an Illusion breaking keeps looking the item up under the key `sheet` , `itemNow` and `pp` are all keyed by. Re-deriving it from a mutating `mon.species` would be a second dialect, which is MILTANK.md §3.7's terrain defect one layer down.
- **null from `sheetItem` means "no sheet" and is NOT `''`** . On null the accessor falls back to `_itemAtEntry` , the literal it replaced, so a closed-sheet board and a board with no item event are both byte-identical to what they were. Collapsing the two would strip the item off every closed-sheet Pokémon — a new bug wearing the shape of a fix.
- **The setter routes to `noteItem`** . `board.js` is 'use strict' , so a getter with no setter would turn a silent staleness into a crash on `mon.item = x` from any caller. Routing the assignment makes it mean what it says.

THE SWEEP CAME FIRST, and it found four more readers of the declared item

Finding a reader after the fix means measuring twice, so the sweep ran before a byte was changed. **All five are fixed in this one pass**; two of them are outside `board.js` .

#	reader	what it got wrong	now
1	the SLOT object, <code>switchIn</code> 's copy	held the sheet's item forever	an accessor
2	<code>dmgMon</code> , <code>board.js:1163</code>	every damage feature MAG scores with , and every playout body	reads the accessor
3	<code>monSpeedMult</code> , <code>board.js:2121</code>	the Choice Scarf $\times 1.5$ — CLAUDE.md's own example	reads the accessor
4	<code>rollout_leaf.sideTeam</code> / <code>sideFallen</code>	the synthesised BENCH body read <code>board.sheet</code> directly	calls <code>board.sheetItem</code>
5	<code>board.js:3306</code> , <code>benchRisk</code> 's foe-bench bodies	same — a benched foe whose Sash was gone still priced as holding it	calls <code>board.sheetItem</code>

Two readers were checked and are CORRECT, and saying so is the point of a sweep.

`board.switchFeatures` and `engine/position_features.js` already went through `sheetItem` , which is why they are the model this fix copied rather than something to repair. `miltank.js:674` 's **team-preview builder reads the sheet and is also right**: at preview nothing has been knocked off, so the sheet IS the observed item. A fix applied there would have been wrong.

The proof, red first

`tests/test-seed-residue.js` was run before the fix existed and again against a **deliberate break** of the accessor (`get: () => mon._itemAtEntry` , the exact old semantics), which is the honest red because two arms were added after the first run:

run	result
before any fix	10 ok / 7 FAIL (aborted in the arm being written)
deliberate break of the accessor	13 passed / 7 failed
after	20 passed, 0 failed

Every failure was a reader holding an item the game removed. **Every CONTROL was green in the same run**, which is what says the file cannot pass by simply emptying every item: a body with no item event keeps what it declared, a body the sheet declares itemless starts with nothing, a body with NO sheet still passes `undefined` rather than `''`, and an item **gained** mid-battle (Trick, Symbiosis) reaches the same readers.

Nothing in it is typed. The damage-changing item is found by probing every legal item in the regulation through the engine's own `dmgRange` until one moves the number (it lands on Life Orb, and would still be right if it did not). The speed-changing item comes out of the dex's own `onModifySpe` handler (Choice Scarf). The species pool is `MC.mons` intersected with `Dex.forFormat` filtered on `isNonstandard / tier`. And the **attacker/defender pair for the speed arm is searched for**, because a Choice Scarf only moves `movesFirst` when it actually flips the order — a hardcoded pair would go silently inert the day the dataset's speeds moved.

One arm carries no assertion and says so. `benchRisk` did not move for any survival-changing item on the fixture pair, so reader 5 is routed through `sheetItem` **by construction** and that is stated here rather than claimed as tested. Reporting it is the alternative to a green row that proves nothing.

Does it change decisions? MEASURED — and this one is SMALLER than the seed batch

`data/rollout-item-prevalence.json`, over **14,288 open-sheet bo3 games / 192,912 decision points**. It reads the store and `data/tags.json`, **plays no game and opens no** `Dex` — no `battleInit`, no rollout, no board — so it is not downstream of MEDICHAM, is not quarantined, and needed no engine release with four agents in the tree. Same denominator as `data/rollout-seed-prevalence.json` and `data/rollout-fallen-prevalence.json` by construction.

	share of decision points
an ACTIVE body is priced holding an item it does not hold	3.197%
a live BENCHED body is	0.667%
ANY body that reaches a feature or a seed — the reach of this fix	3.622%
(context) decision points occurring after any item-affecting click	5.959%
(context) games containing at least one such click	10.631%

2,405 item-affecting clicks over the store. Every move is enumerated from the tag artifact — `removesItem`, `takesTargetItem` (whose params carry the three-way split between destroy, steal and **swap**) and `flingsOwnItem` — so a move a future regulation adds is counted with no edit here.

THIS IS A FLOOR, AND FOR A BIGGER REASON THAN THE USUAL CAVEAT. The store records **no item consumption at all**: a spent Focus Sash, an eaten Citrus Berry, a detonated Air Balloon and a used Weakness Policy each emit `|-enditem|` on the live wire and *nothing* in a stored game. Neither Symbiosis, Pickpocket nor Magician has an observable trigger there. **The live bot sees strictly more than 3.622%**, and how much more is not knowable from the store. It is also a **ceiling on decisions** in the other direction, stated as one: it counts positions priced wrongly, not argmaxes that flip.

(`mineStale` and `foeStale` come out identical at 1.881% by construction, not by coincidence — every turn contributes a decision point for both sides, so "my side is wrong" for p1 is "the foe is wrong" for p2.)

IS THE FIT INVALIDATED? NO — AND THAT WAS MEASURED, NOT ARGUED

The roadmap row filed this as **fit-invalidating** and that turns out to be wrong, for a reason worth writing down because it is itself a defect one row over:

`board.noteItem` **has exactly one caller in the whole repository** — `engine/magnemite.js:574`, **the live protocol reader**. The fitter never calls it. So offline `itemNow` is empty on every board, `sheetItem` returns the sheet's declared item, and the accessor returns **the same string the copy did**. `setSheet` also precedes

switchIn in every offline path (fit_policy.js 625 before 640/784, feature_coverage.js , forced_switch_audit.js , feature_fixture.js), so the live sheet read cannot differ from the entry snapshot either.

Checked rather than reasoned about: engine/feature_fixture.js hashes every feature's column separately over 12 frozen scenario boards, precisely so a changed MEANING under an unchanged NAME is visible. Run with the fix and run again against the deliberate break, the 58 per-feature hashes are **byte-identical**. data/policy-weights.json keeps its meaning and **no refit is owed by this row**.

(node engine/feature_fixture.js --check data/policy-weights.json fails for an unrelated and pre-existing reason — the stamped fixture is 10 scenarios and the code's is 12 — so the comparison above was taken between two runs of the CODE, which is the question this asks.)

What IS owed is one row over and it is MEASURE's: PRIORITIES 13e. The fit never sees an item event at all, so a declared item stands for the whole of a stored game while the live player now tracks it correctly. That is the fitting-environment-and-playing-environment mismatch CLAUDE.md names, still open, and #271 does not close it — #271 fixes the READER, 13e is the missing offline EVENT.

Adjacent gates, run and reported

Green: tests/test-rollout-seed.js (47/47), tests/test-rollout-fallen.js (28/28), tests/test-rollout-switch.js (16/16), tests/test-rollout-gates.js (16/16), tests/test-pp-fact.js (33/33), tests/test-feature-antics.js (18/18), tests/test-engine-consistency.js , tests/test-switch-features.js , tests/test-switch-carry.js (27/27), tests/test-board-browser.js (14/14), tests/test-hazard-side.js (11/11), tests/test-forced-switch.js , tests/test-miltank-release.js (25/25), tests/test-wiring.js , tests/test-artifact-keys.js (4/4), tests/test-provenance-discovery.js , tests/test-timestamps.js (6/6), tests/test-fixture-legality.js , tests/test-docs-current.js (21/21), tests/test-roadmap-register.js (3/3).

tests/test-rollout-switch.js and tests/test-pp-fact.js are the real control again: both seed boards and assert exact, dice-for-dice win probabilities, which is only possible if a board with no item event is byte-identical to what it was.

tests/test-stadium-roster.js **was RED on this work and is GREEN:** the new prevalence generator was undeclared and now carries a NOT_A_MODEL entry with its reason. It was the only gate this work turned red, and it was fixed rather than filed.

Two gates are RED, neither is this work's, and this work contributes ZERO rows to either.

Verified by name, not by assertion. tests/test-rollout-effects.js is 41 passed / 2 failed on Full Metal Body and Guard Dog refusing an Intimidate drop — identical to R14's reading, and it is ENGINE's.

tests/test-effective-identity.js is 18/1 at **1,470** raw reads against a 1,198 baseline — the same 1,470 R14 reported, and neither new file appears in its list. tests/test-no-silent-failure.js is red at **80** NEW silent catches, which is R14's number exactly: the six catch blocks in the new gate were flagged at first, and they now push the probe and the reason onto a list the run prints, so the count came back to 80 rather than being re-baselined.

What was deliberately NOT done

- **No SPRT was prepared and no leaf value was re-measured.** #267, #268, #269 and #270 are still open on the same surface, and a leaf number taken between them describes a build nobody will ship. The paired argmax run belongs after the residue is closed.
- **No refit was run.** MAG's weights are Will's and he is reworking them; the measurement above says none is owed by this row in any case.
- engine/medicham2-browser.js **was not touched.** Nothing in this row needed it.

- **The other four rows of the sweep were left alone**, and the gate reports them as `note` lines rather than red `ok` lines — a red row nobody is fixing in this pass is the "KNOWN FAILURE" shape this repository bans by name.

One thing seen in passing, reported and not touched

`tests/test-seed-residue.js` is the file left behind by the previous, interrupted attempt at this row. **It is kept and finished**, rescoped from all five sweep rows to #271 alone with the other four as reported notes. Nothing was deleted.

R14 — THE SEED WAS WRONG OFTEN ENOUGH TO CLOSE FOUR ROWS; THE HEADLINE SHARE IS WITHHELD. 2026-08-13; five more filed.

ROADMAP #247, #248, #249 and #250 closed — one batch, because they are one surface and fixing them piecemeal makes each result unattributable. Gate: `tests/test-rollout-seed.js` (47 assertions, auto-discovered by `tests/run-all.js`), shown **RED first at 23 passed / 24 failed**. Prevalence artifact: `data/rollout-seed-prevalence.json`, written by `engine/rollout_seed_prevalence.js`. ROADMAP #267, #268, #269, #270, #271 opened by the sweep that followed.

The verdict in one line

The seed handed MEDICHAM a position that differs from the real one often enough to close all four rows, and it now hands over the truth on all four counts — but #271, found in the same sweep, is bigger than any of them and is not SEARCH's to fix.

THE SHARE THAT USED TO BE IN THIS SENTENCE AND IN THIS SECTION'S HEADLINE IS WITHHELD as of 2026-08-23: the generator's species lookup dropped every forme, which could only inflate it. The mechanism, what it does and does not touch, and the pinned command that would restore it are [below](#). Four of the six prevalence figures are unaffected and stand.

Why these four were one batch

Will, 2026-08-13: *"miltanks rollout needs to just play the game out on medicham and have it match showdown perfectly thats the whole point. miltanks just chooses the actions."*

That sentence makes the **seed the only place a correct simulator can still produce a wrong game**, so every approximation on it is a defect by definition rather than a tradeoff. All four were found by sweeping the seeding path while #244 was being closed; all four sit inside two functions; and each one changes what the leaf sees, so a single arm measured after four separate landings could not say which one moved it.

What each one was, and what closed it

row	the seed said	the position says	closed by
#248	a benched body is whole, unstated, and carries the dataset's four moves	it is at 20% and burnt, and its sheet declares four specific moves	<code>sideTeam</code> passes the sheet's <code>moves</code> ; <code>board.switchIn</code> remembers the outgoing body's hp/status in <code>lastSeen</code> , <code>benchState()</code> reads it back
#250	every body may Fake Out	a body that has taken a move action may not	<code>board.noteMove</code> counts <code>moveActs</code> ; <code>seedHistory</code> copies it to <code>_mvActs</code>
#249	no hazards, no screens, no Gravity	Reflect is up, the rocks are down	<code>rollout_leaf.applySideState</code> , reading the board's own per-side record
#247	Supreme Overlord entered with nobody dead	it walked in over two graves	<code>board.switchIn</code> stamps <code>enteredWithFallen</code> ; <code>seedHistory</code> copies it to <code>_fallenStuck</code>

Does it change decisions? MEASURED, and the answer is not uniform

data/rollout-seed-prevalence.json , over **14,102 open-sheet bo3 games / 190,378 decision points**. It reads the store and data/tags.json , **plays no game** — no battleInit , no rollout, no board — so it is not downstream of MEDICHAM, is not quarantined, and needed no engine release even with two agents rewriting the simulator underneath it. That is exactly why it is the measurement that was taken. Same denominator as data/rollout-fallen-prevalence.json by construction, so these numbers are comparable to #244's 8.75% rather than five islands.

	share of decision points
#248 a live benched body's declared moveset differs from the dataset's four	WITHHELD — see below
#250 a first-turn-only move was offerable in the seed and the game refuses it	17.655%
#248 a live benched body is hurt or statused	17.053%
#249 a hazard, a screen or Gravity is up	13.089%
#247 a fallen-count carrier entered over graves	0.061%
ANY of the five — the ceiling on this batch's reach	WITHHELD — see below

TWO OF THESE SIX ARE WITHHELD, 2026-08-23 — ROADMAP #402. THE OTHER FOUR STAND.

The claim. The #248 moveset row and the ANY of the five row are **WITHHELD**. They are not annotated, not footnoted and not reproduced anywhere in this file, because a figure printed with a caveat gets quoted without it. The four rows left standing are unaffected: datasetMoves is called at exactly one site in the generator (engine/rollout_seed_prevalence.js:175) and feeds only movesDiffer , and ANY is an OR that contains movesDiffer .

The mechanism. The generator read MC.mons[base(sp)] , and base() ends in norm() , which strips the hyphen MC.mons keys **every forme** with. So Rotom-Wash asked the table for rotomwash , got nothing, and was handed an **empty** dataset moveset — which the caller then compares against the sheet's declared four and scores as "the moves differ." A miss can only push movesDiffer **up**. Measured over data/games.bo3.jsonl : **47 of 256 store species names, 10,980 of 144,260 brought bodies (7.61%)**; at the sheet-body level **7,177 of 231,924 bodies (3.095%) flip true → false** once the row is actually found. The lookup is fixed and goes through engine/mc_key.js ; data/rollout-seed-prevalence.json was generated before the fix.

The direction is known and the magnitude is not. Both figures are biased **upward**. No corrected value is stated here, and none should be estimated — the body-level flip rate does not translate one-for-one to a decision point, because the decision point ORs over the whole live bench.

The command that settles it — and the sample must be pinned.

```
head -n 14102 data/games.bo3.jsonl > data/_seed-prev-pin.jsonl
node engine/rollout_seed_prevalence.js --store data/_seed-prev-pin.jsonl
```

The store has grown **14,102 → 19,401 games** since the artifact was written on 2026-08-14, so an unpinned re-run answers a different question and is not a before/after. The pin has its own check: a correct pin must return decisionPoints: 190378 exactly, which the bug cannot have touched.

#247 IS CORRECT AND VERY NEARLY INERT, AND THAT IS STATED RATHER THAN DRESSED UP. Only 201 of 190,378 decision points have a Supreme Overlord carrier active at all: Kingambit is real in this format and is almost always brought with Defiant. 116 of those 201 walked in over graves, so the mechanic is right when it fires and it hardly ever fires.

EVERY FIGURE IS A CEILING, and it is stated as one. It counts positions where the seed handed over a state that is not the real one, not decisions whose argmax flips. The paired argmax run is a frozen release and a common-random-numbers comparison, and it was not taken — see the closing note.

Three decisions inside the fix that were load-bearing

1. `turnsActive` **was the wrong quantity, and the roadmap row proposed it.** #250 says "`board.js` tracks `turnsActive` and the seed does not pass it." Passing it would have been wrong in **both directions**: `endTurn` counts every turn a body is on the field, so a LEAD reads 1 on the turn it may still Fake Out, and a body that entered MID-TURN reads 1 on the very turn it came out to use the move — which is when Fake Out is actually clicked. Showdown's quantity is `activeMoveActions`, incremented at the top of `runMove`, so the count is now taken in `noteMove` where a move actually happens. It needs no assumption about when an adapter calls `endTurn`, and that assumption is what made `turnsActive` unusable.
2. `_mvActs` and `_fallenStuck` **are seeded in `buildSide`, NOT in `dmgMon`, and that is not style.** `dmgMon` is `board.js`'s own builder and every damage FEATURE goes through it. `_fallenStuck` multiplies base power inside `dmgRange` and `_mvActs` is read by `bestMoveVs`, so putting either one in the shared builder would have moved fitted feature values and silently invalidated `data/policy-weights.json`. These are facts inside the **playout**; they belong to the seed. `FEATURES` is still 58 and the weights keep their dimensionality.
3. **#249's side is READ, never re-derived, and that is the trap the brief warned about.** ROADMAP #254 resolved whose side a condition lands on **once, at the WRITE**, in `board.sideFor` — of the 11 legal side-condition moves seven are `allySide` and four are `foeSide`. So `applySideState` takes the board's per-side record straight and calls `sideFor` **nowhere**. A second flip would have re-introduced #254 one layer up while looking exactly like a fix, and it would have cancelled invisibly if only one seat were tested. The gate therefore asserts placement **from both seats** for all four hazards, plus a behavioural arm.

The proof, red first

`tests/test-rollout-seed.js` was run before a byte of the fix existed: **23 passed, 24 failed**, every failure one of the four defects reading its floor — bench moves = the dataset's four, bench HP = full, `_mvActs` = 0 with the flinch landing on a body six turns out, `sfA.hz / sc / gravity` empty, `_fallenStuck` = 0 with three allies buried. **Every control was green in the same run**, which is what says the file cannot pass by inventing state. After: **47 passed, 0 failed**.

Nothing in it is typed. Species come from `MC.mons` **intersected with the regulation** (`Dex.forFormat(...)` filtered on `isNonstandard / tier`); a Fake Out user is a species **observed clicking it** on the ladder; the Supreme Overlord carrier is derived from the dex's ability table (exactly one, and the file asserts that); hazards, screens, Safeguard and Gravity come from their tags; and the two damage observables are **inverted out of control tables asserted injective before use**.

Two observables were deliberately chosen to be behavioural rather than field reads:

- **#250 is the FLINCH**, read out of a played turn's trace. Reading `_mvActs` back would only prove the seed wrote what the seed wrote.
- **#249 has a switch-in arm**, and its first version was wrong in a way worth keeping: with one slot pivoting per side the partner that stayed **attacked**, so the incoming body lost HP for a reason that was not the hazard — a green row for the wrong reason on one side and a red control on the other, while the seed was still completely empty. All four slots pivot now, nobody attacks, and any HP that moves is entry damage.

Adjacent gates, run and reported

Green: tests/test-rollout-fallen.js (28/28), tests/test-rollout-switch.js (16/16), tests/test-rollout-gates.js (16/16), tests/test-pp-fact.js (33/33), tests/test-forced-switch.js , tests/test-hazard-side.js (11/11), tests/test-feature-semantics.js (23/23), tests/test-engine-consistency.js , tests/test-switch-features.js , tests/test-switch-carry.js (27/27), tests/test-board-browser.js (14/14), tests/test-miltank-release.js (25/25), tests/test-wiring.js , tests/test-fixture-legality.js , tests/test-artifact-keys.js , tests/test-provenance-discovery.js , tests/test-timestamps.js , tests/test-roadmap-register.js .

tests/test-rollout-switch.js and tests/test-pp-fact.js are the real control here: both seed boards with nothing up and both assert exact, dice-for-dice win probabilities, which is only possible if a seed with no hazards, no bench state and no move history is byte-identical to what it was.

tests/test-stadium-roster.js was RED on this work and is GREEN: the new prevalence generator was undeclared, and it now carries a NOT_A_MODEL entry with a reason. It was the only failure — the six from R13 have since been declared by their owners.

Two gates are RED, neither is this work's, and this work contributes ZERO rows to either. Said plainly rather than filed. tests/test-rollout-effects.js is 41 passed / 2 failed on Full Metal Body and Guard Dog refusing an Intimidate drop — verified pre-existing by stashing this change and re-running: identical 41/2 — and it is ENGINE's. tests/test-effective-identity.js is 18/1 on the raw-read ratchet at 1,470 against a 1,198 baseline; the ten contributing files are named in its own output and none of them is board.js , rollout_leaf.js or tests/test-rollout-seed.js . tests/test-no-silent-failure.js is red at 80 NEW silent catches (ROADMAP #258) and grep of its output for this work's files returns nothing — the one catch added to the new gate names the species that would not build and prints it.

THE SWEEP — five more, and one of them is the biggest thing found tonight

Will's principle makes every approximation on this path a defect, so the path was swept again once the four were closed. Each is registered; none was touched.

#	what is still wrong	why it is not a detail
#271	a knocked-off item is still on the board's body	switchIn copies the item off the SHEET, noteItem writes only itemNow , and dmgMon reads mon.item . Measured: declare a Life Orb, noteItem(..., ''), and sheetItem says '' while dmgMon(...).item says lifeorb . So a Life Orb, a Choice Scarf, an eaten Citrus Berry and a spent Focus Sash keep applying — in MAG's damage features as well as in every playout. This is the CLAUDE.md lesson broken in the place that lesson was written about. Floor from the store: 2,375 item-removing clicks, 10.62% of games, 5.95% of decision points after one — and the live path also sees every consumption, which the store does not record
#270	the seeded field has no clock	applyField never sets weatherT / terrainT , and the engine's tick is if (weatherT > 0 && -- weatherT <= 0) — zero means never expires. A sun with two turns left runs for sixty. applyMegaWeather , one function away, sets it correctly, so the seed's two weather paths disagree. The terrain half is on the board today (fieldLeft); the weather half is not (setWeather records no duration)

#	what is still wrong	why it is not a detail	
#269	every durable volatile	choice lock, Encore, Disable, Taunt, the multi-turn lock, a charge turn, Substitute, Leech Seed, a partial trap, Recharge, the perish count. <code>magemite.js</code> already writes <code>board.startVolatile</code> from `	- start **with a duration** , so the live board holds an answer the seed throws away. First task is a **vocabulary check** , not a patch: <code>_vol</code> is writable from here, but its keys come from a tag param and the board's from <code>move.volatileStatus`</code> , and a mismatch seeds a volatile nothing reads — silently, exactly like the terrain dialect
#268	a permanent hazard is given a duration, and layers are not counted	Stealth Rock, Spikes, Sticky Web and Toxic Spikes carry no <code>condition.duration</code> ; <code>startSide</code> defaults an absent one to one turn offline and the live path to five. Measured on a fixture: laid <code>true</code> , one <code>endTurn()</code> later <code>false</code> . So <code>deadSide</code> returns to 0 and the model re-lays the rocks. Fit-invalidating , therefore not SEARCH's	
#267	a status is seeded and its counter is not	sleep turns and the toxic ramp. <code>medicham2</code> already models the split correctly (sleep carries a switch, the toxic ramp restarts); the BOARD records only the status name, so every playout gives a body two turns into a sleep a fresh one	

Two things deliberately NOT done

- **No leaf value was re-measured and no SPRT was prepared off it.** Two agents were rewriting `engine/medicham2-browser.js` while this landed (`data/tags.json` , `data/abra-tags.js` , `engine/tag_dex.js` and the census moved too), so a rollout measurement against the live tree is void by the rule that cost this project 7,100 games. The prevalence figure above is a store scan by construction and is the number that can honestly be taken today. **The paired argmax run belongs after #270 and #271 land**, because #271 changes what `dmgMon` builds for every body on the board and would move any leaf number taken before it.
- **#270 was found mid-batch and left open on purpose.** It is one fact whose weather half needs board work first, and landing half of it inside a four-row batch is precisely what makes the batch's own result unattributable — which is the reason these four were one batch.

R13 — THE ROLLOUT SEED NOW HANDS MEDICHAM THE DEAD. HALF FIXED 2026-08-13; the other half is one line in ENGINE's file.

ROADMAP #244. Gate: `tests/test-rollout-fallen.js` (27 assertions, auto-discovered by `tests/run-all.js`). Prevalence artifact: `data/rollout-fallen-prevalence.json` , written by `engine/rollout_fallen_prevalence.js` . ROADMAP **#246, #247, #248, #249, #250 opened**; #245 is ENGINE's and was left alone.

The verdict in one line

The seed was telling MEDICHAM that nobody had died, on 8.75% of open-sheet decision points, and it now tells it the truth from turn two. Turn one is still wrong and SEARCH cannot fix it.

The defect, and why the obvious patch was refused

`battleInit` derives **three** things from the **one** array it is handed:

<code>actA</code>	<code>teamA[0..1]</code> — the field
<code>benchA</code>	<code>teamA.slice(2)</code> — who can come in
<code>sfA.team</code>	<code>teamA.filter(Boolean)</code> — the roster, and the denominator of <code>fallenCount</code>

`buildSide` dropped every fainted body, which is right for the first two and deletes the side's dead from the third. `fallenCount` is `sf.team.filter(x => x.fainted).length`, so it returned a confident **0** — Last Respects at 50 where the position says 150, and a Kingambit whose Supreme Overlord snapshot is zero.

Threading a `fallen` count through `battleInit` would have made the card right and left the seed handing MEDICHAM a position that does not exist — a side of four where the real side has six, two of them dead — and it would be a second source for a fact the engine already owns. Will's statement of the design is what rules it out: *"miltanks rollout needs to just play the game out on medicham and have it match showdown perfectly thats the whole point. miltanks just chooses the actions."*

The design question, answered

Yes: `buildSide` 's drop stays right for the ACTIVE and BENCH arrays, and the roster keeps everyone. The corpses are **appended after the living**, which is the only shape that gets both, because the roster is not a separate parameter. Three things had to be true and each was checked rather than assumed:

1. **A corpse in `benchA` is inert.** Every bench reader in `medicham2-browser.js` goes through `_live` (`bringIn`, `switchOut`, `sidewiped`, the explore draw's `outs` filter) or targets a specific chosen body; `battleResult` sums `max(0, curHP)/st.hp`, which is 0. The only unfiltered walk over a bench in the whole file is `fallenCount` 's own fallback, which is what we want.
2. **A corpse in `actA` would NOT be inert**, which is why appending matters: it would put a dead body on the field and make the end-of-turn `_refills` issue its replacement a turn late. Appending also leaves every living body's index unmoved, so `oneMegaPerSide` 's first-mega rule, `rolloutAfterActions` 's slot-index click mapping and `bringIn` 's `surv` splice all behave exactly as before.
3. **The corpses are not only the fainted actives.** `sideTeam` never yielded a Pokemon that died and was replaced — it is gone from `slot()` and excluded from `bench()` — so the drop `buildSide` was making was the *small* half. The list comes from `board.graveyard[side]`, which `board.faint()` writes, deduped against a body still standing dead in its slot (the slot copy is preferred: it is the richer record, so a mega that died still spends the side's mega through `_pre`).

`dmgMon` carries `hp` across and leaves `fainted` alone — it had never been asked for a dead body — so `buildSide` stamps `fainted` **and** pins `curHP` to 0. Both: `_live` tests both and `fallenCount` tests only `fainted`, so a body with one and not the other is dead to half the engine.

One contract had to move with it. `rolloutWinProb` 's *"a side with nothing standing is not a 0%"* guard was `!mine.length`, and the array now legitimately contains bodies that cannot act; it counts the **living** now, so a wiped side still returns `null` rather than a confident 0. Verified.

The proof, red first

`tests/test-rollout-fallen.js` was run before the fix and reported **12 failures**, every one reading `fallenCount -> 0`, with the N=0 control green so the file cannot pass by inventing deaths. After the fix: **27 passed, 0 failed**.

The observable is Last Respects' base power, and it is **not typed**: `base` and `perFallen` are read out of `data/tags.json` at run time (50 and 50), and the power is **inverted** out of a control table computed through the engine's own `dmgRange` at every `k` — 24 / 46 / 68 / 90 / 113 / 135 damage, which the test asserts is

injective before using it as a lookup.

allies already dead in the seeded position	before	after
1	50 BP	100 BP
2	50 BP	150 BP
3	50 BP	200 BP
0 (control)	50 BP	50 BP — unchanged, so the fix does not invent deaths

Adjacent gates run and green: `tests/test-rollout-switch.js` (16/16), `tests/test-pp-fact.js` (33/33), `tests/test-forced-switch.js`, `tests/test-miltank-release.js` (25/25), `tests/test-engine-release.js` (66/66), `tests/test-provenance-discovery.js`. The first two are the real control — both seed zero-dead boards and both assert exact, dice-for-dice win probabilities, which is only possible if the living pass is byte-identical to what it was.

Does it change decisions? YES, and the size is measured rather than asserted

`data/rollout-fallen-prevalence.json`, over **13,592 open-sheet bo3 games / 183,840 decision points**. It reads the store and `data/tags.json` and **plays no game** — no `battleInit`, no rollout, no board — so it is not downstream of MEDICHAM, is not quarantined, and needed no engine release even with ENGINE editing the simulator underneath it. That is why it is the measurement that was taken.

a death has already happened on the acting side	50.83% of decision points
the acting side brought a fallen-count carrier	16.26%
both — the ceiling on what this fix can move	8.75% (16,082 of 183,840)
among carrier decision points	53.82%
games with a carrier on at least one side	30.07%

Conditional on being affected, the mean fallen count is **1.67**, i.e. Last Respects should have been priced at **133.5 BP on average and was priced at 50** — a factor of 2.67 on the base power of a move whose entire identity is that it grows as your team dies.

This is a CEILING, and it is stated as one. It counts positions where the leaf value must change, not decisions whose argmax flips. Whether the argmax moves is a paired run against a frozen release and it was not taken — see the next section for why.

The carriers are enumerated, never typed. `withTag('move','powerFromFallen')` and `withTag('ability','boostsFromFallen')` return exactly `lastrespects` and `supremeoverlord`, which are the only two things `medicham2-browser.js` reads the count for.

WHAT IS STILL WRONG, AND IT IS THE TURN THAT DECIDES — ROADMAP #246, HANDED TO ENGINE

`battleInit` writes the literal `sfA:{fainted:0,...}` and the **only** recount is `sfA.fainted = fallenCount(...)` in the end-of-turn block. So the roster is now correct and the count is still **0 at t=0**. Measured on the fixture at N=3: **50 BP where the roster says 200**.

That is the decision-relevant turn. `rolloutAfterActions` forces the candidate click on turn 1, so a Last Respects being *ranked* is priced at its floor; the fix is live from turn two onward, which is most of a playout and not the part being chosen.

The fix is one line, in `engine/medicham2-browser.js`, **and SEARCH may not make it** — an ENGINE agent is in that file. Derive the initial value where the literal is, after `S.sfA.team` is stamped, exactly as the turn-end block already does:

```
sfA:{fainted:0,...}
```

```
-> S.sfA.fainted = fallenCount(S.sfA, S.actA, S.benchA);  
S.sfB.fainted = fallenCount(S.sfB, S.actB, S.benchB);
```

It needs **nothing further from SEARCH**: the roster it would read is now correct, and `tests/test-rollout-fallen.js` §4 already prints the gap on every run so the day it closes is visible. Same family as #243 (the live count is one action late) and #245 (the guard built for this cannot see either, because it fires on an ABSENT roster and this one was pre-filtered — left alone, it is ENGINE's).

The sweep — four more approximations on the seeding path, FOUND AND NOT FIXED

Will's principle makes every approximation on this path a defect, so they were swept for rather than waited for. Each is registered; none was touched, because each changes what MILTANK clicks and owes its own arm.

#	what the seed pretends	why it is not a detail		
#247	every seeded body enters with <code>_fallenStuck: 0</code>	Supreme Overlord's snapshot is stamped in <code>bringIn</code> , and a body placed by <code>battleInit</code> never goes through it. #246 does not close this — the fact is not on the board at all, since <code>board.js</code> records who is dead and nothing about when each body entered relative to those deaths		
#248	a benched Pokemon is at full HP, unstated, and carries the dataset's moves rather than its sheet's	the board keeps no state for a body that is not on the field, and the bench synthesis passes <code>item / nature / pp</code> from the sheet but not <code>moves</code> . HP and status are public information in a real battle. The biggest of the four		
#249	no hazards, no screens, no Gravity	<code>applyField</code> translates four things (weather, terrain, Tailwind, Trick Room) and <code>sf.hz / sf.sc</code> start empty, while <code>board.startSide</code> has the rest. Chasing it found a defect underneath : <code>board.noteMove</code> starts every side condition on the MOVER's side, and <code>stealthrock / spikes / stickyweb / toxicspikes</code> are all <code>target: foeSide</code> (derived from the format, not recalled). The offline board — the fit's board — records every hazard on the wrong side; the live path reads `	- sidestart	` and is right
#250	every body can Fake Out	<code>firstTurnOnlyRefused</code> gates on <code>_mvActs</code> , which is 0 on a built body; <code>board.js</code> tracks <code>turnsActive</code> and the seed does not pass it. 16,871 corpus uses. Choice lock, Encore, Disable, Taunt, Substitute, Leech Seed and the perish count are unseeded from the same hole, and <code>protectTurns</code> is passed for my side only		

Two things that were deliberately NOT done

- **No feature was added.** `board.js` `FEATURES` is 58 and `data/policy-weights.json` keeps its dimensionality. Nothing MAG scores reads the fallen count; this is a fact inside the payout.
- **No leaf value was re-measured and no SPRT was prepared off it.** The simulator is being edited by ENGINE right now, so a rollout measurement against the live tree is void by the rule that cost this project 7,100 games, and a release cut mid-edit freezes bytes nobody chose. The prevalence figure above is the number that could honestly be taken today, and it is a store scan by construction. The paired leaf-and-argmax run belongs after #246 lands, because measuring the half-fixed seed would produce a number that describes a build nobody will ship.

Two gates were already red when this landed, and neither row is this work's

Said plainly rather than filed. `tests/test-stadium-roster.js` fails on six undeclared artifact generators (`divergence_report`, `million_run`, `mod_audit`, `open_work`, `residual_order`, `speed_vs_pokeenv`) — MEASURE and ENGINE own those files. `tests/test-effective-identity.js` fails at 1,456 raw reads against a

1,198 baseline, contributed by `all_mechanics_fire`, `board_state`, `derive_switch_carry`, `fixture_preflight`, `million_run`, `tests/roster.js`, `test-forme-assert` and `test-ohko-accuracy`. **This work added one row to each and removed both** — a `NOT_A_MODEL` declaration with a reason, and a `DECLARED` entry stating the construction (the prevalence scan holds no live body at all, so a stored sheet's `.ability` is the only thing there is to read). Neither gate was made to pass by re-baselining.

R12 — PP EXISTS IN THE ROLLOUT NOW. It did not, and every position started full. FIXED 2026-08-11.

Artifact: `data/pp-board-probe.json`, written by `engine/pp_board_probe.js`. Gate: `tests/test-pp-fact.js` (31 assertions, auto-discovered by `tests/run-all.js`). ROADMAP #145 closed, #146 opened against ENGINE.

This is a MECHANISM fix and it is not a measurement. No engine release was cut for it and the tree moved while it was made — `engine/medicham2-browser.js` was being edited by ENGINE the whole evening. The probe below is a receipt that a state is representable, not a leaf value, and it is not quotable as one. Everything genuinely downstream stays quarantined and is now further invalidated, which is the right order: **before the refit, not after.**

The defect, in one row

	before	after
a position that has already spent 8 of Protect's 8 PP	the rollout Protects 8 more — 16 out of a move that has 8	0 , and the body Struggles
a position that has spent 5	the rollout Protects 8 more, total 13	3 more, total 8
a position that has spent 0 (control)	8	8 — unchanged, so the fix does not deduct what was never spent

The cap on Protect in a board rollout was `already spent + 8`, **unbounded in the first term. It is now 8, full stop**, which is what `maxpp` says in this format (read off `data/tags.json`, built by `engine/tag_dex.js` from a real `Battle` in `gen9championsvgc2026regmb` — Protect is 8 here against 16 mainline, and the mainline `pp * 8/5` rule matches only **85 of this format's 500 moves**).

Why this is not a rounding error, given games end at turn 6

ROADMAP #38 measured median 6 turns over 53,059 stored games, and `maxTurns` is **60**. That gap is the whole defect rather than a reason to shrug at it: a 60-turn payout with infinite PP can discover unlimited Protect, unlimited recovery and unlimited redirection, and those lines are precisely where the search believes it has found something. The simulator's own header said so before this was fixed:

"WHAT IS NOT CLAIMED: a rollout *STARTS* at full PP, because `board.js` does not track PP and is not ENGINE's to change. So a stall priced 8 turns deep inside a rollout is 8 turns from NOW, not 8 from the start of the game."

Where the state lives, and the one thing that was easy to get wrong

`Board.pp` is **per side and per SPECIES**, not per slot. `switchIn` builds a brand-new mon object every time a Pokemon comes out — correct for stat stages, which belong to the slot's occupant — and a PP table held on that object would have **silently refilled every move on a pivot**. Pivoting is what the stall lines this fix exists to price are made of, so that version would have looked like it worked and fixed nothing. The mon holds a REFERENCE to the side's ledger; the ledger outlives it.

Three call sites, and each one's position is the mechanic:

- `noteMove` **spends it, ABOVE the** `worked` **gate**. Showdown deducts inside `runMove` below every `BeforeMove` refusal and above the `|move|` announcement, so a Protect that fails its own consecutive-use

roll, a move that missed and a move a Protect ate have all been paid for. Only a click that never executed is free, and those emit `|cant|` rather than `|move|`. Charging on `worked` would have refunded every failed stall — the one case this fix is about.

- `dmgMon` **seeds the built body, AFTER the sheet overwrites** `b.moves`. `buildMon` fills the dataset's representative four and the sheet's four replace them; seeding first would key the table to moves the body no longer has, silently.
- `rollout_leaf.sideTeam` **gives the BENCH its ledger too**. A benched Pokemon is synthesised from a species name with no live object, so it is the one that arrives at full PP most easily — and it is the one that spent six turns on the field before it pivoted out.

Measured over the fitter's own corpus replay (`engine/feature_coverage.js` , which calls the same `noteMove` every offline adapter does): **9,502 PP spent, 0 moves with no pp row, 0 artifact lookups failed**. The wire is not inert.

A second defect fell out, and it was in this division's own file

At the shipped `explore=1.0` **the payout was clicking empty slots**. `runPlayout`'s uniform draw bypasses `chooseAction` — which is the entire point of it — and `chooseAction` is also where every selection guard lives, including ENGINE's new empty-slot refusal and the Struggle branch under it. So a drained body answered `|cant|nop` at execution and **wasted the turn instead of Struggling**: 52 `|cant|nop` lines and 0 Struggles on the probe board. The draw now filters on selectability and returns null when nothing is selectable, which hands the body back to the chooser and produces a real Struggle. `pickByPrior` takes the filtered list too, or the priors sampler would put the empty slot straight back — the identical leak WIRE 26 found on Disable, one layer up.

What this deliberately does NOT do

- **No feature was added**. `FEATURES` is still 58 and `data/policy-weights.json` keeps its dimensionality. Nothing MAG scores reads PP.
- `candidates()` **still offers a drained move**. Showdown would not, so this is probably wrong — and fixing it changes what MAG clicks, which makes it a SEARCH decision that deserves its own arm rather than a free ride on a mechanics fix. `Board.slotSelectable()` and `Board.mustStruggle()` exist for whoever takes that arm.
- **Pressure is charged only when it is unambiguous**. The extra is per APPARENT TARGET, so a self-targeting move pays nothing extra (measured: Protect goes 8 → 3 in five clicks against a Pressure foe and against a Levitate foe alike, while Flamethrower goes 16 → 6 against Pressure and 16 → 11 against Levitate). `noteMove` is not told which of two foes a single-target move hit, so the extra is charged when the caller names the target or when every live foe charges the same, and `ppCounters.pressureAmbiguous` counts the rest. Erring low keeps the board's PP an **upper bound** on the truth, which is the safe direction: it can only ever under-spend, never invent a turn. `magnemite.js` DOES have the target on the `|move|` line and could pass it — one argument, in a file this division does not own. Owed.

The FACT has two readers, and that is filed rather than glossed

`engine/pp.js` is the shared implementation. It exists because `medicham2-browser.js`'s five PP functions are **file-local** — on neither `module.exports` nor `root` — so `board.js` cannot call them, and exporting them is an edit in ENGINE's file that SEARCH may not make while ENGINE holds it. The alternative was a copy, which is the defect CLAUDE.md names by name. **ROADMAP #146 is the ask**: have those five delegate to `engine/pp.js`, or export them and let `pp.js` become a thin adapter.

Until then the window is guarded rather than declared. `tests/test-pp-fact.js` compares BEHAVIOUR, not source — it plays a real turn and asserts the number MEDICHAM wrote into its own `_pp` equals the number `pp.js` gives, at every maxpp tier the format produces (8, 12, 16, 20) — because comparing two

implementations by reading one of them proves nothing.

One measured correction to the ENGINE note beside this. docs/_outbox/pp-and-molddbreaker-notes.md says floor(base * 0.8) + 4 fits **500 of 500** rows. It fits **499**: Struggle carries noPPBoosts and stays at 1/1 where the formula gives 4. Nothing downstream is wrong, because the number is READ and never computed — but the claim as written would have to break before the artifact did.

ALL FOUR ROLLOUT GATES ARE QUARANTINED — 2026-08-08 (MEASURE)

R1, R2, R3 and R4 no longer print a number. engine/quarantine.js computes whether MEDICHAM is correct, and while it is not, every figure downstream of the simulator is **withheld rather than captioned**. The caption this replaces was live on the line above: R4 printed [engine moved since; transfer assumed, not measured] beside 55.5%, and 55.5% went on being quoted.

The 55.5% is not retracted. It is unquotable. It was a real measurement of a real build; the build does not exist any more, and the games it was measured on are PRE-CHANGE against a medicham2 that has moved repeatedly since. Re-running is not optional once the gate opens — a quarantined number does not become true when MEDICHAM becomes correct, it becomes **re-runnable**. That is ROADMAP #57.

What SEARCH gets back, and what it costs, per gate:

gate	withheld artifact	what re-runs it
R1 leaf accuracy	data/rollout-r1.json (and the explore1 arm)	node engine/rollout_r1_artifact.js over a fresh dump
R2 leaf cost	data/rollout-cost.json	node engine/rollout_r2.js
R3 divergence	data/rollout-r3.json	node engine/rollout_r3.js — and this time it must WRITE its control floor
R4 does it win	data/rollout-r4.json	node engine/rollout_r4.js over a re-played corpus

R1's shipped arm was nearly let through on a technicality, and that is worth knowing before the next gate is written. data/rollout-r1-explore1.json has no row in the derived artifact graph — nothing detects a writer for it — so asking the quarantine only about the file status.js prefers would have printed the headline while the incumbent arm beside it was withheld. The line now asks about both files. **A gate whose artifact has no detectable generator is invisible to every check in this repository,** not only this one.

The weights are quarantined too, and the refit stays OWED on purpose. data/policy-weights.json and the joint weights were fitted on features computed through the simulator, so mag.js , scoreboard.js , ladder.json and weight-multiplicity.json are held behind them transitively. Running the refit now would produce a vector fitted through the same wrong engine — it is gated behind MEDICHAM, not behind compute.

R11 — GARY, and the four things wrong with the imagined opponent. FOUND 2026-08-06, NOT MEASURED.

Everything below is read out of the source. Nothing here was run. It is filed the day it was found because the last time this division found a built-and-unwired capability it went unrecorded and a division ledger is the place that stops happening.

The opponent inside the search is a coin, by default, in the library and in the live bot.

```

engine/miltank.js:455     DEFAULTS = { defer: true, budgetMs: 20000, foePolicy: 'uniform', ... }
engine/mag_bot.js:173    const MILTANK_FOE = arg('miltank-foe', 'uniform');
engine/rollout_leaf.js:289 const mv = (foePolicy === 'prior' && pickByPrior(mon, rng)) || mvs[random];

```

'prior' samples `data/move-priors.json` — 128,548 recorded clicks over 295 species — and is wired end to end. Nothing turns it on. `engine/rollout_leaf.js:209` says outright that *"which is better"* was never measured.

Four defects, each its own task:

#	defect	evidence
#32	'prior' exists and is off	the two defaults above
#33	no artifact records which policy ran	<code>data/rollout-r1.json</code> , <code>data/rollout-r1-explore-sweep.json</code> have no <code>foePolicy</code> key
#34	the flag steers my side too	<code>rollout_leaf.js:302-303</code> — same <code>pick</code> over <code>S.actA</code> and <code>S.actB</code>
#35	the target is uniform in both modes	<code>rollout_leaf.js:290</code>
#36	two seats, two different opponents	<code>rolloutAfterActions</code> : <i>"The opponent is NOT modelled. It plays chooseAction during the stepped turn."</i>

#33 is the one that makes R1 and R4 harder to read than they look. Neither artifact states its opponent, so *"MILTANK beats MAG on 55.5% of 535 decisive pairs"* is a statement about an unrecorded configuration. That is not a retraction — the arms were paired and the comparison is internally valid — but the result cannot be transferred to a run whose GARY differs, and nothing currently prevents that transfer.

And the screen is run by the same coin

`engine/miltank.js:1204` evaluates every candidate pair with a **cheap rollout** and keeps the top `FINAL_K`. The rollout is the leaf that `data/winrate-backtest.json` measures, and **that measurement is QUARANTINED — withheld, not annotated**: the artifact is downstream of MEDICHAM (`engine/backtest_winrate.js` reaches `engine/medicham2-browser.js` through `require`), so no accuracy, no decisive-call count and no interval is carried here. It becomes quotable again when the gate opens AND this is re-run: `node engine/backtest_winrate.js`. **The structural point does not need the number: the same leaf is not only scoring the finalists, it is choosing them.** MAG is not used for the screen and `miltank.js:1008-1012` records why: `_candsFor` returns candidates **with no scores attached**, so an earlier top-K attempt silently sorted by array order — *"an arbitrary shortlist that LOOKS principled."* (#37)

This is the cheap fix, because MAG runs **once per turn** at the screen rather than once per imagined turn inside a playout. Measured branching, over 7,976 real brought-teams: ~76 action combos per side per turn, ~5,738 joint, against `ROLLOUT_N = 200`.

The horizon this division has been reasoning from is ~10× too long

Measured over 53,059 stored games: **median 6 turns, mean 6.5, p90 10, p99 16**; 0.05% exceed 30 and 0.01% exceed 60. `maxTurns` is **60**. Two consequences: a leaf evaluation is ~5,600 move-decisions rather than ~48,000, which puts MAG-as-GARY back on the table pending #39 (the `board.js` ↔ MEDICHAM translation cost, never measured); and **the value-net-first build order is wrong for this game** — at six turns a rollout reaches a real terminal state, so there is no position left to approximate. `docs/POKER-TO-POKEMON.md` §4b said this before today: *"the binding constraint is breadth, not depth."* (#38)

Correction to a claim made while diagnosing this: MAG is **not** deterministic. `greedy=false` draws from a softmax and `magnemite.js:217` calls it *"the single biggest measured lever in the project"*, so "sampling MAG would collapse the playout variance" is false and was never a reason.

R8 — WOBBUFFET re-run, 2026-08-04. VOID. THE TREE MOVED UNDER IT. DO NOT QUOTE THE NUMBERS.

The re-run was authorised ("*rerun wobba*" ... "*yes do the search once engine is all wrapped up*"), was executed at full size, and **produced no usable statement about MAG's exploitability**, because the two things it was measuring both changed while it was measuring them. The old 63.2% is retracted anyway — see below — so the net position is that **MAG's exploitability is now UNMEASURED**, which is a worse place than this session started but a truer one.

What moved, with times, because this is the entire result

what	when	why it is fatal
data/policy-weights.json — MAG itself — was REFITTED	generated: 2026-08-04T22:15:24.522Z	the search loaded the defender at 21:41 and froze it in a temp file; the held-out replay loaded it again at 22:17 , after the refit. The two legs defended with different vectors . New corpus stamp: 8,759 games / 229,339 decisions
engine/board.js written	22:50 → mtime 21:50:36	mid-search, around round 5. Every candidate is scored through <code>dmgMon</code> , so rounds before and after it are not comparable
engine/medicham2-browser.js — the simulator every score goes through	mtime 22:26:57, then four distinct content digests across three sampling windows : 0e4b2394edfc (22:29:04) → e9a4215e13d4 (22:30:34) → d1a4e497c0e9 (22:35:53) → moved again by 22:37:53	sampled with <code>run_stamp.sourceDigests()</code> , content and not mtime. It was still moving forty minutes after the run ended and while this section was being written
the ENGINE census	157/165 when this task was briefed → 164/171 in <code>status.js</code> at 22:31	ENGINE is mid-band, not wrapped

One thing IS stable and it matters for the re-run: `data/policy-weights.json` has held sha256 5a1930e8926af262 / generated 22:15:24.522Z since the refit, unchanged across 22:29–22:37. The defender is settled; the simulator is not.

`data/engine-release.json` **does not exist**. No release has ever been cut, so DIVISIONS rule 1 was unenforceable here in the same way it has always been unenforceable, and `exploit.js` stamps nothing at all — no engine digest, no target digest, no node version. It cannot detect any of the above and did not.

The brief said the engine was wrapped and committed at 96d82cb . That is not true of the working tree. Reported as observed rather than argued: two content digests ninety seconds apart, printed above. This is not a criticism of the ENGINE band's work — it is the reason the release boundary in Po.5 exists, and it has now cost a 7,100-game run.

The one thing that IS clean, and it is worth keeping

The mirror control at n=782: 49.7% [46.2, 53.2]. Both legs of the held-out replay ran inside one stable window (22:17–22:24: `board.js` stable since 21:50, weights stable since 22:15, `medicham2` not touched until 22:26), so this is a valid measurement of one build. It lands dead on 50, which retires a live worry: the 47.0% and 47.5% round-o mirrors in the two searches are **noise at n=217**, not a seat or pairing asymmetry biasing every other row. `mew.js` 's side alternation is doing its job.

The old 63.2% is retracted regardless, and not because of anything measured today

	2026-07-26	this run (VOID)
features	17	58
games/eval, rounds, seed	220, 18, 90210	220, 24, 90210

	2026-07-26	this run (VOID)
mirror control	47.5%, n=217	47.0%, n=217
best challenger vs MAG	63.2% [56.6, 69.3]	~~55.8% [49.1, 62.3]~~
held-out replay at unseen seeds	never done	~~45.8% [42.3, 49.3], n=782~~

The 63.2% describes a **17-feature** vector on an engine 25 wire-fixes old, computed **before the quality filter existed** — which is exactly why `provenance.js` called it its only `UNSAFE` artifact. It cannot be quoted whether or not a replacement exists, and `docs/MODELS.md` calling it *"the most important number in the repo"* is no longer supportable. **There is now no exploitability number for this project.** That is the honest state.

Two findings that survive the invalidation, because they are about the TOOL

These are properties of `exploit.js` 's search dynamics and of `provenance.js` 's check. Neither depends on which vector was being attacked, so neither is voided by the tree moving.

Finding 1 — THE ATTACK DIED, and a dead search cannot distinguish "safe" from "unsearched"

This is the caveat that matters and it is not the tool's disclaimer, it is a defect in the search:

	2026-07-26	2026-08-04
dimensions searched	17	58
steps ACCEPTED	10 of 18	1 of 24
step scale at the last round ($0.6 \times 0.85^{\text{failures}}$)	0.164	0.0168

`exploit.js` perturbs every coordinate by `gauss() * scale * (|v| + 0.25)` and multiplies `scale` by 0.85 on **every** failure. In 17 dimensions enough steps landed to keep the scale alive. In 58 the step *norm* is $\sqrt{(58/17)} \approx 1.85\times$ larger for the same per-coordinate scale, so round 1 threw the vector off a cliff — **27.7%**, the worst evaluation in either run — and then the geometric decay ran essentially unopposed. From about round 10 onward the challenger was a near-copy of MAG and the "search" was re-measuring the mirror control twenty more times. The 45–50% cluster in rounds 8–24 is that, not evidence.

So even on a still tree this run could not have proved MAG is hard to exploit. A search that takes one step is not a lower bound on anything. The tool's own closing text says the right thing — *"read it as 'this cheap attack failed', nothing more"* — and this time that sentence is doing real work rather than being boilerplate. **Fix the step rule before spending another 7,100 games**, or the re-run on a frozen release will return the same uninformative null for the same reason.

Finding 2 — `provenance.js` CLEARED THIS ARTIFACT AND IT SHOULD NOT HAVE. FILED FOR MEASURE.

Provenance now prints **o UNSAFE** and lists `exploitability.json` as `ok`. That is a false clear, and the mechanism is exact and reproducible:

<code>exploit.js</code> read <code>data/policy-weights.json</code> at	21:41 (module load)
<code>data/policy-weights.json</code> was REFITTED at	22:15:24.522
<code>exploitability.json</code> was written at	22:17:57.624

The check is `mtime(artifact) < mtime(input)`. The artifact is newer than its input by **153 seconds** and passes — while having been computed from a version of that input which is **34 minutes older**. `provenance.js` is mtime-based and structurally cannot see this; `CLAUDE.md` already says *"neither can catch an artifact that records a corpus it did not use"*, and this is the sharpest instance of it the project has produced, because the false clear is what *removed the last UNSAFE row*. **The fix is not in provenance.js**

— it is that a generator must stamp the **content digest** of every input it read, at the moment it read it, and provenance must compare digests rather than timestamps. `engine/run_stamp.js sourceDigests()` already does exactly this for the leaf sources.

Consequence for anyone reading the gate: `provenance.js --strict` **will now pass, and** `data/exploitability.json` **is still not quotable.** Do not treat the green as the answer.

What exploit.js needs before it is re-run. SPECIFIED, NOT APPLIED.

`engine/exploit.js` is not in `docs/DIVISIONS.md` 's ownership table and it produces a claim about whether a number is true, so the fix is proposed here and not made mid-result. Five defects, all observed in this run:

1. **IT STAMPS NOTHING.** No engine digests, no digest of the target vector it read, no node version, no machine, no pool size, no `n_measured / n_unit`. Every other gate in this project carries a stamp and PRIORITIES #20 exists because two of them were missing two fields. This one has none, which is why a mid-run refit of its own defender was invisible. **This is defect zero: fix it first, because it is what would have aborted this run at round 23 instead of after it.**
1. **The step scale is hardcoded** (`let scale = 0.6`) and there is no floor. Expose it, scale the per-coordinate size by `1/√d` so the step *norm* is dimension-invariant, and floor the decay.
2. **Rounds are compared unpaired** — each evaluation uses a different seed (`SEED0 + r*7919`), so a step is accepted on a difference whose standard error is ~4.7 points at n=220. Common random numbers across candidates, exactly as `miltank.js` already does for post-KO replacement, would make a 220-game comparison mean something.
3. **There is no held-out confirmation phase.** The winner should be replayed at fresh seeds and that number, not the selected max, should be the artifact's headline. The scratch generator used here is not in `engine/`, so `provenance.js` does not enumerate `data/exploitability-holdout.json` at all — the confirmation belongs inside `exploit.js` as a `--confirm` phase.
4. **The team pool is not frozen across evaluations.** Each round is a fresh `mew.js` process that rebuilds the pool from the live store, and OPS ingest landed mid-run: `data/games.ladder.jsonl` was written at 22:03:28 UTC, between rounds ~13 and ~15, and the announced pool moved 7,264 → 7,341 **distinct clean teams** by the time the held-out replay ran. Both legs of the held-out replay used one snapshot (7,341, announced identically), so *that* comparison is internally clean; the 24-round search is not exactly reproducible from its seed. `MEW_TEAMS` and `engine/mew_farm.js` exist to pin this and were not used.

The corpus and the flags, recorded rather than implied

- **Pool: 7,264 distinct clean teams** at the start of the search, 7,341 at the held-out replay, drawn by `mew.js` through `engine/quality.js loadGames()`. **The quality filter was on** — it is not opt-in, `MEW_TEAMS` was unset, and `--meta-teams` was NOT used, so this is the full clean pool including the Mickey Mouse teams §3 warns about. Clean ladder games available: 7,228 → 7,316.
- Showdown checkout at the pinned commit `20ad99ffc9a5`, announced by every `mew.js` process.
- Defender = `data/policy-weights.json`, shipped: `reweighted_to_closed`. Verified before the run that the top-level `weights` array is **byte-identical** to `weights_reweighted_to_closed` and is the array `magnemite.loadWeights` actually reads, so the defender is the real shipped MAG. That check is not idle: `exploit.js` reads `weights` while `magnemite.js` also reads `weights`, but the file carries three vectors and only one of them ships.
- `--policy score --policy2 score`, both arms MAG's machinery, sides alternated inside `mew.js` by `swapped`. **MILTANK was not involved: this is a measurement of MAG, not of the search.**
- `exploit.js` 's **challenger is arm 2** (`--weights2`), the **OPPOSITE of the SPRT convention** where arm 1 is the challenger. Its seat attribution was re-derived against `mew.js:502-537` before the run and is

correct. `tests/test-sprt-arm-sign.js` passes 12/12 and pins the *other* convention — it says nothing about this file.

- Cost: 25 evaluations × 220 games = 5,500 games, ~36 min, one node process at `--conc 6` (the concurrency is hardcoded in `exploit.js`). Held-out: 1,600 games, ~7 min. **~7,100 games and ~45 minutes total. The run is cheap; that is the good news about having to repeat it.** The reason to re-run is the moving tree and the five defects, not the price.

THE RE-RUN. PREPARED, NOT LAUNCHED — and R9 says DO NOT RUN IT IN THIS SHAPE.

***Superseded in part by R9 below.** The five defects listed above are now fixed in `engine/exploit.js` and defect 0 is closed. The preconditions and the command below are still the right ones. **But the probe says a 24 x 220 search over 58 weights closes 0.0% ± 0.1 of the distance to a known planted optimum, so running it would buy another uninformative null for another 7,100 games.** Run it only as a deliberately-labelled negative control on the new tooling, or reduce the challenger family to 4–8 numbers first. Read R9 before spending anything.*

Three preconditions, in order, and **each one failed during the 2026-08-04 attempt**:

1. **ENGINE has actually stopped**, verified by content and not by anyone saying so: `` ` node -e "const RS=require('./engine/run_stamp.js');const a=JSON.stringify(RS.sourceDigests()); setTimeout(()=>{delete require.cache[require.resolve('./engine/run_stamp.js')]; const b=JSON.stringify(require('./engine/run_stamp.js').sourceDigests()); console.log(a===b?'STILL':'MOVING - do not start');},120000)" ` ``
2. **A release is cut on the CORRECTED engine** — the 68 interaction disagreements and the 7 missing mechanics closed — and `exploit.js` is pointed at it. `data/policy-weights.json` is inside the release, so the defender freezes with it and precondition 2's old "record its sha before and after" is now enforced by the tool rather than by a person remembering. `` ` node engine/engine_release.js cut "post-interaction-matrix engine, for the WOBBUFFET re-run" node engine/engine_release.js list # must read: 0 of 12 files have moved since ` ` **Cutting is Will's call**, and SEARCH does not cut: the pointer is shared and MEASURE is currently measuring against d3d04b669e18`.`
3. **The team pool is pinned**: build it once with `engine/mew_farm.js` and export `MEW_TEAMS`, so all 26 evaluations draw the same population and the seed reproduces. `exploit.js` now records the announcement of every evaluation and **writes itself** `void: true` **if the pool moves**, so this precondition is checked rather than assumed — but it still cannot pin it.

Then, one process. **Read R9 first — this is a negative control until the challenger family shrinks.**

```
export SHOWDOWN_PATH=C:/Users/willj/Projects/Pokemon/pokemon-showdown
export MEW_TEAMS=$PWD/data/mew-teams-wobbuffet.json # built once by engine/mew_farm.js
node --max-old-space-size=2048 engine/exploit.js \
  --games 220 --rounds 24 --seed 90210 --confirm 800 --conc 6 \
  --release <the id printed by the cut> --tag wobbuffet-e2
```

- **Cost: 25 x 220 = 5,500 games for the search, 2 x 800 = 1,600 for the confirmation. 7,100 games, ~45 minutes, ONE node process at `--conc 6`.** RAM is the ceiling — check `FreePhysicalMemory` first; at 3.4 GB free this is one process, not two.
- **Output:** `data/exploitability-wobbuffet-e2.json`. `--tag` is deliberate so the run cannot overwrite the void `data/exploitability.json`, which stays void.

- **Read it once, at the end**, and read three fields BEFORE the headline: `search.accepted` (a `search_died: true` makes the rest meaningless), `void` (must be absent), and `pool_announcements` (must hold exactly one line).
- The headline is `headline.kind: "held-out confirmation"`. `searchBest` **is not the result** — see the selection floor in R9, where a search of this shape returns a winning-looking share from pure noise; the floor itself is withheld with the void artifact it was measured beside.

Then re-verify: `node engine/engine_release.js list` must still read `0` of `12` files have moved, and `node engine/provenance.js` must show `exploitability-wobbuffet-e2.json` verified by `CONTENT` rather than by `mtime`. If either fails, the run is void and saying so is cheaper than publishing it.

What is still open, and it is the whole question

Nobody has measured whether MAG is exploitable on the build we ship, and after today nobody has a number at all. The 63.2% is retracted and nothing replaces it: the replacement run is void, and even had it been clean, a one-step search is not a measurement. `PRIORITIES.md` #18 is **not closed and not merely stale — it is now empty**, with a diagnosis attached. And the caveat `MODELS.md` already carries still applies: this grades *readability by a prepared opponent*, which is not the same question as *do we win*, and that one has never been measured against a human at all.

R9 — the step rule, FIXED AND PROVED ON A PLANTED OPTIMUM. And the fix is not enough.

Artifact: `data/exploit-step-probe.json`, written by `engine/exploit_step_probe.js`, stamped with `source_digests` from engine release `d3d04b669e18`. R8 said *"fix the step rule before spending another 7,100 games"*. The step rule is fixed. **The probe then says the step rule was never the binding constraint, and the re-run as specified in R8 should NOT be run.**

The verdict in one line

At 58 features and 24 x 220 games the search closes 0.0% ± 0.1 of the distance to a KNOWN planted optimum. It is not a search. No step rule changes that, because the thing that is broken is the ratio between what one step is worth and what 220 games can see.

The number that ends the argument

On the planted objective with a noiseless oracle, **one accepted step at d=58 moves the true win rate by 0.21 points.** Against that:

what the evaluation can resolve	at 220 games
independent seeds per round — what the void run actually did	4.77 pt — 23x larger than the step
perfect common random numbers (the best CRN can ever buy)	0.45 pt — 2x larger than the step

A hill climb cannot accept a step it cannot measure. Everything else below follows from this row.

What the fix is, and what it bought — acceptance rate before and after

`engine/exploit.js` now exports `createClimber()`, and `exploit_step_probe.js` drives **that function**, not a re-typed copy. Four changes:

1. **The proposal is divided by $\sqrt{d}$** , so $E[||\text{step}||^2] = \text{scale}^2 \cdot \text{mean}(|v_k| + 0.25)^2$ is independent of the number of features. `scale` now means the same thing at 17 and at 58.
2. **Acceptance-targeted multiplicative adaptation** replaces decay-on-failure. `accepted` $\rightarrow \text{scale} \times \exp(g(1-p^*))$, `rejected` $\rightarrow \text{scale} \times \exp(-g \cdot p^*)$, so $E[\Delta \log \text{scale}] = g(\hat{p} - p^*)$ and the scale has a **fixed**

point at the target acceptance rate. The old rule multiplied by 0.85 on failure and by 1 on success: its only equilibrium was zero, which is where it went.

3. **A stall restart**, which acceptance-targeting alone does NOT give you and which the probe found rather than the diagnosis predicting. Adaptation assumes acceptance falls monotonically as the step grows. Against a *measured* objective that is false at the small end — once a step's true gain drops below the measurement's resolution it becomes invisible and acceptance collapses **as the step shrinks**. Acceptance is non-monotone, near zero at both ends, and a shrink-on-failure rule parks on the wrong one. Measured before the restart was added: 1% acceptance and 0.1% of the distance closed, **worse than the rule it replaced**.
4. **The acceptance target is 0.05, not the textbook 0.2–0.4**, and that is measured. The classical band is derived for an exact oracle. Swept at $d=58$, 200×1200 , perfect CRN:

target acceptance	0.02	0.05	0.10	0.25	0.40
distance closed	19.4%	19.9%	15.9%	6.7%	3.6%

The textbook band is the worst end of the sweep. Steps must be big enough to be **seen**, not merely big enough to be good.

Acceptance rate, $d=58$, 24 rounds, 40 independent runs per arm:

noise model	rule	accepted	distance closed
noiseless oracle	legacy	8.7/24 (36%)	6.3% $\pm$ 0.5
noiseless oracle	fixed	7.4/24 (31%)	6.1% $\pm$ 0.4 ($z = -0.4$, no difference)
independent seeds — as the void run ran	legacy	2.4/24 (10%)	-1.5% $\pm$ 0.5
independent seeds	fixed	2.5/24 (10%)	0.0% $\pm$ 0.1 ($z = 3.1$, fixed better)
perfect CRN	legacy	0.8/24 (4%)	1.4% $\pm$ 0.3
perfect CRN	fixed	0.5/24 (2%)	0.2% $\pm$ 0.1 ($z = -3.7$, legacy better)

Read honestly, three things, and the third is the uncomfortable one:

- **The 1-of-24 acceptance in the real run is reproduced** — the toy gets 2.4/24 at the same dimension, same games, same noise model, without being tuned to.
- **The fix's only measured win is that it stops the search moving BACKWARDS.** Under the noise the run actually had, the legacy rule closed **-1.5%**: it accepted upward noise flukes and ratcheted away from the optimum. That is what "1 of 24" was doing.
- **Under CRN the legacy rule is BETTER at this budget**, and the reason is instructive rather than embarrassing: its un-normalised step is $\sqrt{58} = 7.6\times$ larger in norm, which is the right direction when the measurement is coarse. That is what moved the acceptance target to 0.05. At 5,280 games both numbers are ~ 0 against a 25-point edge (0.05 vs 0.35 win-rate points), so this is a comparison of two zeroes and neither rule is worth running at that budget.

The two walls, and neither is the step rule

Wall 1 — evaluations. A (1+1) climb makes progress at $\sim 1/d$ per evaluation. 24 evaluations in 58 dimensions is 0.4 of one such unit, and the probe confirms it: **with a NOISELESS oracle the ceiling at 24 rounds is 6.1% of the distance**. At $d=17$ the same 24 rounds close 19.9%. The 2026-07-26 run was not luckier, it was in a smaller space.

Wall 2 — resolution. The table at the top. Below it, no number of rounds helps: the `independent`-noise column of the budget sweep is flat at 0.0% from 5,280 games to **960,000**.

rounds x games	total	independent	perfect CRN
24 x 220	5,280	0.0% $\pm$ 0.1	0.2% $\pm$ 0.1

rounds x games	total	independent	perfect CRN
100 x 220	22,000	-0.0% ± 0.1	1.0% ± 0.5
200 x 220	44,000	-0.0% ± 0.1	1.2% ± 0.7
200 x 1200	240,000	-0.0% ± 0.3	19.9% ± 2.1
400 x 1200	480,000	-0.0% ± 0.3	29.2% ± 2.8
800 x 1200	960,000	-0.0% ± 0.3	36.8% ± 3.4

Cheapest split that closes a material (>25%) fraction of the distance: 960,000 games, and only if common random numbers couple perfectly. That is not a run this project should schedule, and it is the honest reason to stop rather than a reason to argue for a bigger machine.

The lever that IS affordable: search fewer numbers

The other end of the same trade. At the affordable 24 x 220 = 5,280 games, with CRN, with the optimum planted **inside** the searched space:

dimensions searched	4	8	17	30	58
distance closed	49.7% ± 2.2	30.7% ± 3.0	5.9% ± 1.3	1.2% ± 0.2	0.2% ± 0.1

A 5,280-game search buys a real answer about a family of roughly 4 to 8 numbers and nothing at all about a family of 58. So the re-run's design question is no longer "how do we step" — it is **what low-dimensional reparameterisation of MAG's policy is worth attacking**: feature groups, a handful of scalars, a temperature. That is a SEARCH design item and it is now the blocker on R8, ahead of the engine.

Explicitly NOT tested, so it is not smuggled in: perturbing only k of the 58 raw coordinates. On a *dense* optimum a random k-subspace caps at $1 - \sqrt{1 - k/d}$ of the distance no matter the budget, and nobody has measured whether the real exploit direction is sparse. The table above plants the optimum inside the searched space, which is the right question for a reparameterisation and the wrong one for a sparse mask.

The selection floor — what the OLD headline reported when nothing was found

Pure arithmetic on the binomial, and nobody has ever printed it beside the headline. Under the null that every candidate is exactly as good as MAG, the **maximum over R+1 evaluations** is still:

evaluations x games	mean reported "best"	95th pct	99th pct
19 x 220 (the 2026-07-26 run)	56.2%	59.5%	60.9%
25 x 220 (the void run)	56.6%	59.5%	61.4%
25 x 800	53.5%	55.1%	56.0%
201 x 1200	54.0%	54.9%	55.6%

The void run's search-best of 55.8% is BELOW the floor its own procedure produces from pure noise. It was never a finding. The retracted 63.2% sits above the 99th percentile of its floor, so *that* one is not explained by selection alone — which changes nothing about its retraction, since the objection to it was provenance and a 17-feature vector on a 25-wire-fix-old engine, not selection.

This is why the artifact's headline is now the held-out `--confirm` leg and why `searchBest` carries the literal label `SELECTION-BIASED`, not the headline.

What `exploit.js` now does. IMPLEMENTED, and defect zero is closed.

- **Opens** `engine_release.open()` **and REFUSES TO RUN without one.** Prints `REFUSING TO RUN: no engine release has been cut`.
- **Reads the DEFENDER out of the snapshot** — `REL.path('data/policy-weights.json')` — so the file that moved on 2026-08-04 cannot move again. Verified: the refusal path fires today, naming `engine/medicham2-browser.js`, `engine/tags.js`, `data/tags.json`.

- **Re-checks** `drift()` **after every single evaluation** and aborts mid-run with `void: true` and the file list. This is the guard that would have stopped the void run at round 23 instead of after it.
- **Stamps** `REL.stamp()` — 12 content digests, the release id, the Showdown commit — plus the target vector's own sha12, node version, machine, `--conc`, `n_measured / n_unit`, and every MEW: N distinct `clean teams` announcement it saw. **If the pool moved between evaluations the artifact writes itself** `void: true`.
- **Common random numbers** across all search rounds (`--no-crn` restores the old per-round seeds).
- A `--confirm` **phase** at seeds the search never touched, plus a fresh mirror control, and THAT is the headline. It used to live as a scratch file outside `engine/`, which is why `provenance.js` never enumerated `data/exploitability-holdout.json`.
- **Says out loud when the search died:** `THE SEARCH DID NOT MOVE...` it is evidence this search did not look, and `search_died: true` in the artifact.
- `--legacy-step` reproduces the 2026-08-04 rule exactly, so the comparison above stays runnable.

`data/exploitability.json` and `data/exploitability-holdout.json` are **left void and were not regenerated**.

FILED FOR MEASURE — the frozen release is not a loadable engine, and its freeze list has holes

Tripped over while wiring `exploit.js`; **not fixed here, because changing** `SOURCES` **changes every release id and two divisions are measuring against** `d3d04b669e18` **right now**.

`REL.require()` — the usage `CLAUDE.md` documents — throws for **4 of the 12 frozen sources**:

```
OK    engine/medicham2-browser.js    FAIL  engine/board.js    -> Cannot find module './mc_key.js
OK    engine/tags.js                FAIL  engine/rollout_leaf.js -> Cannot find module './mc_key.js
                                           FAIL  engine/position_features.js -> Cannot find module './mc_key
                                           FAIL  engine/champions_sim.js  -> Cannot find module './showdown_
```

The example in `CLAUDE.md` happens to be the one file that works. **Six files are reachable from the freeze list and are not in it**, and they are not inert:

unfrozen	required by	why it can change a number
<code>engine/mc_key.js</code> (16 KB)	<code>board.js</code> , <code>position_features.js</code> , <code>rollout_leaf.js</code>	decides which dex row a species resolves to
<code>engine/lookup.js</code> (5 KB)	<code>board.js</code> , <code>mc_key.js</code>	the lookup path underneath it
<code>engine/set_priors.js</code> (40 KB)	<code>champions_sim.js</code>	what an unknown set is filled with — every self-play game
<code>engine/smogon_priors.js</code> (17 KB)	<code>set_priors.js</code>	same
<code>engine/quality.js</code> (15 KB)	<code>set_priors.js</code>	decides the team pool
<code>engine/showdown_path.js</code> (4 KB)	<code>champions_sim.js</code>	which Showdown checkout is loaded

So the release is a valid **digest set** and not yet a loadable **engine**, and its claim to freeze "every file whose content can change a number" is currently false for six files. `exploit.js` works around it by reading `FEATURES` from the live `board.js` *after* proving zero drift, which is equivalent and verified — and which is exactly the kind of workaround that stops being safe the day someone passes `--allow-drift`. It says so in a comment at the line.

FILED FOR MEASURE — `provenance.js` marks every RELEASE-BASED measurement UNSAFE

Found by being the first artifact to stamp `source_digests` from a release. `provenance.js:650` resolves a stamped input against the **live tree** (`digestOf(src)`). A measurement that reads a frozen release is, by design, computed from bytes the live tree has moved past — so the check that was built to catch a moving tree now fires on the artifacts that handled a moving tree correctly:

```
exploit-step-probe.json  UNSAFE  COMPUTED FROM DIFFERENT CONTENT —  
data/policy-weights.json was 5a1930e8926a at read time, is 01bc43936324 r
```

That digest moved because MEASURE's refit landed, which is exactly the event the release exists to survive. The artifact is **correct**; it names release `d3d04b669e18` and read that release's bytes.

Suggested shape of the fix, which is MEASURE's to make: when an artifact carries `engine_release`, verify its `source_digests` against **that release's manifest** and report `ON RELEASE <id>` — with a separate, non-UNSAFE line saying how far the live tree has since drifted. `UNSAFE` should remain for an artifact whose stamped digest matches neither the live tree nor any release, which is the case that means "computed from bytes nobody can name".

Until that lands, the count printed as `0` verified by CONTENT digest, `92` by mtime alone **cannot reach 1 for any release-based artifact while ENGINE or MEASURE is working** — the row leaves the mtime-only list and lands in `mismatch` instead of `verified`. The ratchet in `data/provenance-stamp.json` still falls (that list is what is ratcheted), so this does not block the ratchet; it does mean the headline verified-count understates the fix.

A second, larger limitation, stated because it is structural rather than a bug: `exploit.js` spawns `mew.js`, which loads the LIVE engine. No release can prevent that without the child being runnable from the snapshot. Detection (drift-check every evaluation, abort, self-declared void) is what is implemented; prevention is not, and pretending otherwise would be the more dangerous choice.

R10 — the reparameterisation memo. ANALYSIS, 2026-08-05. WILL DECIDES; nothing here was run against MAG.

R9 ended with "search fewer numbers" and left open WHICH numbers. This section brings the concrete options. Every figure traces to an artifact; the two new ones are `data/exploit-step-probe-reparam.json` (the R9 toy swept over family sizes 4–12 at the real budget and at twice it, written by `engine/exploit_step_probe.js --reparam` — same `runOne`, same `createClimber`, no games) and the fitted vector itself, `data/policy-weights.json` (generated 2026-08-04T23:37:26.954Z, corpus 8,856 games / 231,722 decisions, frozen in release `6e43710396db` as `01bc43936324`).

The arithmetic that frames every option (data/exploit-step-probe.json)

- One accepted step at `d=58` moves true win rate by **0.202 pt**; 220 games resolve **4.77 pt** (independent seeds) / **0.45 pt** (perfect CRN). The step is invisible, so the search cannot climb.
- Largest family the affordable $24 \times 220 = 5,280$ -game budget can actually search: **about 4** numbers (`largest_searchable_family_at_5280_games`).
- The toy plants a 25-pt edge (`pMax` 0.75), so "distance closed" reads as "fraction of the family's available edge captured". The confirm leg (`--confirm 800`) certifies nothing smaller than **~3.5 pt** ($1.96 \cdot 50 / \sqrt{800}$), whatever the search finds.

Family sizes at the real budget and at 2x, measured, not extrapolated (`data/exploit-step-probe-reparam.json` ; fixed rule; distance closed $\pm$ SE over 40 runs; the truth about CRN coupling in real games is UNMEASURED, so both brackets are printed):

family size	24 x 220 crn	24 x 220 indep	48 x 220 crn	48 x 220 indep	24 x 440 crn
4	49.9 $\pm$ 2.2%	37.0 $\pm$ 3.9%	63.9 $\pm$ 2.4%	49.3 $\pm$ 3.7%	51.6 $\pm$ 2.7%
6	42.3 $\pm$ 2.8%	18.6 $\pm$ 3.3%	52.6 $\pm$ 2.6%	26.9 $\pm$ 3.8%	45.6 $\pm$ 2.2%
8	30.2 $\pm$ 2.9%	11.3 $\pm$ 2.5%	42.8 $\pm$ 3.5%	15.8 $\pm$ 3.2%	36.5 $\pm$ 2.5%
12	21.1 $\pm$ 2.4%	1.1 $\pm$ 0.6%	31.7 $\pm$ 3.1%	4.8 $\pm$ 1.6%	26.5 $\pm$ 2.3%

Three design facts fall out before any family is chosen: **doubling ROUNDS beats doubling games-per-round in every crn row** (the climb is rounds-starved, exactly as R9's O(d)-evaluations argument says); **d=4 is the only size that stays searchable in the independent bracket**, i.e. the only one whose verdict does not depend on how well CRN couples in real battles; and a doubled budget (48 x 220 + 2 x 800 confirm $\approx$ 12,160 games, $\sim$ 75 min by R8's timing) buys d=4 nearly two-thirds of its family edge.

One structural note before the families

MAG **samples** its softmax rather than taking the argmax (`engine/magnemite.js:12-17`), so a global temperature is a real, playable lever — `c.w` and `w` are different players here, unlike in an argmax policy. That is why F1 below is allowed to spend a parameter on it.

The families

F1 — AXIS-4: temperature, prior, kill, initiative. RECOMMENDED. $w'_k = \exp(\tau)$ ·

$\exp(a_P \cdot [k=priorLogP] + a_K \cdot [k \in KILL] + a_I \cdot [k \in INIT]) \cdot w_k$, searched over $z = (\tau, a_P, a_K, a_I)$ from $z_0 = 0$ (so the incumbent MAG is the start point by construction). KILL = {koTarget, dmgFrac, tgtMayProtect, killIsRoll, killsThreat, koFirst, protectThreatened}; INIT = {movesFirst, priority, speedSwing, diesBeforeMoving} — the blocks as `board.js` FEATURES declares them (release digest `54e3d2ca9f85`). *Why these four axes, from the fit itself:* `priorLogP` is the single most-determined coordinate in the whole vector ($w + 0.1474$, SE 0.0026 — the fit pins it hard **for resemblance**, and resemblance is exactly the objective that cannot certify it **for winning**); the kill and initiative blocks are where the fit is weakest — `koTarget` $+0.0348 \pm 0.0170$, `killsThreat` **-0.0610** ± 0.0131 (killing the thing about to kill you fitted *negative*), `priority` -0.0053 ± 0.0159 and `movesFirst` $+0.0075 \pm 0.0126$ (both indistinguishable from zero). A challenger that wants to beat MAG by wanting kills and initiative more than people do lives exactly here. *Can express:* greedy $\rightleftharpoons$ noisy play, prior-reliance up or down, uniform kill-hunger, uniform initiative-hunger, and their combinations. *Cannot:* rotate within a block (raise `koTarget` while lowering `dmgFrac`), touch the switch/support/dead-move axes, flip any individual sign, or form any interaction the 58 features do not already carry. *Resolution:* 49.9 $\pm$ 2.2% of the family edge at 5,280 games (crn) and **37.0 $\pm$ 3.9% even at the independent bracket**; 63.9 $\pm$ 2.4% at the doubled budget. *A WOBBUFFET null here proves:* no re-mix of sharpness/prior/kill/initiative beats shipped MAG by more than the $\sim$ 3.5-pt confirm floor. It says **nothing** about within-block, switch-axis, or novel-interaction exploits, and MAG's general exploitability stays unmeasured. *A positive* hands MEASURE a named, four-number direction to test as a refit objective.

F2 — BLOCK-8: one log-gain per board.js feature family. The eight blocks as the FEATURES list groups them: targeting/move-quality (13), dead-moves (9), order (4), kill (10), disruption/stages (9), switch (8), support/value (4), prior (1) — 58 accounted for. *Can express:* everything F1 can, plus the switch, support, dead-move-discipline and disruption axes. *Cannot:* within-block rotation or sign flips, same as F1. *Resolution:* 30.2 $\pm$ 2.9% at budget (crn) but **11.3 $\pm$ 2.5% at the independent bracket** — its verdict leans on CRN coupling nobody has measured; 42.8 $\pm$ 3.5% at 2x. *A null proves:* no block-level retuning of MAG's vocabulary beats it at the floor. Broader statement than F1's, bought with a real risk that the search under-resolves and the null is about the noise, not the family.

F3 — FLAT-6: the six flattest Fisher directions of the fit. BLOCKED ON MEASURE.

`fit_policy.js` `standardErrors()` already computes the full observed information H (:663-700) and publishes only the diagonal of H^{-1} . The 4–8 bottom eigenvectors of H (preconditioned) are the directions the resemblance likelihood constrains LEAST — the largest moves a challenger can make per unit of "still plays like the corpus". Dense directions, not an axis mask, so the probe's planted-inside-the-family table applies ($d=6$: $42.3 \pm 2.8\%$ at budget crn, $52.6 \pm 2.6\%$ at 2x). *Cannot*: move in stiff directions — which is precisely where a deliberately non-human exploit would live, so this family is biased toward subtle exploits and blind to flagrant ones. Also unstable across refits (eigenvectors rotate with the corpus), so the H snapshot must be pinned in the artifact. *Blocked*: needs MEASURE to export H or its eigendecomposition; filed as a one-flag change to `fit_policy.js`, not made here — MEASURE's file.

F4 — SPARSE-8: the eight raw coordinates the fit barely pins (`{priority, movesFirst, switchKOFast, tgtHurt, switchKOSlow, pivots, koTarget, allyHit}` — the top of the $SE/|w|$ ranking). Interpretable and directly refittable, **but it is an axis-aligned subspace of the raw space**, and the probe's own warning applies verbatim: against a dense exploit direction it caps at $1 - \sqrt{(1 - 8/58)} = 7.2\%$ of the distance at any budget. Its null is therefore the weakest of the four. Fit only as a cheap confirmatory second arm if F1 finds something, or not at all.

Implementation cost, so the decision is priced

`exploit.js` climbs in z -space with `x0 = 0_k` and maps $z \rightarrow w(z)$ before `writeweights` — one `--family` flag, ~30 lines, in the file `SEARCH` already maintains; the climber, CRN, drift-abort, pool-void and confirm phases all apply unchanged. The artifact must stamp the family definition (the block memberships and the mapping) beside the release id, or the result names a challenger nobody can rebuild.

Recommendation, marked

Run F1 at the doubled budget: 48 x 220 search + 2 x 800 confirm $\approx$ 12,160 games, one process, after the current sweep finishes. It is the only family whose resolution survives the independent bracket, its four axes are the four questions this project keeps asking about MAG in prose (too timid? too human? too slow to take kills? too willing to lose initiative?), and either outcome is actionable: a null retires the block-gain hypothesis at a stated floor, a positive is a refit direction with names on it. F2 second if F1 nulls and Will wants the switch/support axes covered. F3 waits on MEASURE. F4 only as a confirmatory arm. And the standing caveat carries: every family grades *readability by a prepared opponent under our own leaf*, which is not "do we win", and the leaf's calibration is MEASURE's open item — a null can be about the leaf, not the search.

What the 2026-08-04 mega-weather fix invalidates

Stated plainly rather than left to be inferred, because a leaf change that moves values and is not declared is how a stale number survives. **Everything below was computed on a leaf in which a mega never brought its weather.** None of it is wrong about its own arm; all of it describes a build that no longer exists.

	why it is affected	what it needs		
R1 leaf accuracy, and the explore sweep	every position scored through <code>rolloutWinProb</code> ; 6.4% of corpus boards carry a mega setter in clear weather and move by ~9.7 pt	re-run at the release boundary. The <i>sign</i> is very unlikely to move; the point estimate will		
R2 leaf cost	already under re-run (PRIORITIES #14). Cost measured unchanged here (17.53 $\rightarrow$ 16.93 ms at $n=40$), so this fix is not the reason	nothing extra		
R3 divergence	<code>rolloutAfterActions</code> moved on the same boards, by more (mean \	$\Delta \backslash$	18.3 pt at $n=24$)	re-run

	why it is affected	what it needs
R4 the SPRT	every leaf call in both arms	re-run. Note both arms shared the defect, so it partly cancels — which is precisely the failure mode CLAUDE.md names, and is not a reason to keep the number
the leaf calibration (53.22% / 50.99%)	already void for the preview under PRIORITIES #38; the in-game half is now void too	MEASURE, after the boundary

This is a release-boundary matter, not a footnote — PRIORITIES Po.5. The runs on disk were already PRE-CHANGE against the engine; they are now PRE-CHANGE against the leaf as well, and the leaf is SEARCH's own file. Do not start a wide run until the boundary is cut.

Po.5 — THE FROZEN ENGINE RELEASE. BUILT AND CUT. THE RESOLVER WAS FALSE-GREEN UNTIL 2026-08-05.

STATUS, 2026-08-05. *The heading used to read "DESIGNED AND PREPARED 2026-08-04. NOT CUT." Both halves are now out of date: engine/engine_release.js exists, freezes 23 files as a real byte SNAPSHOT rather than a digest list, and release 09acd3b404ef is cut and pointed at. What was not true until 2026-08-05 is that MILTANK could tell you whether it was running it — see §3.*

docs/DIVISIONS.md rule 1 says SEARCH plays a frozen, named engine release and never HEAD. **There is no such release and there never has been**, so the rule has been a sentence rather than a mechanism, and every SEARCH baseline on disk is attributed by status.js comparing mtimes — the one thing run_stamp.js says in its own comments is not evidence, because a checkout moves an mtime without moving code.

This section is the mechanism, the freeze list, the re-run order and the commands. **Cutting it is Will's call**, because the cut triggers the refit and seven restamps.

1. What identifies a release: a SNAPSHOT plus its manifest, not a tag and not the pointer

THE JSON THAT USED TO BE PRINTED HERE WAS THE SECOND SCHEMA, AND IT COST A FALSE GREEN. *This section showed data/engine-release.json as {release, cut, supersedes, commit, dirty, digests{5 files}, claims} and called it "written by a cut and never hand-edited". **Nothing has ever written that.** miltank.js read .digests and .release from the real pointer, found neither, compared zero files and stamped ON_RELEASE on every artifact it produced. The schema below is what is actually on disk. Struck and replaced 2026-08-05; see §3 for the receipt.*

Two files, and the difference between them is the whole trap.

data/engine-release.json — **the POINTER. It carries NO digests.** It only says which release is current, plus that release's first and latest cut times, for a human reading cat :


```
{
  "current": "09acd3b404ef",
  "cut": "2026-08-05T02:12:57Z",
  "why": "h60 log leg of the R1 explore-sweep re-run",
  "cuts": 2,
  "latest_cut": "2026-08-05T02:26:04.945Z",
  "latest_why": "R10/click-censoring parallel session"
}
```

`data/releases/<id>/release.json` — **the MANIFEST, which is where the digests live**, beside `data/releases/<id>/<every frozen file>`, which are the actual bytes. `<id>` is the digest of the digests, so an identical tree always yields an identical id.

Ask the tool, never the file. `require('engine/engine_release.js').open()` resolves the pointer, verifies the snapshot against its own manifest, and hands back `stamp()` and `drift()`. Any code that opens `data/engine-release.json` and looks for a key other than `current` is reading the schema that never existed.

Why a digest set and not a git tag. A tag names a commit, and this repo has already published a result whose own stamp reads *"TREE WAS DIRTY — trust source_digests, not the commit"* (R3, in the generated block above). An unattended auto-commit publishes on a timer here, so a commit id is not a stable statement about what a process loaded; the bytes are. A tag is still worth pushing as a human handle (`git tag engine/E1-2026-08-05`), and it is a **convenience, not the authority**.

The digests come from `engine/run_stamp.js` — `sourceDigests()` **and** `gitState()` — **not from a new hasher**. That is not tidiness: `miltank.js` was hashing **4 files with sha1** while `run_stamp.js` hashes **5 with sha256**, so `data/abra-tags.js` — the file ENGINE rewrites most — was invisible to a MILTANK stamp and visible to every other gate's. Two definitions of "the engine these numbers describe" is the `choiceLock` failure in a new costume. `miltank.js buildStamp()` now calls `RS.sourceDigests()` and the divergence is closed.

2. What exactly is frozen, and why each file

The release freezes **every file whose bytes can change a rollout's value**.

CORRECTED 2026-08-05. This section said that list was `run_stamp.LEAF_SOURCES` — five files — and that it "already has the right membership". **The authority is** `engine_release.js SOURCES`, **which is 23 files**, and the extra eighteen are not padding: the loader closure (`mc_key`, `lookup`, `set_priors`, `smogon_priors`, `quality`, `showdown_path`), the lazy data reads (`move-effects`, `ability-blocks`, `smogon-priors`, `regulations`, `quality-filter`), the tag and dex artifacts, and `data/policy-weights.json`, which is the byte that actually moved on 2026-08-04. Read `SOURCES` in that file; do not read the table below as the list. The five rows below are kept because their reasons are still the clearest statement of why a file qualifies.

file	why it is in
<code>engine/medicham2-browser.js</code>	the simulator. Damage, priority, abilities, the payout loop
<code>engine/rollout_leaf.js</code>	the payout and the field boundary. <code>applyField</code> alone has moved two published numbers this week
<code>engine/board.js</code>	<code>dmgMon</code> builds every rollout body; <code>candidates()</code> is the menu
<code>data/engine-data.js</code>	stats, moves, items — the table every body is built from
<code>data/abra-tags.js</code>	every mechanic param the engine reads. The census lives here

Deliberately OUT, each for a stated reason:

- `engine/miltank.js` **is NOT frozen**. It is the thing under test. Freezing the player inside the release would make an H2H between two players impossible to name.
- `data/policy-weights.json` **is NOT frozen**. **REVERSED 2026-08-05, and the reversal is the whole point of the release**. The argument was that MAG's fit is a different invalidation edge. It is — and on 2026-08-04 that edge moved *between the two legs of one measurement*, at 22:15:24, so the 7,100-game WOBBUFFET run defended with two different weight vectors. A measurement of "can anything beat MAG" is a claim about ONE specific vector; leaving it out of the release meant the claim could not be named. It is in `SOURCES` and `exploit.js` reads the defender out of the snapshot.
- `engine/mew.js` , `engine/sprt.js` **are NOT frozen**. DIVISIONS: MEASURE's tools sit beside the graph and invalidate nobody.

ENGINE's freeze list is not the feature path, and the two authorities must not be swapped.

question	authority	what it answers
is the fit stale?	<code>node engine/feature_fixture.js --check data/policy-weights.json</code>	do the fitted weights still mean what <code>board.js</code> computes. Ran clean 2026-08-04: <i>"agrees with board.js on every fixture board"</i>
is a rollout comparable?	<code>node engine/engine_release.js list</code> — the release MANIFEST, not the pointer	did the simulator move under the run. The pointer has no digests; asking it was the \$3 bug

They overlap on `board.js` only, and they can disagree in both directions: the fit is currently **clean** while every rollout on disk is **not comparable**. Reporting one as the other is the silent-default failure DIVISIONS names as the cost of a boundary.

3. How a run declares its release, and how `status.js` marks drift

CORRECTED 2026-08-05. Everything this section said before was written against a pointer schema that never existed, and the resolver built from it was a green that could not be false. Read the receipt below before reading the table.

THE FALSE GREEN, AND THE TWO SCHEMAS THAT CAUSED IT

`engine/miltank.js:145` read `rel.digests` and `rel.release` out of `data/engine-release.json` .
`engine/engine_release.js` **has never written either field**. A cut writes `current` , `cut` , `why` , `cuts` , `latest_cut` , `latest_why` ; the digest set lives in the release's own manifest at `data/releases/<id>/release.json` , one directory down, which the resolver never opened.

So `want` was always `{}` . Zero files were compared, zero were found moved, and **every MILTANK stamp ever written reads** `release: "UNNAMED", release_status: "ON_RELEASE"` . Reproduced against the live pointer on 2026-08-05: feeding the old resolver a digest set in which *every file is wrong* still returned `ON_RELEASE` , because there was nothing for it to be wrong about. A green produced by an empty comparison — structurally incapable of reporting drift, and therefore incapable of being false, in the one field whose entire job is to say which bytes a number describes. Found by ENGINE, who filed it rather than patching SEARCH's file.

It survived because there were two pointer schemas. The real one, written by `engine_release.js` `cut` ; and a hand-rolled `node -e` recipe that used to sit in step 0 of §6 below and would have written `{release, digests, commit, dirty}` . The resolver was coded correctly against the recipe, and the recipe was never what ran. **The recipe is struck**. There is one way to answer "which release am I on" and it is `engine/engine_release.js` — `open()` , `verify()` , `drift()` , `stamp()` . `miltank.js` now CALLS those rather than re-implementing the comparison; a second implementation of a fact is what CLAUDE.md forbids and this is what one costs.

The states, as they now read

`miltank.js buildStamp()` runs at module load — not at the first row, so it cannot describe a file edited underneath a running process — and resolves **five** states, because absent evidence and positive evidence are different events:

stamp reads	meaning
<code>release_status: "ON_RELEASE" , release_files: 23</code>	<i>n</i> frozen files hashed against the live tree, none moved. The only green, and it now carries the count it rests on.
<code>release_status: "OFF_RELEASE" , release_moved: [files]</code>	the live tree has moved off the release, and it names which files. (Was <code>PRE-RELEASE</code> ; renamed because it is drift <i>after</i> a cut.)
<code>release_status: "NO_RELEASE"</code>	no release has ever been cut — rule 1 unenforced , not satisfied
<code>release_status: "RELEASE_UNUSABLE"</code>	a release store exists but the pointer or the snapshot is broken. Never collapsed into <code>NO_RELEASE</code> , which would read as a fresh install
<code>release_status: "UNKNOWN"</code>	the comparison could not be made — including a manifest that names zero files , which is exactly the state that used to read <code>ON_RELEASE</code>

Every non-green state also prints to `stderr` at load and carries a `release_why` a person can read. The stamp additionally carries `engine_release` , `engine_release_cut` , `engine_release_cuts` , `showdown_commit` and `engine_release_digests` — the same answers `REL.stamp()` gives, from the same object, so a MILTANK shard and a gate artifact are read with one set of eyes. `engine_release_digests` is `REL.stamp().source_digests` renamed on the way in, because the stamp already has a `source_digests` (the live four-file player hash that `reduce()` keys its mixed-build check on) and overwriting a live hash with a frozen one would be a quieter version of the same bug.

Proved by `tests/test-miltank-release.js` (25 assertions, green 2026-08-05), which shows the check failing on known-bad input before believing its green:

- the **old** resolver, replayed verbatim against the pointer schema that is actually on disk with *every* digest deliberately wrong, still stamps `ON_RELEASE` off 0 files compared;
- a manifest naming **zero files** now reads `UNKNOWN` , not `ON_RELEASE` ;
- a genuine release with an unmoved tree reads `ON_RELEASE` off 24 files;
- a **genuinely modified** live file the manifest names reads `OFF_RELEASE` and names it — and the mutation is asserted to have actually changed content, so a skipped edit cannot pass.

The drift arm mutates a probe file the test writes itself (`data/.miltank-release-probe-<pid>.jsonl` , ignored by `.gitignore` 's `data/*.jsonl`), **never a frozen engine source**: four divisions write to this repo and a frozen source being different for even a moment voids somebody else's run. Every release is cut into a throwaway store; the real pointer is never written by a test.

The change `status.js` **needs — SPECIFIED, NOT APPLIED, because** `status.js` **is MEASURE's file.** Today `status.js:315-331` finds the newest engine-source **mtime** and prints `PRE-CHANGE` for any run file older than it. Replace the comparison, keep the line:

1. call `require('./engine_release.js').open()` . **Do not read** `data/engine-release.json` **by hand and do not look for a** `digests` **key on it — there isn't one, and believing there was is the whole of the bug above.** If `open()` throws and `list()` is empty, print `NO RELEASE CUT – rule 1 of DIVISIONS.md is unenforced` and fall back to today's mtime inference **labelled as an inference**; if `list()` is non-empty, the store is broken and that is a finding, not a fresh install;
2. for each run, read its stamp (`*.meta.json` sidecar for a gate artifact, the `_stamp` row for a `MILTANK_TIMING` shard, `_stamp` in the games `jsonl` for a mew run) and compare its `engine_digests` to `REL.manifest.files` — or, better, read the run's own `engine_release` and `release_status` , which `miltank.js` now writes and which already carry the answer;

3. print one of `ON <release>` , `OFF-RELEASE (<files that moved>)` , or `NO STAMP` .

`NO STAMP` **must be its own state and must not read as current.** An unstamped run is not evidence about any build, which is strictly worse than a run known to be old — that is the whole finding behind `run_stamp.js` existing. The same rule now applies one level in: **a green that compared zero files is UNKNOWN , not ON_RELEASE** .

One implementation, and it is `engine/engine_release.js` . `miltank.js` no longer holds a comparison of its own — it calls `open()` , `verify()` (via `open`) , `drift()` and `stamp()` , and adds only the classification of the result into the five states above. When MEASURE lands the `status.js` half it should call the same four. **Do not write a second one; a second one is what this section is a retraction of.**

4. What must be re-run once the cut lands, ordered by cost

Cost is stated in leaf calls and payouts, not minutes, because this file's own R2/R6 sections say a duration is a fact about a machine under a load and R2 is being re-run for exactly that.

order	gate	unit of work	shares a corpus with	why it must be re-run		
1	R2 leaf cost	~477 leaf calls	R1 (same walker, same stride)	already owed (PRIORITIES #14); the weather and terrain fixes make payouts longer, so every cost figure quoted downstream is a lower bound until this lands. Run it first — every other estimate below is priced off it		
2	R1 + the explore sweep	9,201 positions × 3 explore arms ≈ 27.6k leaf calls ≈ 1.1M payouts at n=40	R2, R3, R5	every position scored through <code>rolloutWinProb</code> . The sign is very unlikely to move; the point estimate will		
3	R3 divergence	121 decisions at n=600, two searches plus the self-disagreement control	R1, R5	<code>rolloutAfterActions</code> moved by mean \	Δ\	18.3 pt on the same boards
4	R7 timing distribution	12 self-play games	R6a (identical command plus <code>MILTANK_CLOCK=1</code>)	R6's figures describe a build that no longer exists and are lower bounds		
5	the in-game leaf calibration	MEASURE's, corpus-sized	R1, R3, R5	measured on a leaf that could not read weather. MEASURE's item, not SEARCH's		
6	R5 action-ranking backtest	~2,000 decisions × ~63 cells × n=200 ≈ 25M payouts, 4 shards	R1, R3	never run; it is the measurement that decides whether the leaf is worth its cost		
7	R4 the SPRT	~420 decisive pairs, self-play	R6c only	every leaf call in both arms. Last, because 1–6 can redirect it		

What genuinely shares a corpus. R1, R2, R3 and R5 all walk the same clean open-sheet games through `joint_rows.build` 's `onBoard` observer with a stride, so they sample from one population and should record **the same corpus id and the same stride** — and R3 is computable as a **by-product of the R5 pass**, because R5 already enumerates the menu at each decision point that R3 compares two searches over. R4, R6a, R6c and R7 are self-play through `mew.js` and share nothing with the corpus gates; R7 and R6a are the same 12-game command differing by one environment variable.

What does NOT need re-running: the live-budget derivation (read out of the Showdown source, not measured on our engine), the request-length distribution over 30,396 ladder games (a property of the store), and R6b's forfeit answer, which is arithmetic on those two.

5. THE TRAP THAT WOULD MAKE ALL OF THIS A NULL — instrumented 2026-08-04

PRIORITIES ob: `--miltank` **with** `--policy random` **searches nothing and looks completely normal.**

Every `chooseMove` bails silently, and an H2H arm can therefore run a whole job having never called the leaf while still printing a win rate.

Reproduced, and it is worse than filed. A 2-game run with `--policy random --miltank` finished clean, printed `MEW done: 2 games (0 discarded)`, and wrote **no** `MILTANK_TIMING` **file at all** — not an empty one, none — because every decision bails before the recorder is reached. The `--reduce` step answered that with an `ENOENT` stack trace, which reads like a broken tool rather than a run that never searched.

So the counter now exists and every re-run must read it before it reads anything else.

`engine/miltank.js` counts leaf entries and playouts, stamps the running total on every timing row, and `--reduce` publishes:

```
"search": { "leaf_calls": 1142, "playouts": 28415,
            "decisions_with_zero_leaf_calls": 0, "zero_leaf_pct": 0, "VERDICT": "ok" }
```

with `VERDICT` reading `THE SEARCH NEVER RAN` — this artifact is not a measurement of `MILTANK` at zero, and `--reduce` on a missing file returning a named verdict instead of a stack trace. Verified on a 3-game smoke: 36 decisions, 1,142 leaf calls, 28,415 playouts, 0 decisions with zero leaf calls.

The rule for every command below: `MILTANK_TIMING` **is set on BOTH arms, and no verdict is read until** `search.leaf_calls > 0` **on both.** For the corpus gates (R1/R3/R5), which do not go through `miltank.js`, the equivalent guard is the generator's own `nulls / skipped` counters — a gate whose rows are mostly nulls is the same failure wearing the other hat.

6. The commands. PREPARED, NOT RUN.

`SHOWDOWN_PATH` is required by all of them. Check `FreePhysicalMemory` before choosing a process count — it was **3.4 GB** when this was written, which is one to two processes, not six.

Step 0 — the cut itself. Will's call, and it triggers the refit and the restamps.

```
# preconditions, all three, in this order
git status --porcelain           # must be clean, and no rebase in progress
node tests/run-all.js           # the census must not be down
SHOWDOWN_PATH=C:/Users/willj/Projects/Pokemon/pokemon-showdown \
  node engine/feature_fixture.js --check data/policy-weights.json

# THE CUT. One command. It copies the frozen bytes, appends a cut event, and repoints the pointer.
node engine/engine_release.js cut "<why this release exists>"
node engine/engine_release.js list # must read: 0 of N files have moved since

git tag engine/E1-$(date +%F)     # the human handle, NOT the authority
node engine/status.js --write
```

STRUCK 2026-08-05 — the hand-rolled `node -e` **recipe that used to be step 0.** It wrote `data/engine-release.json` as `{release, cut, supersedes, commit, dirty, digests}`. That is **not the**

schema `engine_release.js` **writes**, it snapshots no bytes, and it was never what ran. It is the second of the two pointer schemas named in §3, and `miltank.js` 's release resolver was coded against it — which is why every MILTANK stamp on disk claims `ON_RELEASE` off an empty comparison. A documented recipe that competes with the tool is not a convenience; it is a second implementation of a fact wearing a shell prompt. **There is one way to cut and one way to ask.** Its one good idea is preserved where it belongs: the dirty-tree refusal below is now the `git status --porcelain` precondition on the line above the cut, checked by the person cutting. `engine_release.js` freezes `CONTENT`, so a dirty tree does not corrupt a release — it only makes the release harder to name against a commit, which is why the tag is a convenience and the digests are the authority.

The dirty-tree observation stands even though the recipe that made it is struck. Dry-run 2026-08-04 21:01 UTC: the guard refused, and in the eight minutes either side of it `engine/medicham2-browser.js` went `7649d0760a88 → 3653b857dc29`, `engine/board.js` went `bcf2dab9dc6f → 88506029c850` and `data/abratags.js` went `ea5b89c2afcd → facd3f2f50b4`. **Three of the five frozen files moved while this section was being written**, which is the entire argument for the boundary, observed rather than asserted. The cut happens after ENGINE lands and commits — the answer is never to drop the guard.

Step 1 — R2, first, because everything else is priced off it:

```
SHOWDOWN_PATH=... GAMES=120 EVERY=3 N_LIST=10,40,200 EXPLORE=1.0 MAXTURNS=60 \  
node --max-old-space-size=4096 engine/rollout_r2.js
```

`EXPLORE` and `MAXTURNS` are passed **explicitly** — the committed artifact was measured at the defaults (`explore 0`, `maxTurns 20`) while the shipped leaf runs 1.0 and 60, and not passing them is the whole of PRIORITIES #14.

Step 2 — R1 and the explore sweep, one walk, one process:

```
SHOWDOWN_PATH=... GAMES=800 EVERY=2 N_LIST=40 EXPLORE_LIST=0,0.5,1.0 \  
DUMP=rollout-r1-E1-rows.jsonl \  
node --max-old-space-size=4096 engine/rollout_r1.js  
  
node engine/rollout_r1_artifact.js data/rollout-r1-E1-rows.jsonl
```

`DUMP=` **resolves under** `data/`, and a name already on disk is overwritten. Use a release-stamped filename; the 2026-08-04 sweep nearly destroyed the only evidence for the incumbent arm this way.

Step 3 — R3:

```
SHOWDOWN_PATH=... GAMES=600 EVERY=3 N=600 TOPK=3 EXPLORE=1.0 \  
node --max-old-space-size=4096 engine/rollout_r3.js
```

Step 4 — R7, the timing distribution on the named release (one process, 12 games):

```
SHOWDOWN_PATH=... MILTANK_TIMING=$PWD/data/.miltank-timing/r7.jsonl \  
node --max-old-space-size=1536 engine/mew.js --n 12 --conc 1 \  
--policy score --policy2 score --miltank --miltank-n 200 --miltank-preview-n 40 \  
--seed 90001 --out data/.miltank-timing/r7-games.jsonl  
  
node engine/miltank.js --reduce data/.miltank-timing/r7.jsonl \  
--horizon-store data/games.ladder.jsonl --out data/miltank-timing-r7.json
```

Read `search.leaf_calls` **and** `build.release_status` **in that artifact before reading a single timing figure.** They must be non-zero and `ON_RELEASE` .

Steps 5–7 — R5, then R6c, then R4 are already specified above and below in this file, with their shard commands. Each carries the same two preconditions: `release_status: ON_RELEASE` in the stamp, and a non-zero leaf-call count. Read every SPRT at the bound, once, on the cat of its shards.

And a caution the parity does not cover. `docs/SEARCH.md` item 5b records that the board's weather string has never meant anything to MEDICHAM. Until that is fixed, **a re-run of R1/R3/R4 would still be measuring a leaf that is blind to 60.8% of boards' weather.** Sequencing the two matters: fixing 5b after re-running the gates buys a second round of invalidation.

Read R4 correctly

R4 measured `--miltank-n 30` , uniform-random playout opponents, preview search disabled. It is a **floor, not a description**. It does not say "the bot is good".

What it does say is the thing worth keeping: the pre-fix baseline on the broken engine was -0.28 , and the same search with the same flags came back positive once the model was fixed. **A search is worth exactly what its model is worth.** That is why this division's open items are mostly about what the search *believes*, not how deep it goes.

And note the `PRE-CHANGE` markers in the generated block: those runs predate the current engine source. Under the frozen-release rule in `DIVISIONS.md` that is a re-run, not a judgement call.

The `--rollout-explore` default was re-earned, 2026-08-04

MEASURE retracted R1 that morning: the published `68.18%` had no artifact, and the only committed row dump held the **explore=0** arm, on which R1 is UNDECIDED. `--rollout-explore` defaults to `1.0` and two comments cite that retracted figure as the reason.

It was re-run at explore=1 and it reproduces. Artifacts: `data/rollout-r1-explore-sweep.json` (the arm-vs-arm verdict, written by `engine/rollout_explore_sweep.js`) and `data/rollout-r1-explore1.json` (the gate, written by the existing `engine/rollout_r1_artifact.js` from `data/rollout-r1-explore1-rows.jsonl`).

	explore=0	explore=0.5	explore=1.0	material
accuracy, horizon 20 (9,201 positions)	65.72%	67.58%	67.97%	65.27%
accuracy, horizon 60 (4,487 positions)	64.21%	66.50%	67.46%	63.78%
ECE	0.196	—	0.104	0.050
share saturated in the 0–10 / 90–100 bin	50.7%	—	29.4%	—

Paired, on the identical sample: **+2.25 points, 95% CI [1.31, 3.19]**, monotone in explore at both horizons. The published `68.18%` lands at `67.97%` and the published `+2.91` over material lands at `+2.71 [1.60, 3.82]` — the retraction was right about the *provenance* and the claim survives it.

Three things the re-run settled that were not the question:

- **The committed greedy dump was NOT clobbered.** `DUMP=` resolves under `data/` , so the command MEASURE left would have overwritten the only evidence for the incumbent arm. New filename used.
- **The "64.42% for greedy" half does not reproduce** — greedy measures 65.7% on both the committed dump and a fresh run on the current engine. Same sign, gap 2.25 not 3.76. The two comments overstate it and should be restated against the artifact.
- **Unfinished playouts are not the mechanism.** `battleResult` does score bodies-then-HP whether or not the battle ended, but 99.5–99.8% of playouts end by an actual wipeout at every explore setting and at

both horizons. Cap-hits are 0.2–0.5%. Exploration makes payouts *longer* (4.4 → 6.1 mean turns), not truncated.

This is a verdict on a JUDGE, not on a player. It does not say explore=1.0 wins more games, and engine/mew.js exposes no --miltank-explore, so the A/B that would say is not currently runnable. R4 was itself run at 1.0 and cannot arbitrate its own setting.

The live budget — SETTLED 2026-08-04 from the Showdown source, and it was not 45 seconds

This section previously said "a 7-minute chess clock with a 45-second cap on any one decision", flagged as an **unverified rules assumption** (PRIORITIES #39). It has now been read out of the implementation we actually play against, and **two of its three numbers were wrong**. Correcting the diagnosis, not just the number:

config/formats.ts gives [Gen 9 Champions] VGC 2026 Reg M-B the ruleset ['Flat Rules', 'VGC Timer', 'Open Team Sheets']. data/rulesets.ts vgctimer is, verbatim:

Timer Starting = 420	Timer Grace = 90	Timer Add Per Turn = 0
Timer Max Per Turn = 55	Timer Max First Turn = 90	
Timeout Auto Choose	DC Timer Bank	

and server/room-battle.ts says what those do:

fact	line	consequence
secondsLeft = starting + grace	:210	the bank is 510 s , not 420
turnSecondsLeft = Math.min(secondsLeft, maxTurnTime)	:327	the per-turn cap is DRAWN from the bank. One clock, not two
addPerTurn = 0, and updateTurn adds it	:306	no refill. A true bank, exactly as feared
maxPerTurn = 55	ruleset	the per-decision wall is 55 s , not 45
maxFirstTurn = 90	:326	team preview gets 90 s , from the same bank
turn expires, bank alive → >{slot} default	:451-453	a server-chosen move , not a loss
bank hits 0 → forfeitPlayer(..., 'lost due to inactivity.')	:455	you lose the game

Three further facts read on 2026-08-04, two of which correct the table above

The section above was right that the clock is one drawn bank and wrong about its size. Corrected against the source rather than argued:

1. THE BANK IS 420 s, NOT 510. The grace is use-it-or-lose-it and is not bankable. :209 does initialise secondsLeft = starting + grace = 510. But updateTurn at :305-306 runs

player.secondsLeft = Math.min(player.secondsLeft + addPerTurn, this.settings.starting);

on **every new turn**, addPerTurn is 0 and starting is **420**. So Math.min(510, 420) fires on the second timed request and the 90 s of grace is *clamped away*. It is spendable only on the first timed request. updateTurn returns early the very first time (:270-274, this.turn === null), which is exactly why the clamp lands on the second request and not the first.

Consequence: `budgetMs: 20000` **buys 21 decisions, not 24.** The figure this file replaced — "21 decisions" — was right by accident, off a wrong premise; the corrected derivation is $420,000 / 20,000 = 21$, with preview free because it comes out of the grace that is about to be clamped anyway.

2. THE BANK DOES NOT TICK WHILE THE TIMER IS OFF, so the mid-game switch-on is benign. `secondsLeft` is decremented only inside `nextTick ( :353 )`, which is scheduled only by `nextRequest ( :341 )`, which returns at `:320` when `!this.timerRequesters.size . start()` then calls `nextRequest` itself (`:239-240`). So an opponent typing `/timer on` at turn 9 **does not find a bank MILTANK has already spent** — it finds a full one, and the 90 s `maxFirstTurn` allowance is granted on whatever turn the timer starts, because `isFirstRequest ( :167 , :326 )` is still true. The premise that the constraint arrives with unknown consumption already charged is **false**, and that makes the design easier rather than harder.

3. THE UNIT THAT SPENDS THE BANK IS A REQUEST, NOT A TURN. A post-KO replacement is its own request with its own 55 s window off the same bank — `updateTurn` returns at `:286` for a mid-turn request without clamping and without adding. So a game with KOs has more requests than turns, and "expected remaining decisions" must be estimated in **requests**. Charging is also quantised: `TICK_TIME = 5 ( :41 )` and each tick subtracts five whole seconds, so 12 s of thinking costs 15 s.

What MILTANK can and cannot observe about the clock

It can observe the bank exactly, and that was the surprise. `room-battle.ts:332` sends the player, privately, on **every request while the timer is on**:

```
|inactive|Time left: 55 sec this turn | 420 sec total | 90 sec grace
```

so `turnSecondsLeft` and `secondsLeft` are both **observed, not inferred**. Two parsing traps: the `total` field is `secondsLeft - grace ( :330-332 )`, so the true bank is `total + grace` and reading `total` alone under-reads by 90 s; and the grace field is simply absent once it is gone. `|inactive|Battle timer is ON: ( :237 )` and `|inactiveoff|Battle timer is now OFF. ( :257 )` bracket the on/off state, and both are room-level so both players see them.

But nothing reads any of it. `engine/mag_bot.js` handles **zero** `|inactive|` lines — grepped, not assumed. So the capability exists in the protocol and is absent from the bot, which is CLAUDE.md's signature failure shape verbatim. `engine/miltank.js` now exposes `bot.noteClock(line|{turnSec, totalSec, graceSec})` and counts its calls in `bot.clockStats().notes` ; **that counter is 0 in every run to date and will stay 0 until OPS wires the handler in `mag_bot.js` , which is not SEARCH's file.**

Therefore the adaptive rule is designed against the worst case and does not depend on the observation. With no observation it assumes the timer has been on since the first request and charges itself its own tick-rounded wall clock from a full 420 s. If the timer is actually off it has throttled for nothing — a weaker search, not a lost game. If the timer comes on at turn 9 the estimate has over-charged for turns that were free, so it under-spends. Both errors land on the safe side, which is the only property that function is allowed to have.

The shipped numbers against the walls

	shipped	the wall
in-game decision	<code>budgetMs</code> 20,000	55,000 ms , capped by whatever is left in the bank
team preview	<code>previewMs</code> 15,000	90,000 ms , and it comes out of the grace, so it is free
the whole game	21 decisions at 20 s	420,000 ms, no refill
post-KO replacement	no budget at all until 2026-08-04	its own 55 s request off the same bank

Per decision MILTANK is nowhere near the wall. The binding constraint on one turn is `budgetMs` , which we chose. **Per game it is genuinely tight**, and it is invisible in every H2H we run because `mew.js` has no clock and both arms are equally free — the same *testing environment ≠ playing environment* error CLAUDE.md names, one layer out (PRIORITIES #39a).

R6 — the per-decision wall-clock distribution, MEASURED 2026-08-04

Artifact: `data/miltank-timing-r6.json` , rows in `data/.miltank-timing/r6.jsonl` . 12 self-play games, `--policy score --policy2 score --miltank --miltank-n 200 --miltank-preview-n 40 --seed 90001` , **120 decisions across 11 recorded games**, the 7,264-team clean pool (announced by the run, **not** `--meta-teams`), one process, Ryzen 7 7735HS / node v24.15.0.

THE BUILD THIS DESCRIBES, and it is already PRE-CHANGE. `medicham2-browser.js` `b1b3ea94d5c3` , `rollout_leaf.js` `974c94d92398` , `board.js` `abeb747f3219` , `miltank.js` `2cdf6f5b0924` . The run window was 19:17:21–19:25:29 UTC; ENGINE wrote `data/abra-tags.js` at 19:32:34 and `medicham2-browser.js` at 19:33:47, so **the tree moved seven minutes after the last row**. All 11 per-game stamps carry one digest, so the run is internally consistent — the digest is taken at module *load*, not at the first row, precisely so it cannot describe a file edited underneath a running process. **A duration is a fact about a machine under a load and about a build.** This one is not R2's mistake (PRIORITIES #14) because it says which; it is nonetheless a distribution for a build that no longer exists, and the weather-boundary fix makes playouts longer, so **treat every figure below as a LOWER BOUND on the post-fix engine.**

	median	p90	p99	max	over the 55 s turn wall
all decisions (n=120)	2,169 ms	9,977 ms	23,812 ms	23,866 ms	o
in-game move (n=98)	2,234 ms	13,891 ms	23,866 ms	23,866 ms	o
team preview (n=11)	2,076 ms	4,041 ms	4,195 ms	4,195 ms	o — but censored, see below
post-KO replacement (n=11)	445 ms	607 ms	4,528 ms	4,528 ms	o

A single decision cannot breach the 55 s wall at shipped settings. The worst of 120 was 23.9 s against a 55 s cap, on menus running 2–49 joint options. The brief's worse case — one decision exceeding the per-turn wall — **does not occur**, and that is the good half of this result.

`budgetMs` **IS A CHECKPOINT, NOT A DEADLINE, and that is a real finding.** 9 of 98 move decisions finished *over* the configured 20,000 ms, by up to **3,866 ms**. The budget is tested between finalists (`miltank.js` , the `finalists` loop), so whichever finalist is in flight runs to completion past it. The effective per-decision cap is `budgetMs + one finalist evaluation` ≈ 24 s at n=200. It is comfortably inside 55 s today; it is not a bound anyone should quote as one.

Per game, against the 420 s bank:

	p50	p90	max	over the bank
requests per game	10	13	14	—
total spend per game	25.2 s	128.5 s	137.1 s	o of 11

The worst game observed spent 33% of the bank. To forfeit, a game would have to cost roughly three times the worst one measured.

How long a real game actually is — 30,396 non-forfeit ladder games

`node engine/miltank.js --horizon data/games.ladder.jsonl` , folded into the artifact. Counted in **requests** (turns plus turns in which one of our bodies fainted, because a replacement is its own request off the same bank):

	p50	p75	p90	p95	p99	max
requests per game	9	11	13	15	19	74

0.58% of games exceed 21 requests, 0.30% exceed 24, 0.08% exceed 34. And the self-play run's own request counts (p50 10, p90 13) reproduce the store's (p50 9, p90 13) almost exactly, so the harness is a fair — mildly conservative — proxy for game length.

The verdict, stated plainly because it is not the one this was expected to produce

The shipped flat `budgetMs: 20000` **is not currently a forfeit risk, and the measurement says so rather than the argument.** Two independent legs:

- *Observed*: 0 of 11 games came within $3\times$ of the bank; worst was 137 s of 420 s.
- *Worst case*: even charging the full 20 s to every request, only **0.58%** of real games have enough requests to empty the bank, and MILTANK actually spends a median of 2.2 s.

So the adaptive rule is **a guard against a 0.6% tail and against an engine that is about to get slower**, not a fix for a live bleeding problem. That is a weaker case than PRIORITIES #39a assumed and it should be said out loud before anyone spends 420 games on R6a. **What has NOT been retired is the environment mismatch itself** — nobody has ever watched MILTANK play with a clock running, and the two facts that make the flat constant survivable (games are short, the search is usually fast) are properties of *this* build on *this* pool.

Preview is censored here and is the one number this run cannot give you

`mew.js` hardcodes `previewMs: 4000` and SEARCH may not edit it. **One of 11 previews truncated at 16 of 90 brings in 4,195 ms**; the other ten completed 90/90 in 2.1–3.2 s. Extrapolating the censored one at its own ~262 ms per bring gives **~23.6 s for a full preview** — which would truncate against the shipped `previewMs: 15000` too, and is still far inside the **90 s** first-turn wall. So preview cost varies $\sim 9\times$ with the team, and **preview is** `previewMs` **-bound, not wall-bound.**

Given that preview time comes out of grace that `updateTurn` is about to clamp away regardless, `previewMs: 15000` **is spending 17% of a free 90 s allowance.** Raising it is an accuracy decision, not a clock one, and it belongs with the preview-calibration item (#38) rather than here.

Adaptive spend — implemented 2026-08-04, behind a flag, DEFAULT OFF

`engine/miltank.js` now carries the rule this file asked for. `clock: false` **is the default and the OFF path is byte-for-byte the player R4 measured**, so nothing changes in a live game without a deliberate decision.

The rule, in one line: **spend** `(bank - reserve) / expected remaining requests`, **clamped under the per-turn wall with a safety margin, floored so a starved tail still searches something.** Four properties it was given on purpose:

- **It can only ever lower the budget, never raise it.** `budgetFor()` takes `min(adaptive, configured)`. A rule that could also spend *more* would need accuracy evidence of its own; this one needs only to not be worse.
- **The reserve is asymmetric, because the failures are.** Overrunning the *turn* costs one server-chosen move (`Timeout Auto Choose`, :451-453); emptying the *bank* forfeits the game (:455). So the reserve sits on the bank (`clockReserveMs`, 45,000 — nine ticks) and the turn gets only a tick-quantisation margin (`clockSafetyMs`, 10,000 under the 55 s wall).

- **The horizon is a high quantile of requests per game, not the mean** (`clockHorizonRequests` , default **19** = the measured p99 above) — the same reason the distribution is being measured at all. It is also floored by `clockTailMin` (8), because past the horizon `EXPECT - requests` collapses to 1 and the throttle silently switches *itself off* exactly when the game has proved it is a long one. Caught by driving the clock through 40 requests, not by reading it.
- **It bounds the post-KO replacement search, which had no deadline whatsoever.** Five candidates at `2 × ROLLOUT_N` on a request that draws its own 55 s from the bank was the one decision nothing capped. Only enforced when the flag is on.

knob	default	meaning
<code>clock</code> / <code>MILTANK_CLOCK=1</code>	off	adaptive spend against the bank
<code>clockReserveMs</code>	45,000	bank held back; a bank timeout is a forfeit
<code>clockMinMs</code>	1,500	floor, so a starved tail still searches
<code>clockSafetyMs</code>	10,000	margin under the 55 s per-turn wall
<code>clockHorizonRequests</code>	19	planning horizon in requests — the p99 measured over 30,396 ladder games
<code>clockTailMin</code>	8	never plan for fewer than this many more requests, so the throttle survives past the horizon
<code>clockEarlyDefer</code> / <code>MILTANK_EARLY_DEFER=1</code>	off	stop before the finalist round on positions heading for the tie band
<code>timing</code> / <code>MILTANK_TIMING=<path></code>	off	write the per-decision wall-clock artifact

`clockEarlyDefer` **is not a timing lever and must not be argued as one.** MILTANK hands turns back to MAG *after* paying for the finalist round. **Measured in R6: 31.6% of move decisions deferred, and they consumed 30.5% of the total spend** — deferred and chosen decisions cost almost identically (4,639 ms against 4,585 ms mean), so the saving really is proportional and **30.5% is the ceiling on this lever**, not the "roughly a quarter" this file previously guessed. The screen already has an opinion about the spread; it is only noisier. The estimator subtracts the expected pure-dice range of K estimates of one true value ($\approx 2.8\sigma$ at the screen's `n`) from the observed screen spread and asks whether what is left clears the **final** round's tie band. **It changes what gets clicked on every turn where the screen and the finals would have disagreed, so it gets its own SPRT arm.** Its bias is stated rather than hidden: it is biased *low*, which makes it defer more often than it should.

Why an environment variable rather than a flag. The A/B has to run through `engine/mew.js` , which is MEASURE's file, and the live path is `engine/mag_bot.js` , which is OPS's. SEARCH cannot add `--miltank-clock` to either — the same one-liner PRIORITIES #33 already owes `--miltank-explore` . The env var is recorded in every timing row, so a result is still attributable to the lever. Replace it with a real flag when #33 lands; do not leave two ways to set one thing.

R6 — the validation. SPEC. NOT RUN.

It is three questions and only one of them is an H2H. Collapsing them is the mistake this spec exists to prevent.

R6a — the DIVERGENCE screen first, and it is probably where this stops

Do not open with the 420-game H2H. It is a null by construction and the probe says so before any games are played. Driving the clock through 40 requests at the horizon and reserve that ship:

what MILTANK wants per request	requests where adaptive ≠ flat	first divergence
4.6 s — the R6 measured mean	0 of 40	never
20 s — every request at the full budget	29 of 40	request 12

At the spend actually measured, **the throttle never engages at all:** $(420 - 45)/19 \approx 19.7$ s is already above what MILTANK asks for, and `budgetFor` only ever takes the smaller of the two. That is the design working — it is a guard, and a guard that fires when nothing is wrong is a bug — but it means an H2H between the two arms would compare **two identical players on ~99.4% of games** and return a null that says nothing about the rule.

So R6a is a divergence count, not an SPRT. Run ~40 instrumented games with `MILTANK_TIMING` set on both arms and count the decisions whose `budget` differs from `budgetMs`. Publish that count.

- **If divergence is ~0** — the expected outcome — the rule is inert in normal play, the flag stays off, and **there is nothing to SPRT.** That is the honest end of this item and it costs 40 games, not 420.
- **Only if divergence is material** does the H2H below become worth its cost.

R6a' — the H2H, conditional on R6a showing divergence (~420 decisive pairs)

Paired and seed-matched exactly as R4. **Arm 1 is the challenger:** `MILTANK_CLOCK=1`. Arm 2 is the shipped flat `budgetMs: 20000`. Everything else identical — `--miltank-n 200`, `explore 1.0`, `foe uniform`, turns 60, same seeds, same team pool, pool announcement recorded. `mew.js` having no clock is *correct* here: it isolates the search-quality cost of throttling from the forfeit benefit, which is R6b's job.

Read once, at the bound. `node engine/sprt.js <cat of shards>`.

One behaviour the H2H must be told to expect, or it will be read as a bug. When the bank falls under `clockSafetyMs + 2 ticks` the budget goes to zero, the screen cuts every pair, and MILTANK falls back to MAG for the rest of the game. That is deliberate — with 10 s of bank left, an instant imitation move is strictly better than a forfeit — and it will show up as a burst of fallbacks at the end of long games in the challenger arm only.

R6b — how often does the flat constant actually forfeit? (NOT an H2H) — ANSWERED

An H2H in `mew.js` structurally cannot see this, because `mew` has no clock, so both arms are free and the forfeit never happens in either. It is answered from the timing artifact instead, and it has been: `games_over_bank_pct` is **0.0** (0 of 11; worst game 137 s of 420 s), and only **0.58%** of 30,396 real games have enough requests to empty the bank even at the full 20 s every time. **This is the number that decides the item and it says the lever is a guard, not a fix.**

It must be re-read after the engine release, because the weather-boundary fix lengthens playouts and every figure here is a lower bound on the post-fix build. `node engine/miltank.js --reduce <rows> --horizon-store data/games.ladder.jsonl --out data/miltank-timing-r7.json`.

R6c — `clockEarlyDefer` (a separate H2H, never confounded with R6a')

Arm 1 `MILTANK_CLOCK=1 MILTANK_EARLY_DEFER=1`, arm 2 `MILTANK_CLOCK=1`. Same size, same pairing. Running it inside R6a' would make a play result unattributable between two levers, which is the failure "levers are per arm" exists to stop.

This is the one of the three worth running even though the clock does not bind, and the reason is not the clock at all: it buys **30.5% of the search budget back** on positions the search then throws away, and that time can be spent on `--rollout-n` instead. Its risk is bounded and stated — it can only change decisions that the finalist round would have deferred anyway or that the screen misranks. Size it at ~420 decisive pairs; the null it has to beat is "no worse".

The rule for all three

Run them after the Po.5 release boundary. Started before, they are born `PRE-CHANGE` and describe a build that stopped existing — already true of every R4 shard on disk. And every run must stamp `n_measured / n_unit`, the engine source digests, node version and machine: a duration is a fact about a machine under a

load, which is precisely why PRIORITIES #14 says R2 is re-run or nothing. Do not let R6 become a second R2.

The commands. PREPARED, NOT RUN.

Four processes, not six — RAM is the ceiling and `FreePhysicalMemory` was 2.33 GB when R6 ran, which is **one** process. Check it before choosing a number. `SHOWDOWN_PATH` is required.

R7 — re-measure the distribution on the named release (one process, ~10 min, 12 games):

```
SHOWDOWN_PATH=C:/Users/willj/Projects/Pokemon/pokemon-showdown \
MILTANK_TIMING=$PWD/data/.miltank-timing/r7.jsonl \
node --max-old-space-size=1536 engine/mew.js --n 12 --conc 1 \
  --policy score --policy2 score --miltank --miltank-n 200 --miltank-preview-n 40 \
  --seed 90001 --out data/.miltank-timing/r7-games.jsonl

node engine/miltank.js --reduce data/.miltank-timing/r7.jsonl \
  --horizon-store data/games.ladder.jsonl --out data/miltank-timing-r7.json
```

R6a — the divergence screen (same command, `MILTANK_CLOCK=1` added, shard 0..3 by `--seed`):

```
SHOWDOWN_PATH=... MILTANK_CLOCK=1 MILTANK_TIMING=$PWD/data/.miltank-timing/r6a-s<N>.jsonl \
node --max-old-space-size=1536 engine/mew.js --n 10 --conc 1 \
  --policy score --policy2 score --miltank --miltank-n 200 --miltank-preview-n 40 \
  --seed 9100<N> --out data/.miltank-timing/r6a-s<N>-games.jsonl
```

Cat the shards, then `--reduce` once, and count rows where `budget < budgetMs`. **Read it at the bound, once.**

R6c — the `clockEarlyDefer` H2H, only after R7 has re-stamped the leaf. Arm 1 is the challenger and carries `MILTANK_EARLY_DEFER=1`; both arms carry `MILTANK_CLOCK=1`. ~420 decisive pairs, sharded four ways, then `node engine/sprt.js <cat of shards>` **once**.

Open

6. `mag_bot.js` PARSES NO `|inactive|` LINE — FILED FOR OPS, NOT FIXED HERE

The server hands the player its exact bank on every request while the timer is on (`room-battle.ts:332`) and **the bot throws it away**: `grep engine/mag_bot.js` for `inactive` and it returns zero. `engine/miltank.js` now exposes `bot.noteClock(line)` and counts calls in `bot.clockStats().notes`; **that counter is 0 and stays 0 until something calls it**. Under CLAUDE.md the capability is therefore assumed broken.

It is one handler in the socket loop, and `mag_bot.js` is **OPS's file and the live path** — SEARCH must not touch it while Will may be playing. Two things whoever wires it must get right, both read out of the source rather than guessed:

- the bank is `total + grace`, not `total`. `:330-332` prints them apart and the grace field is absent once it is gone. Reading `total` alone under-reads the bank by 90 s at the start;
- `|inactive|Battle timer is ON: ( :237 )` and `|inactiveoff| ( :257 )` are **room-level**, so both players see them, but the `Time left: line` at `:332` is sent to **that player only** — do not try to read the opponent's clock off it.

Until then the adaptive rule runs on the worst-case self-charged estimate, which is deliberate and is strictly on the safe side, but it is an estimate.

o. ~~The in-game leaf and the preview leaf are different players~~ — CLOSED 2026-08-04

MEASURE's calibration (2026-08-04) ranked the **preview** leaf at 53.22% ($p < 1e-4$) and the **in-game** leaf at 50.99% ($p = 0.47$). Those were not two settings of one thing. `engine/miltank.js` held a **second, hand-rolled** **playout loop** — `battleInit` / `battleTurn` directly, deterministic greedy on both sides — that never called `rolloutWinProb`. MILTANK shipped two leaves and only one of them was ever swept.

There is now one playout. `rollout_leaf.runPlayout` is the single implementation; both leaf entry points call it, and the preview calls `rolloutWinProb` like everything else. What used to be a second implementation is now three parameters:

	preview before	preview now	why
playout policy	deterministic greedy, both sides	<code>explore</code> (default = the in-game 1.0)	one player, not two. <code>previewExplore: 0</code> restores greedy
opponent model	<code>chooseAction</code> , ignored -- <code>miltank-foe</code>	<code>foePolicy</code> — the flag now reaches preview	the foe A/B was silently not running at preview
game start	<code>battleInit({seeded:true})</code> on a game that had not started	<code>seeded:false</code>	see item 1 below
horizon	hardcoded 60	<code>turns</code>	it happened to agree; now it cannot drift
dice across brings	a fresh seed per bring	common random numbers , one seed for the whole preview	each bring was judged on its own independent draws, so the difference between two brings sat under the noise. Same fix, same reason, as <code>replSeed</code> in the post-KO search

This changes what the bot leads with, and the 53.22% no longer describes a policy that ships. That figure was measured on the greedy loop; it is now a number about a deleted implementation and must not be quoted for the shipped preview. Re-measuring it is MEASURE's, after the release boundary.

Two things this does NOT settle. The old 53.22%-vs-50.99% contrast was also measured on different position distributions (fresh full teams against mid-game boards), so it was confounded twice over and unifying removes only one of the confounds. And nothing has measured that preview *wants* `explore=1.0` — defaulting it there is a decision to have one player until something says otherwise.

ob. The preview enumerated brings it could not field — fixed the same pass

Caught by the smoke run for the unification, not by anything that was watching. The bring enumerator mixed **positions in the buildable list** with **team indices**: the lead pair came from one and the back pair from the other. They coincide when all six Pokemon build, which is why it survived. On a team with two unbuildable bodies, measured:

- 19 brings enumerated where exactly **6** exist
- **18 of the 19** named a Pokemon the search had just printed as unbuildable
- 15 "distinct" brings out of a true set of 6

and the missing body was then silently dropped by `.filter(Boolean)`, so a **three**-Pokemon bring was scored and reported as a four-Pokemon one. Full six-buildable teams enumerate 90 before and after, so this only ever bit the case the "drop what cannot be built" path exists for.

1. The preview seeded a game that had not started — fixed 2026-08-04

`chooseTeamPreview` called `battleInit(..., {seeded: true})`. `seeded` exists to stop a **mid-battle** leaf re-firing entry effects that already happened — re-running Intimidate would drop the same Attack a second time on every board with an Incineroar. At team preview nobody has entered yet, so it suppressed the entry effects entirely, and those are most of what a lead decision is. Deciding a lead is largely deciding who eats an Intimidate, and the search could not see one.

Measured directly against the engine, Torkoal + Incineroar leading into Garchomp + Gholdengo:

seeded=true	weather=null	weatherT=0	foe atk stages=[0, 0]
seeded=false	weather="sun"	weatherT=5	foe atk stages=[-1, -1]

So before the fix every preview playout ran with **no Drought, no Drizzle, no Sand Stream, no Snow Warning, no terrain setter and no Intimidate on turn one** — the whole switch-in-ability class, deleted, in the one decision they matter most for. Fixed by making `seeded` a parameter of `rolloutWinProb` (default unchanged at `true`) and passing `false` from preview only.

1b. Opponent model — the A/B in flight

Playouts move uniformly at random. Real Charizard clicks Protect 60.6% of the time, not 25%. `--miltank-foe prior` exists and is being compared against uniform; shards land in `data/.mew-shards/foe-s*.jsonl`.

If prior wins this changes every evaluation in the project, because every leaf number was computed against a foe that does not exist.

Counter-consideration, and it is not small: a fully random rollout has repeatedly judged *better* than a greedy one. Do not assume a more realistic playout is a better estimator — that is exactly what the A/B is for. Read it at the bound.

2. Which mega to take

Currently "the lead keeps it", which is arbitrary. It should be a search decision, and it is cheap to make one — only two-stone brings branch at all.

"Biggest stat gain" was **measured and discarded**: every Champions mega is +101 to +104. Do not re-propose it.

3. Team quality

`--meta-teams` yields 169 teams, but the base filter is **completeness, not quality** — so the pool contains Mickey Mouse teams: real, open-sheet, and still terrible. The pool is announced on every start, on or off. Read the announcement before attributing a result to a lever.

4. Leaf calibration blocks everything here

Every decision this division makes is an argmax over the leaf. If the leaf is uncalibrated, a better search is a better-aimed error. This is MEASURE's item, not SEARCH's — but SEARCH should know that a null result here may not be about the search at all.

5. `applyMegaWeather` — FIXED 2026-08-04, with #40b, and measured

`engine/rollout_leaf.js` called `applyMegaWeather(S)` and then assigned the caller's field over the top — `S.field.weather = f.weather || ''` — **unconditionally, one line later**. `battleInit({seeded:true})` leaves `S.field.weather` null, so the guard `if (S.field.weather)` `return` never fired, the function always ran, and its write was always discarded. Mega Charizard Y stood in clear weather in every mid-battle rollout this project ever ran.

Fixed by ORDER: the caller's field is applied first and `applyMegaWeather` second, so the guard arbitrates instead of being overwritten. A real weather the board reports still wins.

Two corrections to #40b as filed, both measured rather than reasoned.

- **The raw ability read was not returning a wrong answer on this path.** `dmgMon` already calls `effAbility` itself, so a Charizard + Charizardite Y body arrives at the function carrying `drought`, not `blaze`. Probed directly (`species=charizard item=Charizardite Y` → body ability `drought`; same for

Tyranitar→ sandstream , Abomasnow→ snowwarning). #40b is a **latent** hazard here — live the moment a caller omits dex — not the live defect the filing describes. #37 alone was the visible bug. Routed through effective() anyway, and rollout_leaf.js now holds 0 raw reads of a transforming field against a baseline of 0, down from 2 at bd8f388 .

- **The field the fix actually needed was mega , not the ability.** The discarded version had **no mega check at all** — it took the weather of the first ACTIVE with a weather-setting ability, mega or not. Landing #37 without adding that gate would have **invented weather**: a Torkoal standing on a board the tracker says is clear is a legal observed state, and re-setting its sun overrules the one thing that knows. The board is authoritative for a body that is what the board says it is; a mega is the sole exception, because dmgMon has already upgraded it to a forme the real game has not seen. **So #37 and #40b were coupled, but not for the reason filed.**

Parity, same boards, same seeds, both entry points, HEAD leaf against the working tree, in one process so the engine cannot differ between arms.

	before	after
boards walked	60	250
rolloutWinProb different	0	14
rolloutAfterActions different	0	14
boards moved on at least one	0	15
boards moved that are NOT a mega-setter-on-a-clear-board	0	0

15 of the 16 boards that hold a mega weather setter with no board weather moved; the 16th sits at 0.975/1.000 and is saturated at n=40. **Nothing else moved at all** — the reorder's only other side effect, weatherT no longer being left at 5 on boards that already have weather, is inert, and the parity proves it rather than assuming it.

Effect size, and it is not small: mean |Δ| **9.67 pt** on rolloutWinProb (max 17.5) and **18.33 pt** on rolloutAfterActions (max 62.5, n=24 and correspondingly noisier). The direction is the correctness evidence — **Charizard-Mega-Y's sun is worth +11.0 pt to the side that owns it and -12.5 pt to the side that faces it.** Tyranitar-Mega's sand moves ±5 pt, which is the right order for a weather whose main modelled effect is a Rock special-defence multiplier.

Direct counter, because a parity delta is indirect. battleTurn wrapped, field read on turn 1 of every payout, 250 boards × 40:

HEAD leaf	: 0 of 9,040	payouts began in a weather MEDICHAM can read	(0.00%)
working tree	: 640 of 9,040		(7.08%) sun 480, sand 160

Rate in the corpus sample: 32 of 250 boards (12.8%) hold a mega weather setter among the actives — Charizard-Mega-Y 17, Tyranitar-Mega 15 — and 16 of 250 (6.4%) are the ones with no board weather that #37 can move. **That is the honest exposure figure, and it is not the "~26% of format usage" the filing quoted:** 26% is megas in general, most of which set no weather.

Leaf cost: 17.53 → 16.93 ms per call at n=40 over 120 boards, i.e. no regression (one machine, one load — an order of magnitude, not a figure).

The unseeded preview path is untouched, and that was checked rather than argued: 12 preview-shaped calls (seeded:false , buildTeams , a mega setter on both sides) are identical across the two leaves.

The engine moved under this work. engine/medicham2-browser.js was committed and then modified again between the before-run and the after-run — 4 of the first 60 HEAD-leaf values differ across the two runs. The parity verdict survives it because both arms run in one process against one engine, and the after-run

carries its own before-state column measured on the current engine. The absolute numbers above describe the tree at 9a4f82d plus uncommitted ENGINE work, not a named release.

5c. THE SEARCH USED A THIRD TERRAIN VOCABULARY — FIXED 2026-08-04, and the effect is near zero

This is 5b's sibling, one layer further out, and it is the reason ENGINE's terrain fix measured **nothing**. ENGINE routed every terrain read in medicham2-browser.js through terrainId() and then counted that **0 of 863 terrain-carrying boards reach the leaf at all**. The defect was not in the engine: engine/miltank.js:794 and engine/rollout_r1.js:175 built the field object with

```
terrain: ['electric', 'grassy', 'misty', 'psychic'].find(t => board.hasField(t)) || ''
```

which are the **engine's** words probed against a Board that stores the dex's electricterrain, grassyterrain, mistyterrain, psychicterrain. **Three vocabularies, and the one doing the asking was the one nothing spoke**. Reproduced on an independent walk before the fix: the short-word probe matched **0 of 3,256** boards.

Fixed with rollout_leaf.terrainOnBoard(board) — **one implementation, no fourth map**. It probes the board's own four keys and translates with MEDI.terrainId, exactly as applyField already translates weather with MEDI.weatherId. Three call sites now use it, and the third was a hole nobody had filed: miltank.js 's **post-KO replacement search had** terrain: '' **hardcoded**, so every replacement was judged on a bare field. applyField also now runs f.terrain through terrainId, because that boundary is handed both vocabularies and terrainId is idempotent.

Two deliberate choices, both recorded so they are not re-litigated:

- **Probe four named keys rather than walking** board.pseudoWeather. Trick Room lives in the same namespace, so a walk would hand trickroom to terrainId and score a bogus MEDI.fails.terrainUnknown on nearly every board. A swallowed-failure counter that fires when nothing is wrong is a counter that gets ignored.
- engine/rollout_r3.js:98 **still passes** terrain: '' and is not SEARCH's file this pass. It is the same hole and it wants the same one-line call.

What it moved: NEAR ZERO, which is the expected honest answer

Both arms in one process against one engine, same boards, same seeds, so only the field object differs. 800 games, every 2nd board, n=40.

boards walked	3,256		
hits by the OLD short-word probe	0		
boards carrying a terrain	29 (0.891%; ENGINE's whole-corpus figure is 1.24%)		
which terrains	electric 19, psychic 10		
boards rolled out	205 — all 29 terrain boards, plus 176 terrain-free controls		
boards that moved	4		
controls that moved	0 of 176 — the control that says this is the terrain and nothing else		
mean \	Δ\	on the movers	8.75 pt (max 25.0)

All four movers are Psychic Terrain; not one of the 19 Electric Terrain boards moved. That is not a bug to hunt, it is the reader set: the engine consumes terrain in exactly four places — the Psychic Terrain priority block, Grassy Glide's priority, Hadron Engine (0 corpus uses), and `terrainScaled` (Expanding Force 182 uses, Rising Voltage 114). A 1.24% condition times a thin reader is a small number. **1.24% exposure and a 4/205 movement rate is the result; do not go looking for more.**

One thing tripped over and FILED FOR ENGINE, not fixed here. The engine models **no generic terrain type boost** — Electric Terrain does not multiply a grounded Electric move, Grassy Terrain does not multiply Grass, Psychic Terrain does not multiply Psychic, and Grassy Terrain's end-of-turn heal and Misty Terrain's status block are absent. Probed directly: a Psychic-type move into Garchomp reads `103-123` with no terrain and `103-123` under `psychic` and under `psychicterrain` alike. `terrainScaled` covers only the two moves whose handler names a number. Recorded here rather than in `docs/ENGINE.md` because an ENGINE agent owns that file right now — the same reason 5b was recorded here — so it is not lost. **It also bounds 5c:** the movement above will grow when the generic boost lands, and only then.

5b. THE LEAF'S WEATHER STRING HAS NEVER MEANT ANYTHING TO THE ENGINE — FILED, NOT FIXED

Tripped over while measuring #37, and it is **much larger than #37**. `applyField` assigns `f.weather` straight into `S.field.weather`. `f.weather` comes from `board.weather`, which is Showdown's `|-weather|` line normalised, so its values are **move names** — `sunnyday`, `raindance`, `sandstorm`, `snowscape`. `engine/medicham2-browser.js` compares against `sun / rain / sand / snow ( :464 , :486-487 , :934 )`. **They have never matched.**

Measured on the shipped engine, Charizard Flamethrower into Garchomp:

<code>weather=""</code>	61-72	<code>weather="sun"</code>	92-109	<code>weather="sunnyday"</code>	61-72
		<code>weather="rain"</code>	29-35	<code>weather="raindance"</code>	61-72

So the weather a mid-battle board reports is **truthy enough to suppress a guard and meaningless to every formula**. The turn-1 counter above says it exactly: **0 of 9,040 playouts on the HEAD leaf began in a weather MEDICHAM could read**, while 5,320 of them had a weather string. 152 of 250 sampled boards (60.8%) carry one.

Not fixed here, deliberately. Correcting it moves roughly 65% of in-game leaf values — an order of magnitude more than #37 — and landing it in the same pass would have made the #37 parity unreadable, which is the exact confound #37 was deferred to avoid in the first place. It is an ENGINE item; SEARCH could not write `docs/ENGINE.md` this pass and it is recorded here so it is not lost. The translation point is `rollout_leaf.js` `applyField`, which now carries the finding in a comment.

It also bounds what #37 bought. With the board's weather inert-but-truthy, the mega's weather is applied only where the board reports *no* weather at all. Under a real weather the mega still stands in nothing — because the board's weather is nothing too. Fixing 5b will move #37's exposure up.

R5 — the action-ranking backtest. SPEC. NOT RUN.

This is the measurement nobody has done, and it is the one that decides whether the rollout leaf is worth its 142 ms. Every leaf number produced so far — R1, the explore sweep, the calibration — scores a **predictor of a game outcome from a fixed position**. A search leaf is asked something different: *which of ~63 candidate joint actions is best*. Those are not the same job. **Two leaves with identical Brier can order actions completely differently**, and ordering is the only thing a search consumes.

The question, and what is NOT being asked

At a real decision point, does `rolloutWinProb` **order** the expressible joint actions differently from `materialP` on the post-action board?

Not "which is more accurate" — that is R1 and it is answered. **Do not size this as a superiority test.** At R1's effect size a decision-level superiority test needs roughly **190,000 decisions**; this run is ~2,000 and would be catastrophically underpowered for that question. Agreement needs no effect size, which is the whole reason this is the affordable measurement.

Procedure

1. **Sample.** Walk clean corpus games with the same `JR.build` walker `engine/rollout_r2.js` and `rollout_r3.js` use. Take every 3rd board so one long game cannot dominate. Target **~2,000 decision points**. Record the game id on every row — the analysis clusters on it.
2. **Enumerate the menu the search actually sees.** `board.js` `candidates()` per slot, then the *same* expressibility filter `miltank.js` applies: drop anything MEDICHAM resolves as `pass`, and drop a pure-status click whose every effect is refused. Offering a cell the engine cannot express makes distinct options collapse into one and the agreement rate becomes an artefact. Corpus median is ~8 options a slot, so ~63 joint cells.
3. **Score each cell twice, on the SAME dice.** One seed per decision point, shared across all cells — common random numbers, the same variance reduction the post-KO search already uses. Without it the disagreement set is mostly noise.
4. **rollout:** `RL.rolloutAfterActions(board, side, {n: 200, explore: 1.0, foePolicy: 'uniform', maxTurns: 60, myClicks, seed, dex, field})`.
5. **material:** step **one turn** with the same forced clicks and the same seed, then evaluate the material estimator on the **post-action** board. It must be the post-action board: scored on the pre-action board, material returns the same number for every cell and has no ordering at all, which would produce a meaningless 100% disagreement. Because the stepped turn is stochastic, average it over **m = 8** steps under the same seed and report `m`.
6. **Use the material estimator R1 was scored against — the existing one, by name, wherever it lives.** Do not write a second one. FACTS ARE GLOBAL; two implementations of "count the bodies" will disagree eventually and the disagreement will be invisible. Stamp which file it came from.
7. **Per decision point, record:** Spearman rho, Kendall tau-b (ties matter — a rollout at n=200 has granularity 1/200 and material is coarser still), whether the two argmaxes are the same **cell**, whether the rollout's argmax is inside material's top 3, and the rollout argmax's rank under material.
8. **Across decision points, report WITH INTERVALS — not a winner.** Mean rho and mean tau with a 95% CI, and the argmax-agreement rate with a 95% CI. **Bootstrap clustered on GAME, not on decision.** Decisions inside one game are not independent and treating them as such is the single easiest way to publish an interval three times too narrow here.

The decision rule, stated before the run

- **If argmax agreement is ~90% or higher and the lower CI bound holds above ~85%,** the two leaves choose the same action almost always and the 142 ms is **provably wasted as an action ranker**. That is a real, publishable, negative result and it redirects the division onto the leaf itself.
- **If they disagree materially, the disagreement set is the sample the H2H should then run on** — those positions and only those are where the two players differ, which is a far cheaper and far sharper H2H than 420 random games.

Cost, and why ~2,000 is the size

	per leaf call	per decision (~63 cells)	2,000 decisions
n=200, explore=1.0, maxTurns=60	141.85 ms	8.94 s	~5.0 h, one process
n=40 screen	37.48 ms	2.36 s	~1.3 h, one process

Both leaf costs are R2's, and R2 is itself under re-run (PRIORITIES #14) — treat them as the order of magnitude, not as the figure. A 40-rollout screen followed by n=200 on the survivors is the cheap version, but note it changes the question: it measures agreement on the *shortlist*, not on the menu. Run the full version if it can be afforded.

Running it

Four processes, not six — ENGINE is live and PRIORITIES sets the working cap at 4. Shard by game id modulo 4, one JSON row per decision point:

```
SHOWDOWN_PATH=C:/Users/willj/Projects/Pokemon/pokemon-showdown \
  SHARD=<0..3> SHARDS=4 N_DECISIONS=500 ROLLOUT_N=200 EXPLORE=1.0 MAXTURNS=60 \
  node --max-old-space-size=4096 engine/rollout_r5.js --out data/.r5-shards/r5-s<0..3>.jsonl
```

Cat the shards, then reduce once. **Read it at the bound, once.**

What the run must stamp on itself, or it is not evidence

n, explore, foePolicy, maxTurns, m, the engine source digest, the material estimator's file and function, the corpus id and the team-pool announcement, node version and machine, and n_measured / n_unit — PRIORITIES #20 records R1 and R4 missing exactly those two fields; do not make it three.

Run it after the Po.5 release boundary. Started before, it is born PRE-CHANGE and describes a build that stopped existing — which is already true of every R4 shard on disk.

Running a comparison

Levers are **per arm**, and **arm 1 is the challenger** — check winnerWeights before ever "fixing" an analyser that looks broken. SPRT-gate it and read it at the bound, never during.

Size the run to the question: an H2H decides in roughly 420 games, not 200,000.

Done looks like

- A gated, artifact-backed SPRT verdict against a **named engine release**, not against HEAD.
- Every arm's flags recorded in the run, so a result can be attributed to a lever without guessing.
- The opponent-model A/B read once, at the bound, and written to an artifact.

Where this is going

docs/MILTANK.md §3.1 explains why the current best-response player is exploitable by construction. An opponent-aware playout is the first step toward an equilibrium player; ship it only if the A/B says so.

R10 status note (the router, 2026-08-05, at the 3.41.0 close) — CLOSED by R11 below

The explore-sweep re-run was STOPPED MID-FLIGHT by Will's order to close the session — its A/B row dumps and per-release shards are committed as evidence, its final artifact was not regenerated, and data/rollout-r1-explore-sweep.json therefore REMAINS UNSAFE in provenance, named and not hidden.

Finishing it is the first SEARCH item next session. The R10 memo above was complete before the stop; nothing in it depends on the unfinished run.

Correction of fact, 2026-08-05. Two of the three sentences above were already false when they were written, and the reason is worth keeping: the run was ordered stopped, the ORDER did not stop the process. The `MAXTURNS=60` leg kept going after the session closed and finished at 02:27:41Z; `engine/rollout_explore_sweep.js` then ran at 02:27:42Z and wrote the final artifact. All of that landed in the working tree UNCOMMITTED, so the committed evidence — which is what the note was written against — still showed a half-finished run. **An order to a person is not a signal to a process.** A stopped session and a stopped run are different events and this file recorded the first as the second.

R11 — the explore-sweep re-run is FINISHED, at a release, and it PASSES OUTRIGHT. 2026-08-05.

`data/rollout-r1-explore-sweep.json` is no longer UNSAFE. `node engine/provenance.js` now reads it as "pinned to engine release 3932186b59ef — `engine/medicham2-browser.js` matches the frozen copy", verified by CONTENT and not by mtime. Of the two artifacts the dispatch named UNSAFE, this one is cleared; the other, `exploitability.json`, is self-declared void by its own generator (R8) and is a separate item that no re-run of this gate touches. **At the close of this session `exploitability.json` is the only UNSAFE artifact in the repository.**

A third, `conformance-baseline.json`, went UNSAFE during the session when ENGINE moved `engine/conformance.js` underneath it, and had cleared again before the session ended. Recorded because it is a live demonstration of the thing this whole exercise is about: **the UNSAFE list is a photograph of a moving tree, so read it once, at the end, and say when you read it.**

The verdict, on the arm MILTANK actually runs

Both arms out of **one process and one walk** (`DUMP0`), so the artifact's own `build_caveat` reads `SAME BUILD, BY CONSTRUCTION` rather than the cross-build hedge the 2026-08-04 version carried. There is no between-run window for ENGINE to land in — which is the defect that killed the first attempt at this pairing and is why `DUMP0` exists.

judge, 9,201 positions, n=40, horizon 20	2026-08-04 (pre-boundary)	R11, release 3932186b59ef
explore = 1.0 — the shipped default	67.971%	69.840%
explore = 0.5	67.58%	68.91%
explore = 0 (deterministic greedy)	65.721%	66.645%
material, porygon2 form	65.265%	65.265%
paired, 1.0 over greedy	+2.25 [1.31, 3.19]	+3.195 [2.237, 4.153]
lift over material	+2.706 [1.596, 3.817]	+4.576 [3.473, 5.678]
R1 gate	PASS_ON_BASELINE	PASS_OUTRIGHT

The gate upgraded, and the upgrade is the news. R1's threshold is PORYGON2's published +3.42 lift over the same baseline. The pre-boundary interval contained it; the post-boundary **lower bound clears it** (3.473 > 3.42), so the rollout now carries more than the learned model adds over counting bodies. `data/rollout-r1-explore1.json` was regenerated from the new rows and reads `PASS_OUTRIGHT`; `status.js` picks the R1 line up from that file.

Read the size honestly: the material column is **unchanged to three decimals**, exactly as it must be — it never touches the leaf. Every point of movement is in the payout, and both payout arms rose. The mega-weather and terrain fixes this file predicted would "move the point estimate and very unlikely move the sign" did precisely that, in the predicted direction.

The effect clears this run's own noise floor: split-half spread 0.941 to 1.913 points against an effect of 4.576.

The second horizon, on the NEWER release, agrees

data/rollout-r1-explore-sweep-h60-09acd3b404ef.txt , 4,586 positions at MAXTURNS=60 — the horizon the live leaf actually runs — quoted verbatim in the artifact: explore=1.0 **69.86%**, 0.5 68.80%, greedy 66.16%, material 64.24%, lift **+5.63 [4.06, 7.19]**, R1 PASSES OUTRIGHT . Two horizons, two samples, two releases, same verdict and the same ordering of arms.

How much does the release boundary actually matter? MEASURED, at zero cost

The two committed explore=1 dumps (6e43710396db and 3932186b59ef) walk the identical 9,201 positions with identical seeds and differ only in engine/medicham2-browser.js and data/tags.json — one ENGINE landing. Pairing them row for row (0 misaligned witnesses on all five):

what an ENGINE landing did to the leaf			
rows whose leaf value moved at all	1,882 of 9,201 (20.5%)		
mean \	Δ \	on the rows that moved	5.06 pt , max 35.0 pt
rows whose ≥ 0.5 CALL flipped	148 (1.61%)		
the headline accuracy	69.688% $\rightarrow$ 69.840%, +0.152 pt		

Per position the leaf is volatile; in aggregate it is stable. That is the quantitative case for the release boundary and against panic about it in the same table: one ENGINE landing moves a fifth of all positions by five points, so any *per-position* claim must name its release — and it moves the headline by a seventh of the interval's half-width, so the aggregate verdict is not being carried by which release it ran on. It is also the reason the paragraph below is a footnote rather than a retraction.

THE BAD NEWS: the run I was told to do was killed, and the artifact is on the PREVIOUS release

I was dispatched to measure against release 09acd3b404ef and the paired A/B on disk is stamped 3932186b59ef — the cut before it. Stated plainly rather than papered over:

- I launched exactly the run asked for (one process, RELEASE=09acd3b404ef , GAMES=2500 EVERY=2 N_LIST=40 EXPLORE_LIST=0,0.5,1 , both dumps). It started clean — release drift 0 of 23 , leaf self-check 73.3% sane — ran for **40 minutes and was killed**: exit 1, no stack, no stderr, no partial dump. Free physical memory fell **4.03 GB $\rightarrow$ 1.73 GB $\rightarrow$ 0.62 GB** across the run while four foreign node.exe processes (peak 3.46 GB and 1.36 GB) held the box. That is an OOM kill by memory pressure from concurrent work, not a defect in the gate.
- **I did not relaunch it.** At 0.62 GB free it would die the same way, and a second dead 40 minutes is not evidence. Handing the command over is the correct end of this task.
- **I did NOT restamp the artifact to 09acd3b404ef .** rollout_explore_sweep.js takes the release from the arm's own sidecar, which is REL.stamp() written by the process that rolled the playouts. Editing it to name a release the playouts never ran on would be the precise failure the release boundary exists to prevent, and it would have looked completely fine.

What it costs, bounded by the table above: 09acd3b404ef differs from 3932186b59ef in the simulator and tags.json — one landing's worth. The h60 leg is on 09acd3b404ef and lands within 0.02 pt of the h20 leg's explore=1.0 figure. So the expected cost of the misalignment is on the order of 0.15 points and the verdict is 4.576 with a 1.10-point half-width.

The command, when the box is quiet. One process. Check FreePhysicalMemory is above ~5 GB first; this needs ~2.5 GB resident for ~35 minutes and it is the only thing that should be running.

```

SHOWDOWN_PATH=C:/Users/willj/Projects/Pokemon/pokemon-showdown \
RELEASE=09acd3b404ef \
GAMES=2500 EVERY=2 N_LIST=40 EXPLORE_LIST=0,0.5,1 \
DUMP=rollout-r1-explore1-rows-09acd3b404ef.jsonl \
DUMP0=rollout-r1-greedy-rows-09acd3b404ef.jsonl \
node --max-old-space-size=4096 engine/rollout_r1.js \
> data/rollout-r1-explore-sweep-h20-09acd3b404ef.txt 2>&1

node engine/rollout_explore_sweep.js \
--greedy data/rollout-r1-greedy-rows-09acd3b404ef.jsonl \
--explore data/rollout-r1-explore1-rows-09acd3b404ef.jsonl \
data/rollout-r1-explore-sweep-h20-09acd3b404ef.txt \
data/rollout-r1-explore-sweep-h60-09acd3b404ef.txt

node engine/rollout_r1_artifact.js --print \
data/rollout-r1-explore1-rows-09acd3b404ef.jsonl > data/rollout-r1-explore1.json

```

rollout_r1.js refuses to start if the live tree has moved off the named release, so a stale `RELEASE=` fails loudly rather than quietly — the refusal text is on disk twice already (`rollout-r1-explore-sweep-h60-3932186b59ef.txt` , `rollout-r1-greedy-h20-6e43710396db.txt`) and both times it was the guard working.

Two defects the regeneration exposed. ONE FIXED, ONE FILED — and the fixed one was hiding.

Defect 1 — `rollout_r1_artifact.js` **hardcoded a sentence that** `status.js` **prints as fact. FIXED.**

`which_rollout_is_this.consequence` was a string constant reading *"The published +2.91 gate result cannot be recomputed from anything committed... on it R1 is UNDECIDED."* True of the 2026-08-04 greedy dump and of nothing since. `status.js:325-329` prints it **verbatim, directly under the gate line**, so regenerating the gate produced a handoff that read `PASS_OUTRIGHT ... +4.576 [3.473, 5.678]` and then, on the very next line, `R1 is UNDECIDED` .

It had been patched by HAND into `data/rollout-r1-explore1.json` **instead of in the generator.** That is why nobody saw it for a day: the screen was right, the generator was wrong, and the two only disagreed the moment somebody re-ran it. A hand edit inside a *generated artifact* is the failure CLAUDE.md names for the `<!-- GENERATED -->` blocks, one layer down, where there is no marker to warn you that you are editing an output. **The generated block is the only place this could surface, and it surfaced by being regenerated — which is the argument for regenerating things.**

The sentence is now derived from the sidecar's recorded `explore` and from the verdict this run computed, so it cannot go stale without the number going with it.

Also fixed in the same pass, because it is the same file and the same reader: the sidecar loader whitelisted `p_column` , `sweep` and `source_digests` and **filtered** `engine_release` **out**. It predates `engine_release.js` . So a release-aware run handed the gate its release id and the gate dropped it. `data/rollout-r1-explore1.json` now carries `engine_release: "3932186b59ef"` , `engine_release_cut` , `showdown_commit` and all 25 `source_digests` at the top level.

The order was: fix the generator, **then** regenerate. Doing it the other way round makes the artifact read stale against its own generator the instant it is written.

Defect 2 — `provenance.js` **DOES NOT SEE** `data/rollout-r1-explore1.json` **AT ALL. FILED.**

The R1 headline the handoff prints is **not in provenance's enumeration**. Confirmed by reading the full listing: `rollout-r1.json` , `rollout-r1-withdrawn-join.json` and `rollout-r1-explore-sweep.json` are all there and this one is absent. The mechanism is the one `rollout_r1_artifact.js` documents about itself — provenance credits a generator with an output when the filename **sits beside a write verb**, and this file is produced by `--print` redirected by the shell, so no generator names it.

So the gate now carries a release stamp that nothing checks. Fixing it means giving the generator a real `--out` that names the path in a `writeFileSync` — small, but it changes the generator's digest again and there is no test covering this file (`grep -r1 rollout_r1_artifact tests/` returns nothing), so it should land with one. **Not done here** — changing how an artifact is enumerated is not a thing to land at the end of a session on the back of a run that died.

The suite, run rather than assumed: 72 passed, 6 failed, and none of the six is SEARCH's `node tests/run-all.js` after the changes above. Named, because "known failure" is banned and a list without owners is a filing:

red	owner	what it is
<code>tests/test-degradation-budgets.js</code>	MEASURE	four NEW counters (<code>fit_policy.decisionsUnreadable</code> , <code>.coercedActions</code> , <code>fit_joint.turnsUnreadable</code> , <code>.coercedTurns</code>) have no declared ceiling. The refit landed them today. The six counters that DO have ceilings all pass
<code>tests/test-effective-identity.js</code>	ENGINE	<code>.ability</code> read directly in <code>tests/test-interaction-matrix.js</code> and <code>tests/test-paste.js</code>
<code>tests/test-no-silent-failure.js</code>	ENGINE + MEASURE (baseline drift)	new empty catches at <code>em_validation.js:64,257</code> , <code>engine_release.js:202</code> , <code>miltank.js:182,188,218</code> and three release tests. All of it is the release machinery landed today; it needs re-baselining with <code>--update</code> , which is a deliberate act and not mine to perform on another division's code
<code>tests/test-web-status.js</code>	WEB, caused by ENGINE	the board is older than <code>data/engine-diff.json</code> and <code>data/mechanics-census.json</code> , which ENGINE rewrote while the suite was running . It will go red again the next time ENGINE lands; rebuilding it now buys minutes
<code>engine/provenance.js</code>	R8	exits non-zero while ANY artifact is UNSAFE, and one is: <code>exploitability.json</code> , deliberately void by its own generator. It stays red until the WOBBUFFET re-run happens, and R9 says do not run that in its current shape. Red for a reason that is written down, owned and dated — which is not the same as filed
<code>engine/em_validation.js</code>	MEASURE	the EM censoring validation did not clear its own noise floor and records no source digests

A **seventh** went red between the suite finishing and this section being written: `tests/test-docs-current.js` (16 passed, 1 failed) on `docs/CLICK-CENSORING-FIX.md` — a document MEASURE created mid-session, carrying 3 untraceable figures — plus `docs/MODELS.md` moving 28 → 29. `docs/SEARCH.md` **is not among the ten documents it names**, which is the check I care about here and the reason it is quoted rather than summarised. That a guard's colour changed twice in one hour without anyone touching the guard is the same lesson as the UNSAFE list above.

I changed `engine/rollout_r1_artifact.js` **and** `data/rollout-r1-explore1.json` **and nothing else in** `engine/` . **No red test names either file**. I did not touch the six: four of them are being actively written by the two agents running beside me, and re-baselining another division's guard is how a guard stops meaning anything. This is a report with owners, not a filing — it says which agent each one is waiting on.

FILED FOR MEASURE — `node engine/status.js` **CRASHES. The handoff command is down.**

Found at 03:41Z, after the ledgers had been stamped, so `docs/SEARCH.md` 's generated block above is current at `_stamped 2026-08-05 03:39_` and this does not affect it.

```
TypeError: Cannot read properties of undefined (reading 'toFixed')
    at measure (engine/status.js:368:79)
```

`status.js:367-368` reads `data/partial-label-em.json` and formats `A.censoring_bias.toFixed(3)` / `A.noise_floor_oracle_spread.toFixed(3)` after guarding only on `em.regimes.amplified` **existing**. The artifact is brand new, still untracked, and was being written by MEASURE while this session ran; its `amplified` regime is present without those two keys, so the guard passes and the format throws.

Every one of the other ~40 reads in that function is written defensively —

``(cen.raw_protocol_arm`

in the block. **CLAUDE.md's own instruction is** `node engine/status.js` , **so a throw here takes the whole handoff down, including four ledgers that have nothing to do with EM.** It is a one-line guard and it belongs to MEASURE (`status.js` and `em_validation.js` are both theirs); SEARCH does not patch another division's file mid-result. `engine/em_validation.js` is red in the suite for a related reason — *"the artifact records no source digests"* — so the artifact is known to be incomplete and `status.js` is trusting it anyway.

What this does NOT say, since the caveat is the same one and has not moved

This is a verdict on a **JUDGE**. It says the `explore=1.0` payout scores a human position better than the greedy one and better than counting bodies. It does **not** say `explore=1.0` wins more games: `engine/mew.js` still exposes no `--miltank-explore` (PRIORITIES #33), R4 was itself run at 1.0 and cannot arbitrate its own setting, and R5 — whether the leaf ORDERS actions differently at all — is still unrun. Every decision MILTANK makes is an argmax over this leaf, and a leaf that judges 4.6 points better than material may still rank 63 candidate cells identically. **R5 is the measurement that decides whether any of this buys a click**, and it is now the top of SEARCH's queue.

DUSK SIZE GATE (#40) — TOO BIG, and the reach rate is the finding that outranks the size

Run 2026-08-06. Artifact: `data/dusk-size-gate.json` . Generator: `engine/dusk_size_gate.js` . **The threshold block was written to disk before any count was run**, and the generator refuses to write a gate whose declared threshold is missing. Pure store analysis — no engine module is loaded, so no release is stamped and none is needed.

Verdict: TOO BIG. A tablebase keyed on what DUSK actually claims to look up — the declared set on both sides, HP, status and boosts — is **1.6 x 10¹³ entries**. It becomes shippable only by throwing away the set, the status, the boosts and the field, at which point it is **1,575,300 entries / 36.1 MB packed** and knows nothing except two species and two HP bars.

The declared threshold, and where it came from

SHIPPABLE means a table the JavaScript bot loads at startup and holds in memory. Budget **64 MB resident** — six processes on this box against `--max-old-space-size=4096` gives ~680 MB a process, and a table that exists to *replace* search must not take more than ~10% of the process it helps — and **50 MB on disk**, which

is GitHub's per-file warning (100 MB is its hard reject) and is also what this repo already lives inside: the largest tracked file in `data/` is 44.9 MB. At 24 B/entry packed that is 2.7M entries; at the ~200 B/entry a plain JSON-into-a-Map actually costs in V8 — which is what every other data file here does — it is 335,000.

band	entries
SHIPPABLE	<= 335,000
SHIPPABLE_PACKED	335,000 - 2,700,000
TOO BIG	> 2,700,000

THE REACH RATE COMES FIRST, BECAUSE IT CAN KILL DUSK ON ITS OWN

Measured on **5,815 clean open-sheet Bo3 games** (6,022 clean of 9,191 stored; 207 have no raw log).

games reaching 1v1	955 — 16.42% , 95% CI [15.49, 17.40]
games reaching 2v1	2,671 — 45.93% [44.66, 47.22]
1v1 decision points	1,758 — 3.75% of all 46,909 decision points in the corpus
mean 1v1 decision points per game that reaches one	1.84
1v1s over after a single decision	563 — 58.95%
games with a 1v1 lasting more than one decision	392 — 6.74% of the corpus

Five games in six never reach 1v1 at all, and three in five of the ones that do are finished in a single turn. The positions DUSK exists to solve are 3.75% of the turns played, and the subset where more than one decision is taken is 6.74% of games. That is the ceiling on its value and it is independent of how big the table is.

But when a 1v1 does happen it is genuinely contested, so this is a volume problem and not a formality problem. The side ahead on HP at 1v1 entry goes on to win **65.71%** of the time [62.55, 68.73] — a third of these are won from behind, which is not what a decided position looks like. Only 8.48% of entries are lopsided (one side >=90% HP, the other <=20%). 9.01% of them end in a forfeit. The longest 1v1 in the corpus ran 29 turns.

Concentration — the coverage curve

Distinct **species pairs** at 1v1: **648** over 1,758 observed positions, from a pool of **177 distinct on-field formes**. Canonicalised as unordered pairs, because solving one simultaneous-move matrix returns both seats' equilibrium strategies at once — `engine/slowking/nash.py` already does that, so the halving is real rather than optimistic.

to cover	species pairs	species+item	species+full set
50%	111	163	220
80%	326	448	576
95%	561	712	840
99%	631	782	910
all observed	648	799	927

It is not concentrated. The most common 1v1 species pair is 2.97% of positions; 298 of the 648 pairs occur exactly once. There is no small head to cache.

Adding state — and why the observed distinct-position count must NOT be read as a table size

level	distinct observed	saturation	held-out hit rate
species pair	648	0.37	34.9%
+ item	799	0.45	16.5%

level	distinct observed	saturation	held-out hit rate
+ full declared set	927	0.53	1.9%
+ HP 10% buckets	1,383	0.79	0.11%
+ HP 5% buckets	1,467	0.83	0%
+ status	1,407	0.80	0%
+ boost stages	1,456	0.83	0%
+ field (weather/room/Tailwind)	1,519	0.86	0%

Sets collapsed to species, same axes: 1,362 / 1,389 / 1,445 / 1,515, held-out 1.25% / 0.68% / 0.57% / 0.23%.

Two things have to be read together here, and reading either alone gives the wrong answer.

The distinct counts look tiny and they are meaningless. saturation is distinct / observed. At 0.86 the count is measuring the size of the corpus, not the size of the state space — 1,519 distinct positions out of 1,758 observed is a near-bijection, and adding data would add positions roughly one for one. **Anyone quoting "1,456 entries, ships as JSON" is quoting the sample size.**

The held-out hit rate says a memo of observed positions cannot work. Build the key set from the older half of the 1v1 positions, test on the newer half: once HP is in the key the newer half lands on a key you already have **0.11% of the time**, and past that, never. Even at bare species-pair level it is 34.9%. **A shippable DUSK has to ENUMERATE a matchup pool. It cannot memoise a corpus.**

The enumerated designs — where the wall is

entries = matchups x (HP buckets x statuses x boost states)^2 x field states . HP is bucketed at **10% of max**: a damage roll in this game spans 85-100% of its own mean, so consecutive rolls of one move already smear a target across ~15% of a bar, and a bucket finer than 10% is finer than the resolution of the thing being reasoned about. 5% is reported beside it so the tradeoff is visible.

design	matchups	entries	MB packed	band
A1 species pool, all pairs, HP 25%	15,753	252,048	5.8	SHIPPABLE
A2 species pool, all pairs, HP 20%	15,753	393,825	9.0	SHIPPABLE_PACKED
A3 species pool, all pairs, HP 10%	15,753	1,575,300	36.1	SHIPPABLE_PACKED
A4 species pool, all pairs, HP 5%	15,753	6,301,200	144	TOO BIG
B1 A3 + status restricted to none/brn/par	15,753	14,177,700	325	TOO BIG
B2 A3 + all 7 statuses	15,753	77,189,700	1,767	TOO BIG
B3 A3 + statuses + boosts covering 95% of sides	15,753	6.25e11	14 TB	TOO BIG
B5 A3 + statuses + full analytic boosts (13^5/side)	15,753	1.06e19	—	TOO BIG
C1 SET pool, all pairs, HP 10% only	392,055	39,205,500	897	TOO BIG
C3 SET pool, all pairs, HP 10% + status + boosts 95%	392,055	1.56e13	—	TOO BIG (headline)
C4 top-3 sets per species, all pairs, HP 10% only	141,246	14,124,600	323	TOO BIG
D1 only the 95%-coverage species pairs, HP 10% + status	561	2,748,900	62.9	TOO BIG (just)

The set axis is what breaks it, and that is the painful part. Open team sheets are DUSK's whole premise — the set is declared, so the game is perfect-information. Measured at 1v1 there are **885 distinct species+set sides over 177 species, 5.0 sets per species**, which turns 15,753 matchups into **392,055**. Paying for the information OTS gives you is a **25x** multiplier on the matchup axis before a single HP bucket is added, and C1 — set pool with nothing but HP — is already 897 MB.

The boost axis is what makes it hopeless. 66.7% of 1v1 sides are at neutral stages, but 213 distinct boost vectors occur and 90 are needed to cover 95% of sides. As a squared multiplier that is 8,100x. The full analytic space (13^5 per side) is 25,990,510 states per side.

So the only shippable table is A3: 177 species x 177 species x HP bucket, and nothing else. It cannot tell a Choice Scarf Garchomp from a Life Orb one, cannot see a burn, cannot see a Swords Dance. Every one of those decides a 1v1.

2v1 is not a fallback

3,644 2v1 decision points, 2,476 distinct species configurations observed (saturation 0.68), from a pool of 269 formes. One mon against an unordered pair is $S \times S(S+1)/2 = 9,768,735$ matchups, and the mirror halving does not apply because the two seats are structurally different. The matchup axis alone, before any HP or status, is six times the entire shippable 1v1 table. Coverage is worse than 1v1's: 695 configurations for 50%, 2,294 for 95%.

How this was measured, and what is wrong with it

Two independent reconstructions, reported against each other rather than one asserted. The stored turn events from `engine/durable-ingest.js` and the raw Showdown protocol were each replayed and asked "did this game reach 1v1": **98.55% agreement** (900 both yes, 4,860 both no, 55 protocol-only, 30 events-only). Headline figures come from the protocol, because the stored events are lossy in three ways that all inflate the position count — a spread move overwrites its own target field so `tgthp` names the last victim, `|-curestatus|` is not captured so a status never heals, and `|-weather|none` is filtered so weather never ends.

The protocol replay reads a 1v1 off the field and checks rather than assumes that it may: in doubles both slots stay filled while a bench exists, so at ≤ 2 alive every survivor is on the field. **Anomalies where the faint count and the field disagreed: 0 of 46,909 positions.**

Stated limitations, the first of which flatters DUSK: sheets carry `evs: null` in this corpus, so two identical declared sets on different Champions SP spreads collapse to one entry — **the set axis is larger than measured, not smaller.** Illusion is corrected at `|replace|` and not retroactively. "Already decided" is a proxy — no equilibrium solver was run over these positions.

Corpus snapshot, taken before the counts and re-checked after: `data/games.bo3.jsonl` 9,191 lines, sha256 `cd4d6ffea1b8fbf5...` ; `data/games.bo3.raw-logs.jsonl` 9,059 lines, sha256 `ee48e579d7f4266d...` . **It did not move during the run.**

What this means for #41, the Python-to-JavaScript bridge

DUSK was the route that let `engine/slowking/nash.py` reach the JavaScript bot without a second implementation of the solver. **That justification is gone: the precomputed table is TOO BIG at every fidelity that would make it worth having.** #41 should not be argued for on DUSK's back.

The measurement points somewhere better, and this is SEARCH's call rather than MEASURE's to make. **A 1v1 under open sheets does not need a table — it needs one solve.** With one mon a side there is no switch and no target choice, so the action set is exactly the ≤ 4 declared moves: a 4x4 simultaneous-move matrix with known payoffs, which is the smallest thing `nash.py` solves. There are 1,758 such decision points in 5,815 games, **3.75% of all turns**, so an online solve is paid for on one turn in twenty-seven. The size question becomes a latency question, and latency is an R-rung measurement this gate did not run and should not guess at — `data/rollout-cost.json` is the file that would have to answer it, and MEASURE has already recorded that R2 is re-run or it is nothing.

Two further consequences worth stating plainly:

- **The turns DUSK would accelerate are the turns MILTANK's search is already cheapest on.** At 1v1 the candidate set is at its smallest for exactly the reason that makes the matrix small. A lookup table buys the least time precisely where it is affordable.
- **If the bridge is built as an online solver it needs no corpus at all**, so none of the coverage or held-out numbers above constrain it. They only constrain the table.